

TSOpt walkthrough: every function, from data upload to training set

A step-by-step check of the whole package on your own data, in every breeding system

PlantNura Consulting & Solutions

30 September 2026 · TSOpt 0.5.0

Contents

How to use this document	3
1 Set-up	4
2 Uploading genotype data	5
2.1 Your file	5
2.2 Every format TSOpt reads	5
2.3 Writing and converting	6
2.4 Unusual inputs	6
3 Quality control	8
3.1 Allele orientation across breeding cycles	9
4 Relationship matrices	10
4.1 Additive (VanRaden)	10
4.2 Dominance, weighted and pedigree kernels	11
4.3 Epistatic, non-parametric and single-step kernels	12
5 The data object and the breeding systems	14
5.1 Inbred lines (wheat, rice, barley, soybean)	14
5.2 Clonal crops: diploid (cassava, banana) and tetraploid (potato)	15
5.3 Outcrossing crops (ryegrass, alfalfa, oil palm): families	15
5.4 Hybrids (maize, sorghum, rice hybrids)	15
5.5 Genebank or diversity panel	15
5.6 Pedigree or kinship only	15
5.7 Multi-environment and multi-trait data	16
6 Is optimisation worth it here?	17
6.1 tso_diagnose()	17
6.2 tso_sample_size(): how many lines?	19
7 Every criterion	20
7.1 Your own criterion, and the direction check	23
8 Every search	25
9 Building training sets: the core options	27
9.1 The default	27

9.2	Targeted	29
9.3	Heritability: known value or your own prior grid	30
9.4	A budget with unit costs	30
9.5	Field reality: checks, lost seed and last season's data	30
9.6	Quotas, caps and replicates	31
9.7	Fixed effects: trials, blocks, covariates	31
9.8	Genetic model: total or breeding value	31
10	Training sets for every crop and breeding system	33
10.1	Wheat, rice, barley: inbred lines, several goals	33
10.2	Maize, sorghum: hybrids	33
10.3	Potato (tetraploid) and cassava (diploid): clones	34
10.4	Ryegrass, alfalfa: outcrossing families (CDpop)	35
10.5	Genebank core collection	35
10.6	Rice, multi-location: sparse multi-environment testing	35
10.7	Wheat quality: multi-trait index (which trait on which line)	37
10.8	Very large panels	37
11	Major genes and haplotypes	39
11.1	Scanning your phenotypes	39
11.2	A trait with a planted major gene	39
11.3	Marker effects you already have	40
11.4	Protecting the alleles in the training set	41
11.5	Genome-wide allele and haplotype coverage	42
11.6	Haplotype blocks, the haplotype kernel and haplotype coverage	42
12	Checking a plan	44
12.1	Why this set: <code>tso_explain()</code>	44
12.2	Any set, any criterion: <code>tso_score()</code>	44
12.3	Against the alternatives: <code>tso_compare()</code>	44
12.4	Top-k identification: <code>tso_topk()</code>	45
12.5	Accuracy against diversity: <code>tso_pareto()</code>	46
13	After the season: evaluate the plan	48
13.1	Real accuracy of every criterion	49
13.2	Unreliable heritability estimates	50
14	Across stages, seasons and generations	51
14.1	Second stage: <code>tso_next_batch()</code>	51
14.2	Next season: <code>tso_update()</code>	51
14.3	Choosing parents: <code>tso_contribution()</code>	52
14.4	The long term: <code>tso_simulate()</code>	52
15	Decisions around the design	54
15.1	Traits and environments together, hedged over uncertain correlations	54
15.2	What is each trait worth?	54
15.3	The value of each plot	55
15.4	How sure is the advantage? <code>tso_uncertainty()</code>	56
15.5	How many plots pay off? <code>tso_economics()</code>	57
15.6	Laying out the field: <code>tso_layout()</code>	58
15.7	Whom to genotype, and hedging over the model of gene action	59
15.8	Training in one programme, predicting another: <code>tso_transfer()</code>	60
15.9	Straight from a breeding database (BrAPI)	60
16	Programme planning and hand-over	61
16.1	Variance components from your phenotypes: <code>tso_variance()</code>	61
16.2	Forward validation: <code>tso_backtest()</code>	61

16.3 The season across stages: <code>tso_season()</code>	62
16.4 Which crosses to make: <code>tso_crosses()</code>	62
16.5 Genotyping, imputation and plots: <code>tso_imputation_plan()</code>	63
16.6 Testcrosses and the factorised form on your panel	63
16.7 Omics profiles: <code>tso_omics_kernel()</code> and <code>tso_omics_plan()</code>	63
16.8 To the field and the database	64
17 Figures, export and reports	65
17.1 Every plan figure	65
17.2 Journal-ready files, the field list and the report	66
18 Robustness: inputs that should fail clearly	68
19 Scorecard	69

How to use this document

This document runs **every exported function of TSOpt** in the order a breeder uses them: upload the genotype file, check its quality, build the relationship matrices, pack everything into one data object, ask whether optimisation is worth it, try every criterion and every search, build training sets for every breeding system, protect major genes, check the plans, and plan the next season. Each step prints its result and a short explanation of what to look for.

Run it on your own data. The document has parameters. In RStudio, open it and choose *Knit > Knit with Parameters...*, then fill in the paths to your files. Or render it from the console:

```
rmarkdown::render("docs/walkthrough/TSOpt_Walkthrough.Rmd",
  params = list(geno = "my_panel.vcf.gz",      # any format tso_read() reads
               pheno = "my_pheno.csv",        # columns: id, y
               groups = "my_groups.csv",      # columns: id, group (optional)
               costs = "my_costs.csv",        # columns: id, cost (optional)
               effects = "my_effects.csv",    # marker, effect[, allele, p]
               ↪ (optional)
               history = "my_history.csv",    # id, cycle, trait: forward
               ↪ validation (optional)
               cm_per_mb = 0.5,               # map density, for cross
               ↪ prediction (optional)
               omics = "my_profiles.csv",     # id, one column per
               ↪ feature/wavelength (optional)
               n_train = 100, h2 = 0.4))
```

Every parameter left empty falls back to the example data shipped with TSOpt: a simulated panel of 300 inbred lines and 1,000 SNPs, with phenotypes, true genetic values, groups and per-line costs. Because the example is simulated, the true genetic values are known, so Section 12 can measure how good a training set really was.

Your licence key must be active (`tso_licence("TSO1....")` once on this computer; `tso_licence()` shows it). TSOpt is free for everyone; request a key with `tso_licence_request()`.

Run it one chunk at a time. Put the cursor in a chunk and press Ctrl/Cmd + Shift + Enter. The chunks depend on each other in order, so run them from the top.

Spot anything odd. After almost every function there is an automatic check, printed as [PASS] or [FAIL]. It tests something that must hold if the function worked: the relationship matrix is symmetric, no target line ends up in its own training set, a budget is respected, and so on. Section 16 collects every check into a scorecard. A [FAIL] is either a bug in TSOpt or a problem with the input data, and the check's text says which quantity to look at.

1 Set-up

```
library(TSOpt)
library(ggplot2)
packageVersion("TSOpt")
#> [1] '0.5.0'

ex <- function(f) system.file("extdata", f, package = "TSOpt") # shipped example files
pick <- function(p, default) if (nzchar(p)) p else default
geno_file <- pick(params$geno, ex("example_panel.vcf.gz"))
pheno_file <- pick(params$pheno, ex("example_pheno.csv"))
truth_file <- pick(params$truth, if (nzchar(params$geno)) "" else
  ↪ ex("example_true_values.csv"))
group_file <- pick(params$groups, if (nzchar(params$geno)) "" else
  ↪ ex("example_groups.csv"))
cost_file <- pick(params$costs, if (nzchar(params$geno)) "" else ex("example_costs.csv"))
eff_file <- pick(params$effects, if (nzchar(params$geno)) "" else
  ↪ ex("example_marker_effects.csv"))
using_example <- !nzchar(params$geno)
out <- file.path(tempdir(), "tsopt_walkthrough"); dir.create(out, showWarnings = FALSE)
cat("Genotypes:", geno_file, "\nPhenotypes:", pheno_file,
    "\nUsing the shipped example data:", using_example, "\n")
#> Genotypes: /Library/Frameworks/R.framework/Versions/4.5-
  ↪ arm64/Resources/library/TSOpt/extdata/example_panel.vcf.gz
#> Phenotypes: /Library/Frameworks/R.framework/Versions/4.5-
  ↪ arm64/Resources/library/TSOpt/extdata/example_pheno.csv
#> Using the shipped example data: TRUE
```

The scorecard helper records every check:

```
checks <- data.frame(section = character(), check = character(), result = character())
section <- "Set-up"
chk <- function(what, ok, detail = "") {
  ok <- isTRUE(ok)
  checks[nrow(checks) + 1, ] <- list(section, what, if (ok) "PASS" else "FAIL")
  cat(if (ok) "[PASS] " else "[FAIL] ", what, if (nzchar(detail)) paste0(" (", detail,
    ↪ ")"), "\n",
    sep = "")
  invisible(ok)
}
timed <- function(expr) { t0 <- proc.time()[["elapsed"]]; v <- expr
  attr(v, "seconds") <- proc.time()[["elapsed"]] - t0; v }
```

2 Uploading genotype data

2.1 Your file

`tso_read()` detects the file format, the genotype coding (0/1/2, -1/0/1, letters, IUPAC, AB calls, dosages), the missing-value codes, the ploidy and whether markers are in rows or columns. It records every decision in a log.

```
g_raw <- tso_read(geno_file)
g_raw
#> <tso_geno> 300 lines x 1,000 markers · ploidy 2 · from VCF
#> Read example_panel.vcf.gz (VCF) -> 300 lines x 1,000 markers
#> Coding VCF GT -> ALT-allele count 0..2 · ploidy 2 (detected)
#> Missing 0.0%
```

What to look for: the number of lines and markers must match what your genotyping provider delivered, the detected coding must be right, and the missing rate should be plausible (a few per cent for arrays, often more for GBS).

```
D <- g_raw$dosage
chk("dosage matrix has lines in rows and markers in columns", nrow(D) > 1 && ncol(D) > 1,
    paste(nrow(D), "lines x", ncol(D), "markers"))
#> [PASS] dosage matrix has lines in rows and markers in columns (300 lines x 1000 markers)
chk("line IDs are unique", !anyDuplicated(rownames(D)))
#> [PASS] line IDs are unique
chk("marker IDs are unique", !anyDuplicated(colnames(D)))
#> [PASS] marker IDs are unique
chk("dosages lie between 0 and the ploidy", all(D >= 0 & D <= g_raw$ploidy, na.rm = TRUE),
    paste("ploidy", g_raw$ploidy))
#> [PASS] dosages lie between 0 and the ploidy (ploidy 2)
chk("missing rate below 50%", mean(is.na(D)) < 0.5, paste0(round(100 * mean(is.na(D)), 2),
    "% missing"))
#> [PASS] missing rate below 50% (0% missing)
```

2.2 Every format TSOpt reads

The shipped example panel exists in five formats. Reading all of them must give the same lines and markers, and the same relationship matrix (allele coding may flip between formats, but relationships must not change).

```
fmt <- list(VCF = ex("example_panel.vcf.gz"),
            HapMap = ex("example_panel.hmp.txt.gz"),
            CSV = ex("example_panel.csv.gz"),
            PLINK = sub("\\.bed$", "", ex("example_plink.bed")))
gs <- lapply(fmt, tso_read, verbose = FALSE)
data.frame(format = names(gs), lines = sapply(gs, function(x) nrow(x$dosage)),
            markers = sapply(gs, function(x) ncol(x$dosage)),
            coding = sapply(gs, function(x) x$format), row.names = NULL)
#> format lines markers coding
#> 1 VCF 300 1000 VCF
#> 2 HapMap 300 1000 HapMap
#> 3 CSV 300 1000 delimited table
#> 4 PLINK 300 1000 PLINK binary
K_by_format <- lapply(gs, tso_kinship, scale = FALSE)
diffs <- sapply(K_by_format[-1], function(K) max(abs(K - K_by_format$VCF)))
round(diffs, 12)
#> HapMap CSV PLINK
#> 0 0 0
chk("VCF, HapMap, CSV and PLINK give the same relationship matrix", all(diffs < 1e-8))
#> [PASS] VCF, HapMap, CSV and PLINK give the same relationship matrix
```

```
g_dart <- tso_read(ex("example_dart.csv"), verbose = FALSE)
chk("DARTseq one-row file is read", nrow(g_dart$dosage) > 0,
    paste(nrow(g_dart$dosage), "lines x", ncol(g_dart$dosage), "markers"))
#> [PASS] DARTseq one-row file is read (60 lines x 200 markers)
```

2.3 Writing and converting

`tso_write()` converts between formats; the extension chooses the format. Every written file is read back and must give the same relationship matrix as the original.

```
exts <- c(".vcf", ".vcf.gz", ".hmp.txt", ".bed", ".csv", ".raw", ".rds")
K0 <- tso_kinship(gs$VCF, scale = FALSE)
rt <- sapply(exts, function(e) {
  f <- file.path(out, paste0("roundtrip", e))
  tso_write(gs$VCF, f)
  back <- tso_read(if (e == ".bed") sub("\\.bed$", "", f) else f, verbose = FALSE)
  max(abs(tso_kinship(back, scale = FALSE) - K0))
})
round(rt, 12)
#>      .vcf .vcf.gz .hmp.txt .bed .csv .raw .rds
#>      0      0      0      0      0      0      0
chk("every writable format round-trips to the same kinship", all(rt < 1e-8))
#> [PASS] every writable format round-trips to the same kinship
if (requireNamespace("writexl", quietly = TRUE) && requireNamespace("readxl", quietly =
  TRUE)) {
  xl <- file.path(out, "panel.xlsx")
  writexl::write_xlsx(data.frame(id = rownames(gs$VCF$dosage), gs$VCF$dosage, check.names
    = FALSE), xl)
  g_xl <- tso_read(xl, verbose = FALSE)
  chk("an Excel sheet is read", identical(dim(g_xl$dosage), dim(gs$VCF$dosage)))
}
#> [PASS] an Excel sheet is read
```

2.4 Unusual inputs

Real files are messy. These small inputs test the codings breeders meet most often: allele letters in several notations, a matrix with markers in rows, Axiom-style -1 for missing, and an autotetraploid dosage matrix.

```
ids5 <- paste0("L", 1:5)
calls <- list(pairs = c("AA", "AG", "GG", "AA", "NN"), slash = c("A/A", "A/G", "G/G",
  "A/A", "./."),
  iupac = c("A", "R", "G", "A", "N"), ab = c("AA", "AB", "BB", "AA", "--"))
dos <- sapply(calls, function(v) as.vector(tso_read(matrix(v, 5, 1, dimnames = list(ids5,
  "m1"))),
  markers_in = "columns", verbose =
    FALSE)$dosage))

dos
#>      pairs slash iupac ab
#> [1,]      0      0      0  0
#> [2,]      1      1      1  1
#> [3,]      2      2      2  2
#> [4,]      0      0      0  0
#> [5,]     NA     NA     NA NA
chk("allele pairs, slashes, IUPAC and AB calls give the same dosages",
    all(apply(dos, 1, function(r) length(unique(r)) == 1), na.rm = TRUE))
#> [PASS] allele pairs, slashes, IUPAC and AB calls give the same dosages
Xs <- gs$VCF$dosage[1:20, 1:40]
rows_df <- data.frame(marker = colnames(Xs), t(Xs), check.names = FALSE)
```

```
chk("markers in rows are transposed automatically",
    identical(rownames(tso_read(rows_df, verbose = FALSE)$dosage), rownames(Xs)))
#> [PASS] markers in rows are transposed automatically
ax <- Xs - 1; ax[1, 1] <- -9
chk("-1/0/1 coding is recognised", isTRUE(all.equal(unname(tso_read(Xs - 1, verbose =
  ↪ FALSE)$dosage), unname(Xs))))
#> [PASS] -1/0/1 coding is recognised
set.seed(4)
tet <- matrix(sample(0:4, 30 * 50, TRUE), 30, 50, dimnames = list(paste0("C", 1:30),
  ↪ paste0("m", 1:50)))
chk("0-4 dosages are read as an autotetraploid", tso_read(tet, verbose = FALSE)$ploidy ==
  ↪ 4)
#> [PASS] 0-4 dosages are read as an autotetraploid
```

3 Quality control

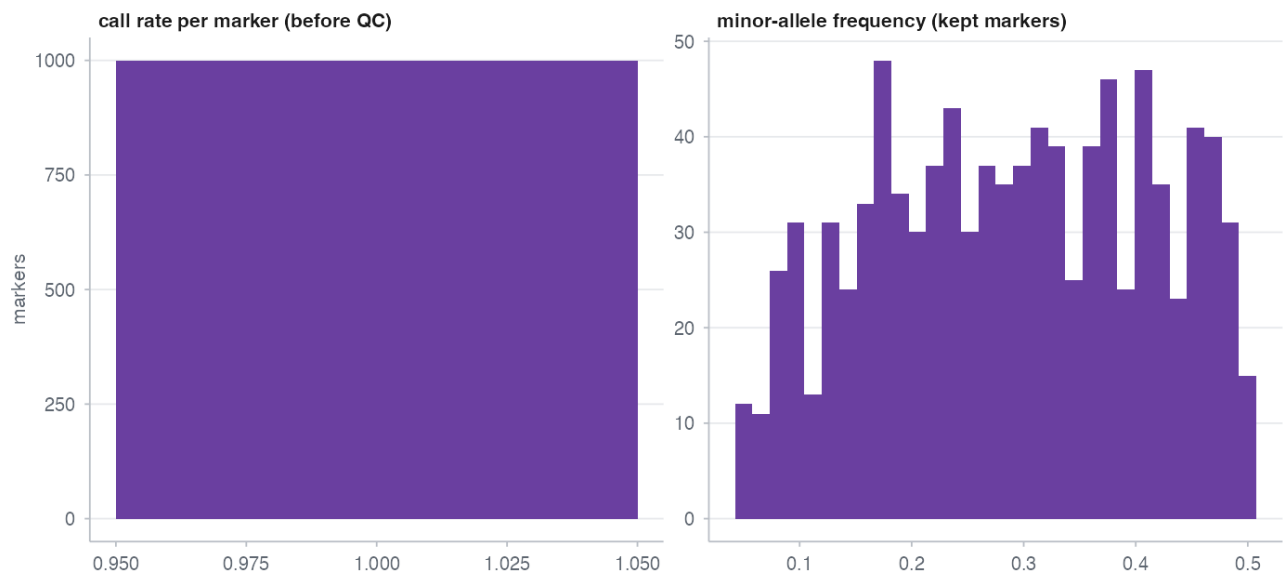
`tso_qc()` removes badly called lines, then markers with a low call rate or a low minor-allele frequency, imputes what is left missing, and stores the base allele frequencies. Every step goes into the log.

```
g <- tso_qc(g_raw)

chk("QC keeps at least half of the markers", ncol(g$dosage) >= 0.5 * ncol(g_raw$dosage),
    paste(ncol(g$dosage), "of", ncol(g_raw$dosage)))
#> [PASS] QC keeps at least half of the markers (958 of 1000)
chk("no missing values left after imputation", !anyNA(g$dosage))
#> [PASS] no missing values left after imputation
mafs <- pmin(g$base_freq, 1 - g$base_freq)
chk("every kept marker has MAF >= 0.05", all(mafs >= 0.05 - 1e-12))
#> [PASS] every kept marker has MAF >= 0.05
chk("base allele frequencies are stored, one per marker", length(g$base_freq) ==
    ncol(g$dosage))
#> [PASS] base allele frequencies are stored, one per marker
chk("counted alleles are stored with the base frequencies",
    !is.null(attr(g$base_freq, "counted")) || is.null(g_raw$map$counted))
#> [PASS] counted alleles are stored with the base frequencies
```

The distributions behind these decisions:

```
qc_df <- rbind(data.frame(what = "minor-allele frequency (kept markers)", value = mafs),
    data.frame(what = "call rate per marker (before QC)", value =
        colMeans(!is.na(g_raw$dosage))))
ggplot(qc_df, aes(value)) + geom_histogram(bins = 30, fill = tso_colors()[["training"]]) +
    facet_wrap(~ what, scales = "free") + tso_theme() + labs(x = NULL, y = "markers")
```



A stricter set of thresholds, for comparison:

```
g_strict <- tso_qc(g_raw, maf = 0.10, call_rate = 0.95, line_call_rate = 0.9, verbose =
    FALSE)
c(default = ncol(g$dosage), strict = ncol(g_strict$dosage))
#> default strict
#> 958 890
chk("stricter thresholds keep fewer (or equal) markers", ncol(g_strict$dosage) <=
    ncol(g$dosage))
```

```
#> [PASS] stricter thresholds keep fewer (or equal) markers
```

3.1 Allele orientation across breeding cycles

HapMap and allele-letter files are coded by each file's own minor allele, so the same marker can flip between two files. When the base frequencies of an earlier cycle are reused, TSOpt re-oriens the new file to the stored counted allele.

```
if (using_example) {
  hmp_sub <- gs$HapMap; hmp_sub$dosage <- hmp_sub$dosage[1:60, ]
  bf <- tso_qc(gs$VCF, verbose = FALSE)$base_freq
  aligned <- tso_qc(hmp_sub, base_freq = bf, verbose = FALSE)
  flipped <- grep("re-oriented", aligned$log, value = TRUE)
  cat(if (length(flipped)) flipped else "no marker needed re-orienting", "\n")
  vcf_qc <- tso_qc(gs$VCF, verbose = FALSE)
  cm <- intersect(colnames(aligned$dosage), colnames(vcf_qc$dosage))
  called <- !is.na(gs$VCF$dosage[1:60, cm]) & !is.na(gs$HapMap$dosage[1:60, cm])
  agree <- mean((aligned$dosage[, cm] == vcf_qc$dosage[1:60, cm])[called])
  cat("re-oriented HapMap dosages agree with the VCF on", round(100 * agree, 2), "% of
    ↪ called genotypes\n")
  chk("re-oriented HapMap dosages equal the VCF dosages", agree == 1)
  # K is rescaled to mean diagonal 1 within each panel, so compare the unscaled matrices
  sub_vcf <- vcf_qc; sub_vcf$dosage <- vcf_qc$dosage[1:60, cm]
  chk("and give the same (unscaled) relationship matrix",
    max(abs(tso_kinship(aligned, base_freq = bf[cm], scale = FALSE) -
      tso_kinship(sub_vcf, base_freq = bf[cm], scale = FALSE))) < 1e-8)
}
#> Alleles 471 markers re-oriented to the base population's counted allele
#> re-oriented HapMap dosages agree with the VCF on 100 % of called genotypes
#> [PASS] re-oriented HapMap dosages equal the VCF dosages
#> [PASS] and give the same (unscaled) relationship matrix
```

4 Relationship matrices

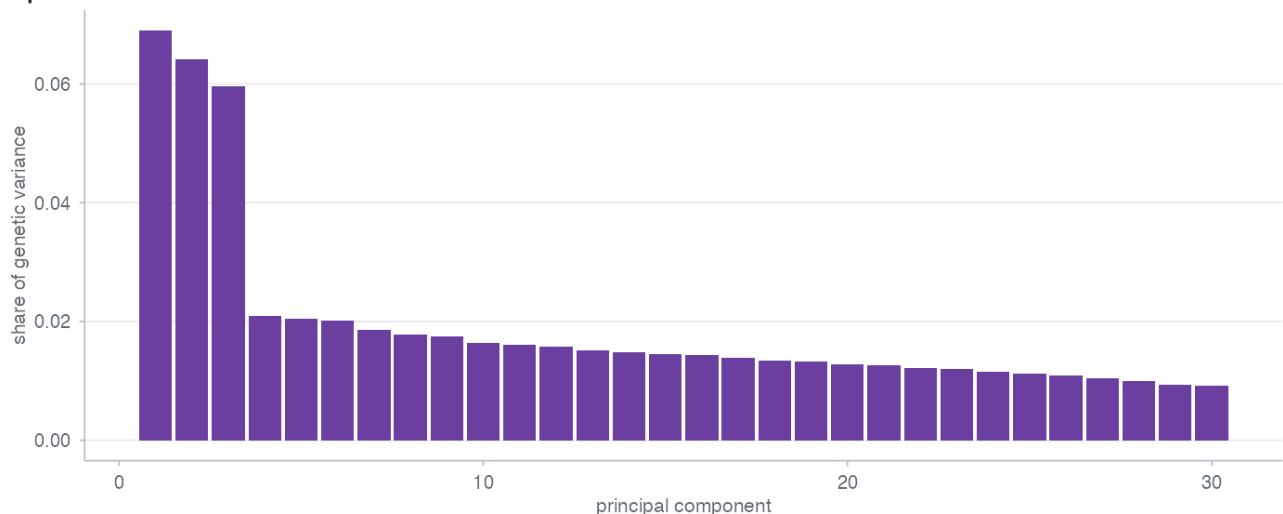
4.1 Additive (VanRaden)

```
K <- tso_kinship(g)
round(K[1:6, 1:6], 3)
#>      L001 L002 L003 L004 L005 L006
#> L001 0.989 0.039 0.106 0.078 0.093 0.355
#> L002 0.039 0.978 0.070 0.300 0.046 0.238
#> L003 0.106 0.070 0.992 0.077 0.289 0.091
#> L004 0.078 0.300 0.077 0.936 0.047 0.058
#> L005 0.093 0.046 0.289 0.047 0.971 0.089
#> L006 0.355 0.238 0.091 0.058 0.089 0.976
ev <- eigen(K, symmetric = TRUE, only.values = TRUE)$values
c(mean_diagonal = mean(diag(K)), min_eigenvalue = min(ev), mean_off_diagonal =
  ↪ mean(K[upper.tri(K)]))
#>      mean_diagonal      min_eigenvalue mean_off_diagonal
#>      1.000100          0.000100          -0.003344
chk("K is symmetric", isSymmetric(unname(K)))
#> [PASS] K is symmetric
chk("K is positive semi-definite", min(ev) > -1e-8)
#> [PASS] K is positive semi-definite
chk("mean diagonal of K is 1 (entry-mean scale)", abs(mean(diag(K)) - 1) < 1e-3)
#> [PASS] mean diagonal of K is 1 (entry-mean scale)
chk("row and column names are the line IDs", identical(rownames(K), rownames(g$dosage)))
#> [PASS] row and column names are the line IDs
```

The eigenvalue spectrum tells how structured the panel is: a few dominant eigenvalues mean strong population structure; a flat spectrum means a continuum.

```
ggplot(data.frame(k = seq_along(ev)[1:30], share = (ev / sum(ev))[1:30]), aes(k, share)) +
  geom_col(fill = tso_colors()[["training"]]) + tso_theme() +
  labs(x = "principal component", y = "share of genetic variance", title = "Spectrum of K")
```

Spectrum of K

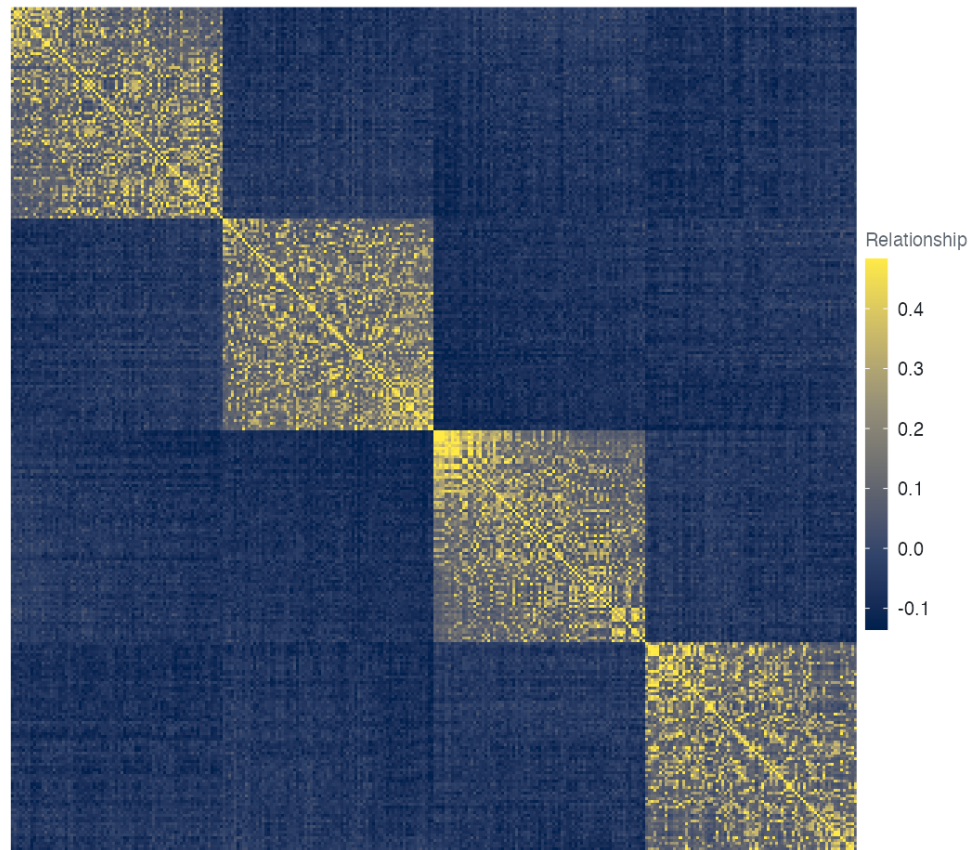


The relationship matrix as a heat map, lines ordered by genetic group (each block on the diagonal is a subpopulation or family):

```
tso_plot(tso_data(g, groups = if (nzchar(group_file)) {
  d <- utils::read.csv(group_file); stats::setNames(d$group, as.character(d$id)) }, qc =
  ↪ FALSE,
  verbose = FALSE), "kinship")
```

Genomic relationships

300 lines, ordered by group



4.2 Dominance, weighted and pedigree kernels

```

if (g$ploidy == 2) {
  Kd <- tso_kinship(g, method = "dominance")
  chk("dominance kernel is symmetric and PSD",
      isSymmetric(unname(Kd)) && min(eigen(Kd, TRUE, only.values = TRUE)$values) > -1e-8)
}
#> [PASS] dominance kernel is symmetric and PSD
w <- setNames(rep(1, ncol(g$dosage)), colnames(g$dosage)); w[1:50] <- 10
Kw <- tso_kinship(g, weights = w)
chk("weighted kinship differs from the plain one", max(abs(Kw - K)) > 1e-4)
#> [PASS] weighted kinship differs from the plain one
w0 <- w; w0[] <- 1
chk("weights all equal to 1 reproduce the plain kinship", max(abs(tso_kinship(g, weights =
  ↪ w0) - K)) < 1e-10)
#> [PASS] weights all equal to 1 reproduce the plain kinship
ped <- data.frame(id = c("F1", "F2", "F3", "S1", "S2", "S3", "G1"),
  sire = c(NA, NA, NA, "F1", "F1", "F1", "S1"),
  dam = c(NA, NA, NA, "F2", "F2", "F3", "S3"))
A <- tso_kinship(method = "pedigree", pedigree = ped, scale = FALSE, ridge = 0)
A
#>      F1  F2  F3  S1  S2  S3  G1
#> F1 1.0 0.00 0.00 0.500 0.500 0.500 0.500
#> F2 0.0 1.00 0.00 0.500 0.500 0.000 0.250
#> F3 0.0 0.00 1.00 0.000 0.000 0.500 0.250

```

```
#> S1 0.5 0.50 0.00 1.000 0.500 0.250 0.625
#> S2 0.5 0.50 0.00 0.500 1.000 0.250 0.375
#> S3 0.5 0.00 0.50 0.250 0.250 1.000 0.625
#> G1 0.5 0.25 0.25 0.625 0.375 0.625 1.125
chk("pedigree: full sibs S1, S2 are related 0.5", abs(A["S1", "S2"] - 0.5) < 1e-12)
#> [PASS] pedigree: full sibs S1, S2 are related 0.5
chk("pedigree: half sibs S1, S3 are related 0.25", abs(A["S1", "S3"] - 0.25) < 1e-12)
#> [PASS] pedigree: half sibs S1, S3 are related 0.25
chk("pedigree: G1 (S1 x S3) is inbred, A = 1.125", abs(A["G1", "G1"] - 1.125) < 1e-12)
#> [PASS] pedigree: G1 (S1 x S3) is inbred, A = 1.125
```

4.3 Epistatic, non-parametric and single-step kernels

`tso_kernels()` builds every covariance model the design can use. Dominance kernels are only built when your material is heterozygous (inbred lines carry no dominance):

```
het <- mean(abs(g$dosage - 1) < 0.5, na.rm = TRUE)
which_k <- c("A", if (g$ploidy == 2 && het > 0.01) "D", "AA", "gaussian")
kk <- tso_kernels(g, which = which_k)
kk
#> TSOpt kernels · 3 · each scaled to mean diagonal 1
#> A additive 300 lines additive genomic relationship (VanRaden 2008)
#> AA epistatic 300 lines additive x additive epistasis, A # A
↳ (Henderson 1985; Jiang & Reif 2015)
#> gaussian non-parametric 300 lines Gaussian kernel exp(-d^2 / h), RKHS (Gianola
↳ & van Kaam 2008)
#> Next: tso_data(geno, kernels = kernels, kernel_share = "robust")
chk("every kernel has mean diagonal 1", all(abs(sapply(kk$K, function(M) mean(diag(M))) -
↳ 1) < 1e-10))
#> [PASS] every kernel has mean diagonal 1
chk("every kernel is positive semi-definite",
all(sapply(kk$K, function(M) min(eigen(M, TRUE, only.values = TRUE)$values)) > -1e-8))
#> [PASS] every kernel is positive semi-definite
AA0 <- kk$K$A * kk$K$A
chk("AA is the element-wise square of A", max(abs(kk$K$AA - AA0 / mean(diag(AA0)))) < 1e-8)
#> [PASS] AA is the element-wise square of A
off <- upper.tri(kk$K$A)
chk("epistatic relationships fall off faster than additive ones",
mean(abs(kk$K$AA[off])) < mean(abs(kk$K$A[off])))
#> [PASS] epistatic relationships fall off faster than additive ones
Dp <- TSOpt:::pedigree_D(ped, TSOpt:::pedigree_A(ped))
chk("pedigree dominance: full sibs share 0.25, half sibs 0", abs(Dp["S1", "S2"] - 0.25) <
↳ 1e-12 && Dp["S1", "S3"] == 0)
#> [PASS] pedigree dominance: full sibs share 0.25, half sibs 0
# single-step H: 10 new crosses of genotyped parents, not genotyped themselves
par_ids <- rownames(g$dosage)[1:20]
ped_h <- data.frame(id = c(par_ids, paste0("new", 1:10)), p1 = c(rep(NA, 20),
↳ par_ids[1:10]),
p2 = c(rep(NA, 20), par_ids[11:20]))
gH <- g; gH$dosage <- g$dosage[par_ids, ]
H <- tso_kernels(gH, pedigree = ped_h, which = "H")$K$H
chk("H covers the ungenotyped crosses", all(paste0("new", 1:10) %in% rownames(H)))
#> [PASS] H covers the ungenotyped crosses
chk("a new cross is closer to its parent than to other lines", H["new1", par_ids[1]] >
↳ mean(H["new1", par_ids[-c(1, 11)]]))
#> [PASS] a new cross is closer to its parent than to other lines
tp_k <- tso_data(g, kernels = kk, kernel_share = "robust", qc = FALSE, verbose = FALSE)
chk("tso_data combines the kernels at the centre shares",
max(abs(tp_k$K - TSOpt:::combine_kernels(tp_k$kernels$K, tp_k$kernels$share))) <
↳ 1e-10)
```

```
#> [PASS] tso_data combines the kernels at the centre shares
```

5 The data object and the breeding systems

`tso_data()` packs genotypes, the relationship matrix, the breeding system, groups, families and costs into one object. A *unit* is anything you can phenotype: a line, a hybrid, a clone, a line in one environment, or one trait on one line.

5.1 Inbred lines (wheat, rice, barley, soybean)

```
read_side <- function(f, col) if (nzchar(f) && file.exists(f)) { d <- utils::read.csv(f)
  stats::setNames(d[[col]], as.character(d$id)) } else NULL
groups <- read_side(group_file, "group")
costs <- read_side(cost_file, "cost")
tp <- tso_data(g, groups = groups, cost = costs, qc = FALSE)
ids <- tp$units$id; N <- length(ids)
n_train <- min(params$n_train, floor(0.3 * N))
targets <- ids[seq(N - round(0.15 * N) + 1, N)] # the last 15% of lines play "this
  ↪ season"
cat("units:", N, " training-set size used below:", n_train, " targets:", length(targets),
  ↪ "\n")
#> units: 300 training-set size used below: 60 targets: 45
chk("one unit per line", nrow(tp$units) == nrow(g$dosage))
#> [PASS] one unit per line
chk("the kernel covers every unit", identical(dim(tp$K), c(N, N)))
#> [PASS] the kernel covers every unit
if (!is.null(costs)) chk("costs attached to every unit", !anyNA(tp$units$cost))
#> [PASS] costs attached to every unit
```

The population structure the design will see:

```
tso_plot(tp)
```



5.2 Clonal crops: diploid (cassava, banana) and tetraploid (potato)

For clones the non-additive part matters, so a dominance kernel is added for diploids. The tetraploid panel is built from two outbred diploid simulations (dosages 0 to 4).

```
tp_clone <- tso_data(g, system = "clonal", qc = FALSE, verbose = FALSE)
chk("clonal kernel differs from the additive one", max(abs(tp_clone$K - tp$K)) > 1e-6)
#> [PASS] clonal kernel differs from the additive one
p_a <- tso_simulate_panel(n = 150, markers = 600, seed = 31, type = "outbred")
p_b <- tso_simulate_panel(n = 150, markers = 600, seed = 32, type = "outbred")
X4 <- p_a$dosage + p_b$dosage; rownames(X4) <- paste0("P0T", seq_len(nrow(X4)))
g_potato <- tso_read(X4, ploidy = 4, verbose = FALSE)
tp_potato <- tso_data(g_potato, system = "clonal")
chk("tetraploid ploidy detected", g_potato$ploidy == 4)
#> [PASS] tetraploid ploidy detected
```

5.3 Outcrossing crops (ryegrass, alfalfa, oil palm): families

```
fam <- setNames(paste0("F", ceiling(seq_len(N) / 10)), ids)
tp_out <- tso_data(g, system = "outcross", families = fam, qc = FALSE, verbose = FALSE)
chk("families attached", length(unique(tp_out$units$family)) == length(unique(fam)))
#> [PASS] families attached
```

5.4 Hybrids (maize, sorghum, rice hybrids)

Units are all crosses between a female and a male pool; their relationships are built from the parents (GCA of each pool plus SCA). Hybrids involving parents not declared crossable can be targets but not training units.

```
n_pool <- min(30, floor(N / 4))
fem <- ids[1:n_pool]; mal <- ids[(n_pool + 1):(2 * n_pool)]
tp_hyb <- tso_data(g, system = "hybrid", pools = list(fem, mal), qc = FALSE,
  crossable = c(fem[1:(n_pool - 5)], mal[1:(n_pool - 5)]))
chk("every female x male cross is a unit", nrow(tp_hyb$units) == n_pool^2)
#> [PASS] every female x male cross is a unit
chk("hybrids of non-crossable parents are not candidates", sum(!tp_hyb$units$crossable) >
  0)
#> [PASS] hybrids of non-crossable parents are not candidates
```

5.5 Genebank or diversity panel

```
tp_bank <- tso_data(g, system = "genebank", qc = FALSE, verbose = FALSE)
chk("genebank object built", inherits(tp_bank, "tso_data"))
#> [PASS] genebank object built
```

5.6 Pedigree or kinship only

When there are no markers, a pedigree or a relationship matrix from elsewhere is enough. A matrix that is not positive semi-definite (bent pedigrees, edited imports) is repaired with a warning.

```
tp_ped <- tso_data(pedigree = ped)
tp_ped
#> <tso_data> system: inbred · 7 units
#> Kinship pedigree A (7 individuals)
Kbad <- K[1:40, 1:40]; Kbad[1, 2] <- Kbad[2, 1] <- 5
bent <- tryCatch(tso_data(kinship = Kbad, verbose = FALSE), warning = function(w)
  conditionMessage(w))
```

```
cat(substr(bent, 1, 110), "...\\n")
#> The supplied kinship is not positive semi-definite (smallest eigenvalue -4.18). It was
  ↪ bent: negative eigenval ...
chk("a non-PSD kinship is detected and bent", grepl("positive semi-definite", bent))
#> [PASS] a non-PSD kinship is detected and bent
```

5.7 Multi-environment and multi-trait data

```
Om <- matrix(c(1, .7, .4, .7, 1, .5, .4, .5, 1), 3, dimnames = rep(list(c("Site1", "Site2",
  ↪ "Site3")), 2))
tp_met <- tso_data(g, envs = colnames(Om), env_cor = Om, qc = FALSE,
  env_noise = c(Site1 = 1, Site2 = 2, Site3 = 1))
chk("units are line x environment cells", nrow(tp_met$units) == 3 * N)
#> [PASS] units are line x environment cells
tp_mt <- tso_data(g, traits = c("yield", "protein", "ndvi"), trait_h2 = c(0.3, 0.5, 0.6),
  gcor = matrix(c(1, .4, .2, .4, 1, .1, .2, .1, 1), 3),
  trait_cost = c(yield = 20, protein = 8, ndvi = 1), plot_cost = 10, qc =
  ↪ FALSE,
  verbose = FALSE)
chk("units are line x trait cells", nrow(tp_mt$units) == 3 * N)
#> [PASS] units are line x trait cells
```

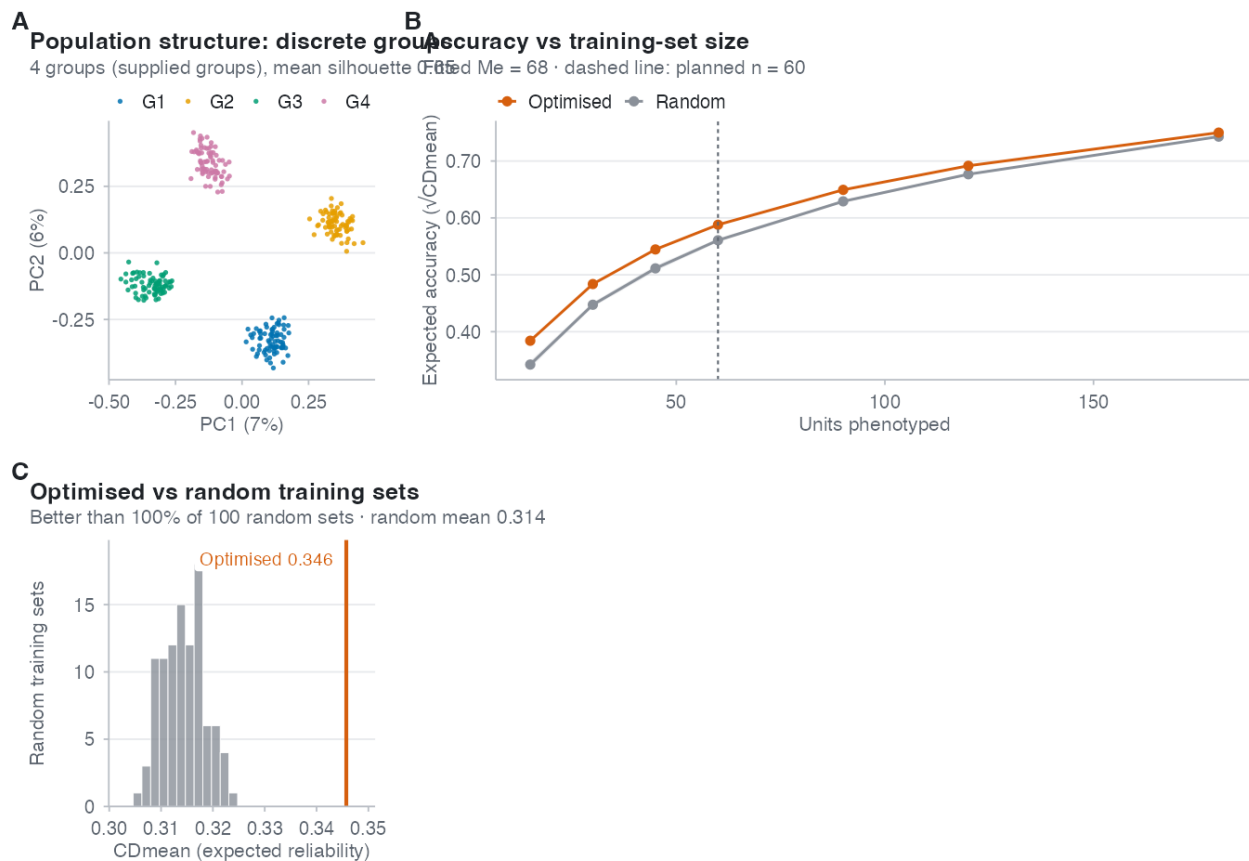
6 Is optimisation worth it here?

6.1 tso_diagnose()

```
dg <- tso_diagnose(tp, n = n_train)
dg
#> TSOpt diagnosis · 300 candidates · planned n = 60 · h2 = 0.5
#> Verdict      AMBER expected accuracy gain over random at n = 60: 4.9%
#> Advice      Optimisation helps a little; tso_design() is still the better choice, but
#> ↳ random sampling would not be a disaster. The panel has discrete groups: tso_design()
#> ↳ will stratify automatically.
#> Structure    discrete groups (4 groups, silhouette 0.65, supplied groups)
#> Diversity    51 effective independent lines (of 300) · fitted Me = 68
#> Size curve   n=15: 0.34 -> 0.38 · n=30: 0.45 -> 0.48 · n=45: 0.51 -> 0.54 · n=60: 0.56
#> ↳ -> 0.59 · n=90: 0.63 -> 0.65 · n=120: 0.68 -> 0.69 · n=180: 0.74 -> 0.75
#> expected accuracy (sqrt CDmean), random -> optimised
#> Next        tso_plot(diagnosis) · tso_design(data, n = 60)
chk("diagnosis gives a traffic-light verdict", dg$verdict$light %in% c("GREEN", "AMBER",
#> ↳ "RED"))
#> [PASS] diagnosis gives a traffic-light verdict
chk("optimised accuracy >= random at every size", all(dg$curve$optimised_acc >=
#> ↳ dg$curve$random_acc - 1e-9))
#> [PASS] optimised accuracy >= random at every size
```

tso_plot(dg)

Diagnosis: AMBER · expected gain 4.9% at n = 60



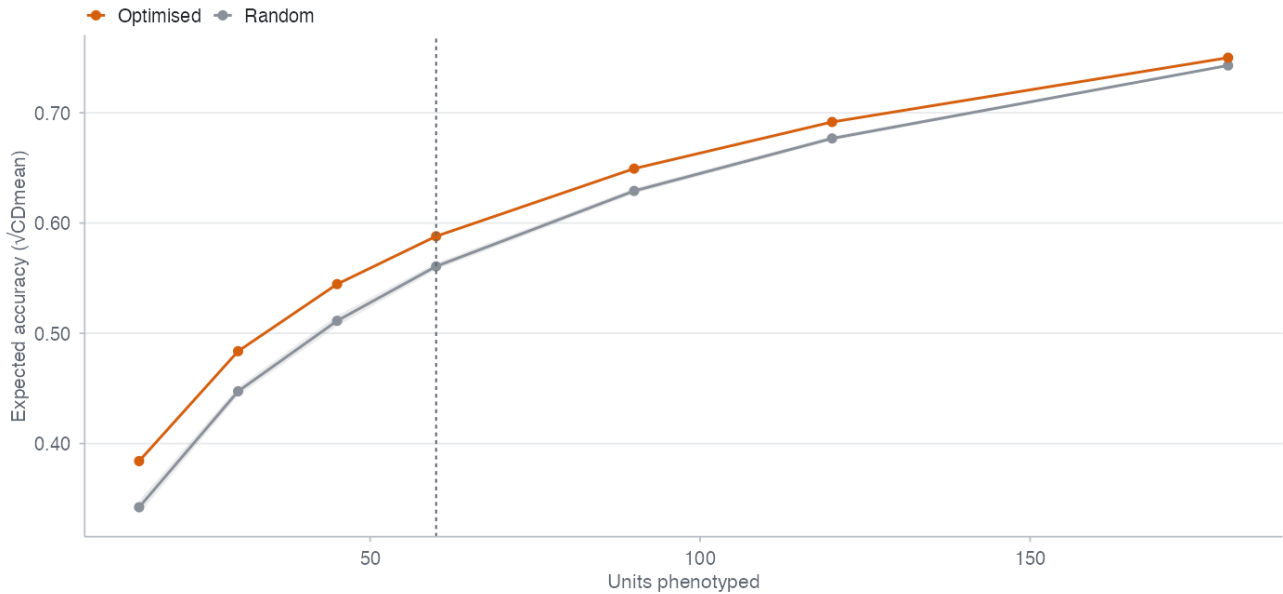
The two panels separately. The size curve shows accuracy against the number of lines pheno-

typed, for optimised and random sets; the gap between the curves is what optimisation buys. The structure panel shows the groups the stratification will use.

```
tso_plot(dg, "size_curve")
```

Accuracy vs training-set size

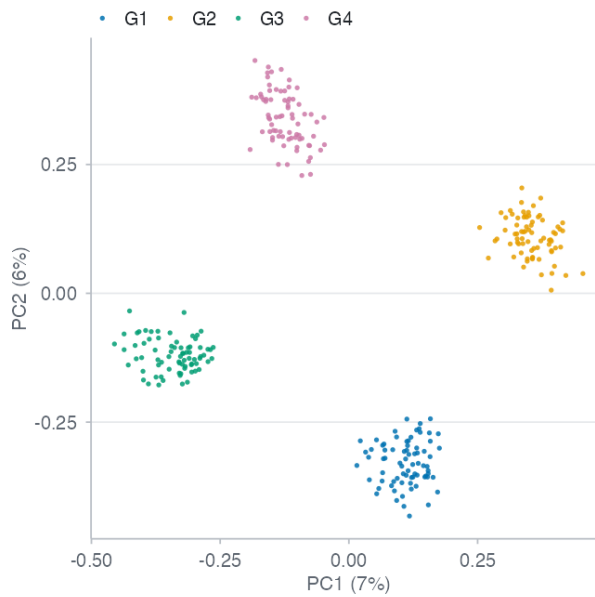
Fitted Me = 68 • dashed line: planned n = 60



```
tso_plot(dg, "structure")
```

Population structure: discrete groups

4 groups (supplied groups), mean silhouette 0.65



```
knitr::kable(dg$curve, digits = 3, row.names = FALSE)
```

n	random_cd	random_sd	optimised_cd	random_acc	optimised_acc	gain_acc
15	0.117	0.004	0.148	0.342	0.384	0.122
30	0.200	0.004	0.234	0.447	0.484	0.081

n	random_cd	random_sd	optimised_cd	random_acc	optimised_acc	gain_acc
45	0.261	0.005	0.297	0.511	0.545	0.065
60	0.314	0.003	0.346	0.561	0.588	0.049
90	0.396	0.003	0.422	0.629	0.649	0.032
120	0.458	0.003	0.478	0.677	0.692	0.022
180	0.552	0.001	0.562	0.743	0.750	0.009

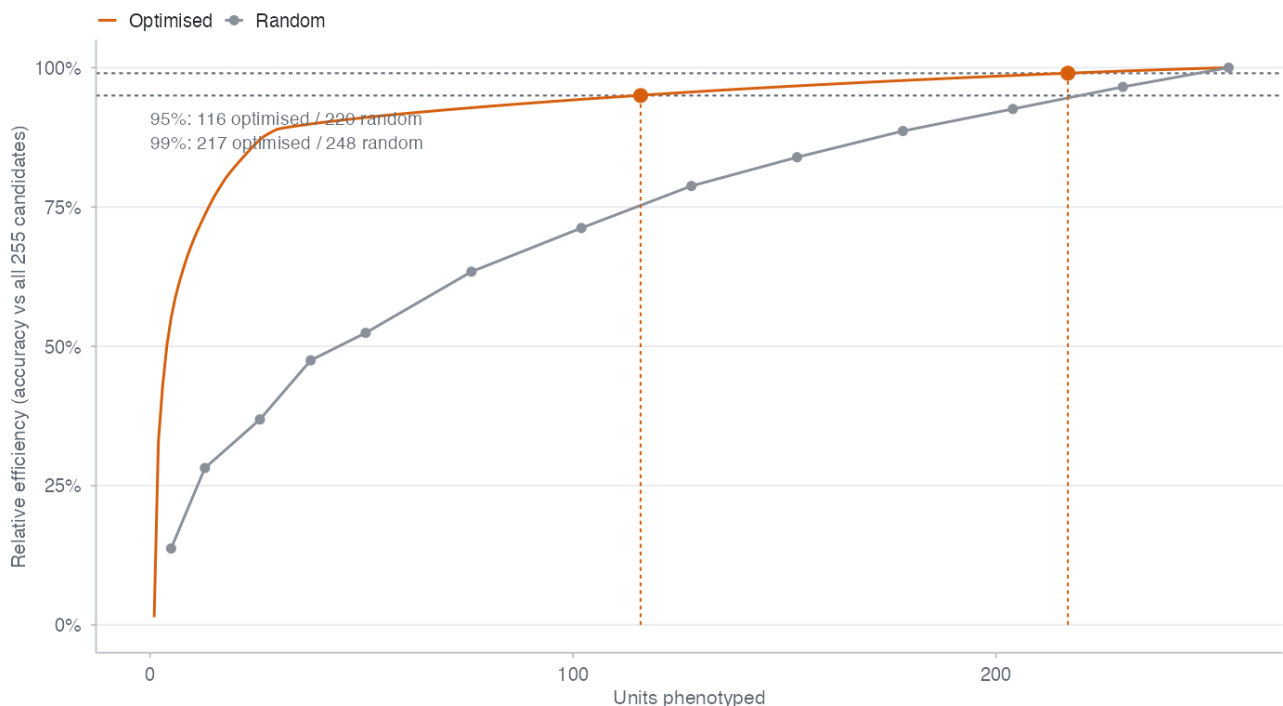
6.2 tso_sample_size(): how many lines?

```
ss <- tso_sample_size(tp, target = targets)
ss
#> TSOpt sample size · 255 candidates · 45 targets · averaged over h2 = 0.2, 0.5, 0.8
#> Reference expected accuracy 0.532 (phenotyping all 255)
#> → Phenotype 116 lines to reach 95% of the accuracy of phenotyping all 255; random
  ↳ sampling would need 220.
#> Target RE
#>          90%      optimised random plots saved
#>          95%      116      220      104
#>          99%      217      248      31
#> Fit      logistic acc(n) = 0.510 / (1 + exp(0.60 - 0.1262 n)), RMSE 0.0196
  ↳ (smoothing only)
#> Method    exact greedy curve (optimised) · mean of 20 random sets per size (random)
chk("n for 95% <= n for 99%", ss$n_for[["0.95"]] <= ss$n_for[["0.99"]])
#> [PASS] n for 95% <= n for 99%
chk("the optimised curve never needs more plots than random", all(ss$savings >= 0, na.rm =
  ↳ TRUE))
#> [PASS] the optimised curve never needs more plots than random
```

tso_plot(ss)

Phenotype 116 to reach 95% of the accuracy of all 255 candidates

Exact greedy CDmean curve (averaged over h2) vs mean of 20 random sets



7 Every criterion

TSOpt has 18 built-in criteria in five tiers, plus any function you write:

```
crit <- tso_criteria()
knitr::kable(crit[, c("criterion", "tier", "search")], row.names = FALSE)
```

criterion	tier	search
cdmean	Recommended	greedy (robust)
pevmean	Specialist	tabu
cdpop	Specialist	tabu
reliability	Specialist	tabu
cdmin	Specialist	tabu
pevmax	Specialist	tabu
gain	Specialist	tabu
dopt	Classical	tabu
aopt	Classical	tabu
eopt	Classical	ga
rscore	Classical	ga
coverage	Model-free	greedy
haplotype	Model-free	greedy
maximin	Model-free	greedy
pam	Model-free	pam
stratified	Model-free	sampling
random	Baseline	sampling
kinship	Baseline (harmful)	ranking

Each criterion now designs a training set for the same problem (predict the targets, choose `n_train` lines). Every set is then scored with the same yardstick, the exact CDmean at the given heritability, so the table shows what each criterion trades away.

```
crit_sets <- list()
run_crit <- function(cr) {
  d <- if (cr == "cdpop") tp_out else tp          # CDpop compares family means: families
  ↪ below
  p <- timed(tso_design(d, n = n_train, target = targets, criterion = cr, h2 = params$h2,
    iterations = 100, n_random = 20, verbose = FALSE))
  crit_sets[[cr]] <- p$selected$id
  data.frame(criterion = cr, lines = nrow(p$selected),
    targets_in_training = sum(p$selected$id %in% targets),
    exact_CDmean = tso_score(tp, train = p$selected$id, target = targets, h2 =
      ↪ params$h2,
      criteria = "cdmean_exact")["cdmean_exact"],
    seconds = round(attr(p, "seconds"), 2))
}
crit_res <- do.call(rbind, lapply(crit_sets, run_crit))
crit_res$exact_CDmean <- round(crit_res$exact_CDmean, 4)
knitr::kable(crit_res[order(-crit_res$exact_CDmean), ], row.names = FALSE)
```

criterion	lines	targets_in_training	exact_CDmean	seconds
cdmean	60	0	0.2438	0.04
pevmean	60	0	0.2438	0.04
eopt	60	0	0.2436	0.71
rscore	60	0	0.2436	3.07
pevmax	60	0	0.2420	0.06

criterion	lines	targets_in_training	exact_CDmean	seconds
cdmin	60	0	0.2416	0.05
cdpop	60	0	0.2412	0.06
gain	60	0	0.2412	0.31
reliability	60	0	0.2409	0.06
aopt	60	0	0.2408	0.05
dopt	60	0	0.2404	0.02
haplotype	60	0	0.2397	0.44
stratified	60	0	0.2396	0.04
kinship	60	0	0.2395	0.12
coverage	60	0	0.2004	0.05
maximin	60	0	0.1547	0.04
pam	60	0	0.0938	0.05
random	60	0	0.0694	0.05

```

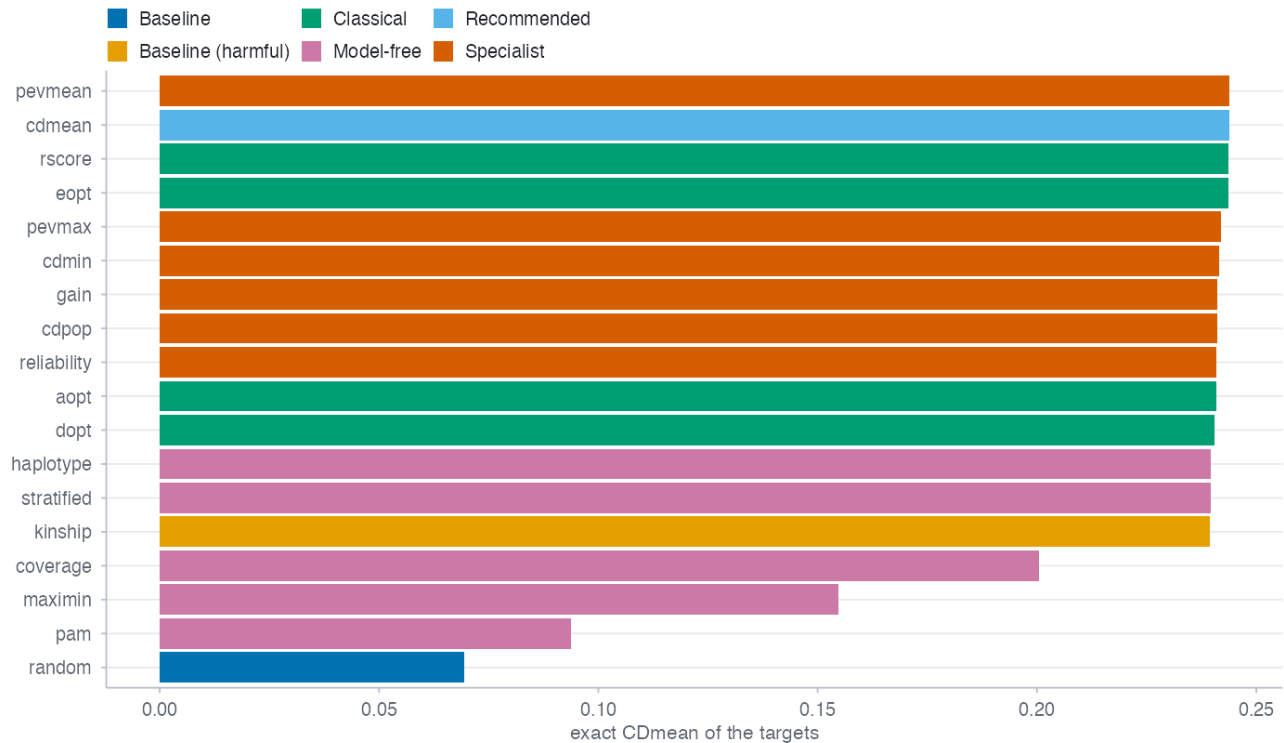
chk("every criterion returns n_train lines", all(crit_res$lines == n_train))
#> [PASS] every criterion returns n_train lines
chk("no criterion puts a target in the training set", all(crit_res$targets_in_training ==
  ↪ 0))
#> [PASS] no criterion puts a target in the training set
chk("cdmean (the default) beats random sampling",
  crit_res$exact_CDmean[crit_res$criterion == "cdmean"] >
  ↪ crit_res$exact_CDmean[crit_res$criterion == "random"])
#> [PASS] cdmean (the default) beats random sampling

crit_res$tier <- crit$tier[match(crit_res$criterion, crit$criterion)]
ggplot(crit_res, aes(stats::reorder(criterion, exact_CDmean), exact_CDmean, fill = tier)) +
  geom_col() + coord_flip() + tso_theme() +
  scale_fill_manual(values = unname(tso_colors("groups", length(unique(crit_res$tier)))))
  ↪ +
  labs(x = NULL, y = "exact CDmean of the targets", fill = NULL,
    title = "Every criterion on the same problem", subtitle = "higher is better")

```

Every criterion on the same problem

higher is better



Where the training sets of four contrasting criteria sit in the genetic space:

```
pcs <- tp$pca$x[, 1:2]; colnames(pcs) <- c("PC1", "PC2")
show_c <- intersect(c("cdmean", "coverage", "maximin", "kinship"), names(crit_sets))
pc_df <- do.call(rbind, lapply(show_c, function(cr) data.frame(pcs, criterion = cr,
  role = ifelse(ids %in% crit_sets[[cr]], "training", ifelse(ids %in% targets, "target",
    "candidate")))))
ggplot(pc_df, aes(PC1, PC2, colour = role)) + geom_point(size = 0.9, alpha = 0.8) +
  facet_wrap(~ criterion) + tso_theme() +
  scale_colour_manual(values = c(training = tso_colors()[["training"]], target =
    tso_colors()[["target"]],
    candidate = tso_colors()[["candidate"]])) + labs(colour =
    NULL)
```



What to look at: the CDmean-family criteria sit at the top because the yardstick is CDmean; the model-free ones (coverage, maximin, PAM) and the baselines show how much a model-based design adds on this panel. The kinship baseline (the lines most related to the targets) is kept only for comparison: in the head-to-head benchmark it was worse than random on average.

7.1 Your own criterion, and the direction check

A user function receives the selected IDs and the data object and returns a number. Things you want small (error, similarity) must be minimised; TSOpt probes the function and warns when it looks like it is optimised in the wrong direction.

```
similarity <- function(ids, data) mean(data$K[ids, ids])
warn <- tryCatch({ tso_design(tp, n = 30, criterion = similarity, iterations = 50, verbose
  ↪ = FALSE); "" },
  warning = function(w) conditionMessage(w))
cat(substr(warn, 1, 120), "...\\n")
#> Your criterion scores a set of near-duplicate relatives higher than every random set.
  ↪ Criteria that measure error, PEV o ...
chk("maximising a similarity triggers the wrong-direction warning", grepl("minimi", warn))
#> [PASS] maximising a similarity triggers the wrong-direction warning
p_div <- tso_design(tp, n = 30, criterion = similarity, direction = "minimize", iterations
  ↪ = 100,
  verbose = FALSE)
set.seed(1); rand_sim <- mean(replicate(20, similarity(sample(ids, 30), tp)))
```

```
c(minimised = similarity(p_div$selected$id, tp), random_sets = rand_sim)
#>   minimised random_sets
#>   0.01376   0.03209
chk("minimised similarity is below that of random sets", similarity(p_div$selected$id, tp)
    < rand_sim)
#> [PASS] minimised similarity is below that of random sets
```

8 Every search

The same criterion (CDmean at one heritability) optimised by each search. Greedy adds the best line one at a time; exchange then swaps; ROTS (tabu) keeps moving past local optima; the genetic algorithm with simulated annealing works for any criterion; restarts run the local searches from several perturbed starts.

```
run_search <- function(s, restarts = 0) {
  p <- timed(tso_design(tp, n = n_train, target = targets, h2 = params$h2, search = s,
    restarts = restarts, iterations = 200, n_random = 20, verbose =
    FALSE))
  data.frame(search = paste0(s, if (restarts) paste0(" + ", restarts, " restarts") else
    ""),
    criterion_value = p$value$criterion_value %||% NA,
    exact_CDmean = tso_score(tp, train = p$selected$id, target = targets, h2 =
    params$h2,
    criteria = "cdmean_exact")["cdmean_exact"],
    seconds = round(attr(p, "seconds"), 2))
}
`%||` <- function(a, b) if (is.null(a)) b else a
sr <- rbind(run_search("greedy"), run_search("greedy+exchange"), run_search("tabu"),
  run_search("tabu", restarts = 3), run_search("ga"))
sr[, 2:3] <- round(sr[, 2:3], 5)
knitr::kable(sr, row.names = FALSE)
```

search	criterion_value	exact_CDmean	seconds
greedy	0.2438	0.2438	0.02
greedy+exchange	0.2438	0.2438	0.03
tabu	0.2438	0.2438	0.05
tabu + 3 restarts	0.2438	0.2438	0.15
ga	0.2438	0.2438	0.43

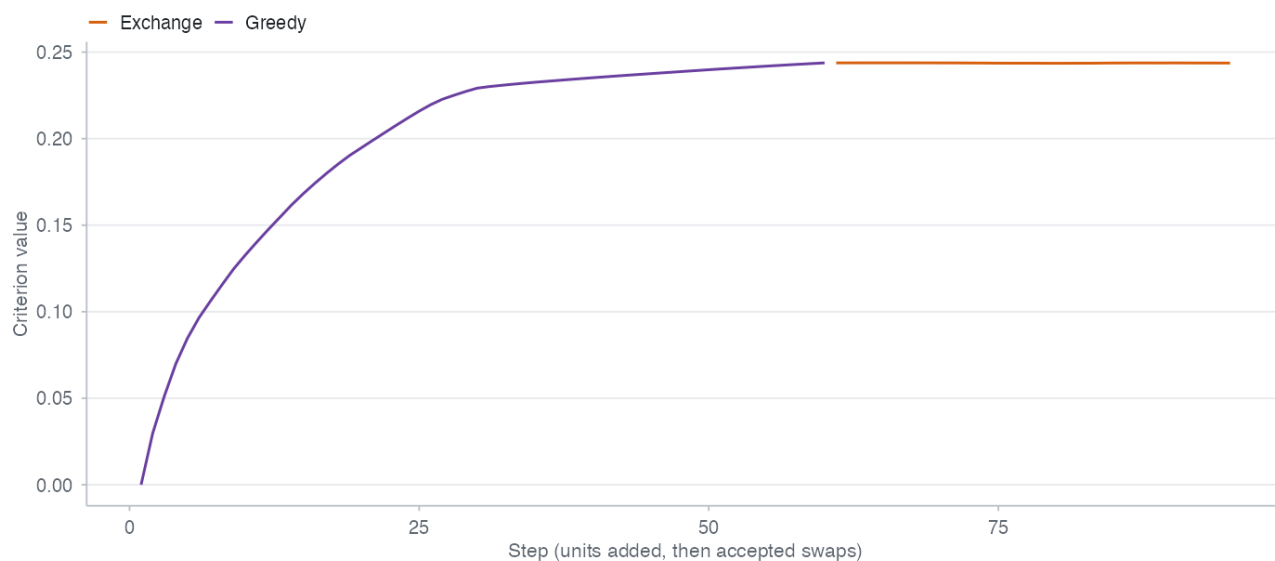
```
cv <- sr$exact_CDmean
chk("exchange is at least as good as greedy", cv[2] >= cv[1] - 1e-6)
#> [PASS] exchange is at least as good as greedy
chk("tabu is at least as good as exchange", cv[3] >= cv[2] - 1e-6)
#> [PASS] tabu is at least as good as exchange
chk("restarts never make tabu worse", cv[4] >= cv[3] - 1e-6)
#> [PASS] restarts never make tabu worse
```

The search trace of the tabu search: the criterion after every accepted move. Greedy climbs first, then exchange and tabu moves refine the set, sometimes accepting a worse move to escape a local optimum:

```
p_tabu <- tso_design(tp, n = n_train, target = targets, h2 = params$h2, search = "tabu",
  verbose = FALSE)
tso_plot(p_tabu, "search")
```

Search path

5 improving swaps in 3 exchange passes



What to look at: the harder searches raise the criterion a little. In the benchmark this extra criterion did **not** give better predictions, which is why the default is the plain greedy search on robust CDmean.

9 Building training sets: the core options

9.1 The default

Robust CDmean (averaged over heritability 0.2, 0.5 and 0.8), exact fixed intercept, greedy search, automatic stratification when the panel has discrete groups.

```
plan <- tso_design(tp, n = n_train)
plan
#> TSOpt plan · inbred · 60 lines of 300 candidates · untargeted (whole pool)
#> Criterion CDmean, exact (fixed intercept) (averaged over h2 = 0.2, 0.5, 0.8) Search
  ↳ greedy (robust CDmean: average over h2) (0.1 s)
#> Expected CDmean 0.346 (at h2 = 0.50) (exact, intercept projected: 0.345) vs random
  ↳ 0.315 ± 0.004 (better than 100% of 100 random sets)
#> Verdict AMBER modest gain (~4.9% accuracy over random); worth using, but do not
  ↳ expect much
#> Structure discrete groups (4 groups, silhouette 0.65) -> stratified allocation
  ↳ 15/15/15/15
#> Coverage nearest training relative >= 0.20 for 100% of targets · 0 targets unreachable
  ↳ (< 0.05)
#> Divide 0 candidates effectively unselectable (reliability < 0.06) · see
  ↳ tso_plot(plan, "divide")
#> Alleles 100.0% of target alleles present in training (0 markers not covered) · 90.0%
  ↳ of target 5-marker haplotypes
#> Plot value dropping one plot costs 0.86% to 1.72% of the CDmean (median 1.04%) · see
  ↳ tso_plot(plan, "value")
#> Next tso_plot(plan) · tso_export(plan, "trial.csv") · tso_explain(plan)
chk("default returns n_train lines", nrow(plan$selected) == n_train)
#> [PASS] default returns n_train lines
chk("default beats every random set", plan$value$percentile >= 0.95,
    paste0("better than ", round(100 * plan$value$percentile), "% of random sets"))
#> [PASS] default beats every random set (better than 100% of random sets)
chk("robust design reports CD at each heritability", length(plan$value$cd_by_h2) == 3)
#> [PASS] robust design reports CD at each heritability
```

Reading the printout, line by line.

- *Criterion*: what was optimised. “exact (fixed intercept)” means the trait mean is treated as an unknown fixed effect, as it is in the GBLUP you will fit; “averaged over h2 = 0.2, 0.5, 0.8” is the robust default.
- *Expected*: the CDmean (expected reliability of the target contrasts) of the chosen set, against the mean and spread of random sets of the same size, and the share of random sets it beats.
- *Verdict*: GREEN when the expected accuracy gain over random is 5% or more and the targets are covered, AMBER for 2 to 5%, RED below 2% (random sampling is then fine).
- *Structure*: whether the panel has discrete groups and how the lines were allocated to them.
- *Coverage*: the share of targets with a close relative (relationship of 0.20 or more) in the training set, and how many targets have no relative at all.
- *Divide*: candidates whose predictions will be too unreliable to select on.
- *Alleles*: the share of the targets’ alleles and 5-marker haplotypes that also occur in the training set.

The selected lines, their expected reliability, and every unit’s role:

```
knitr::kable(utils::head(plan$selected[, intersect(c("order", "id", "group", "cost",
  ↳ "reliability")),
  names(plan$selected)], 12), digits = 3)
```

order	id	group	cost	reliability
1	L234	G4	2.86	0.575

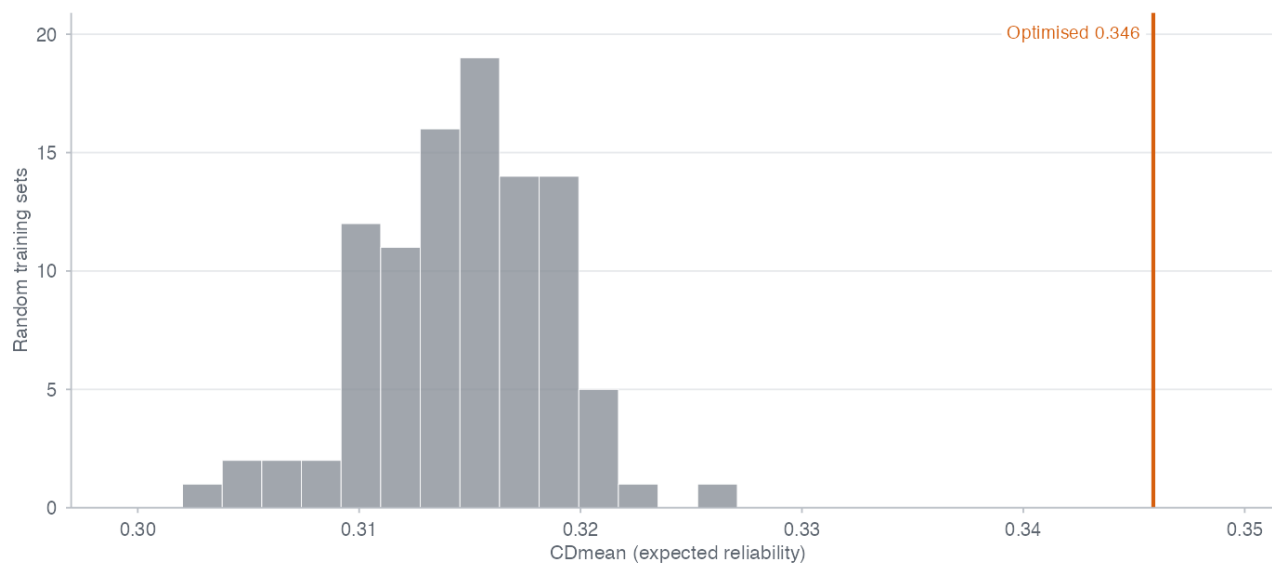
order	id	group	cost	reliability
2	L083	G2	1.22	0.586
3	L068	G1	1.12	0.599
4	L185	G3	2.50	0.585
5	L193	G3	3.00	0.569
6	L128	G2	2.72	0.556
7	L030	G1	2.80	0.586
8	L276	G4	2.98	0.572
9	L075	G1	2.82	0.575
10	L127	G2	1.24	0.575
11	L180	G3	2.88	0.559
12	L235	G4	1.19	0.576

```
table(plan$units$role)
#>
#> candidate training
#>      240      60
```

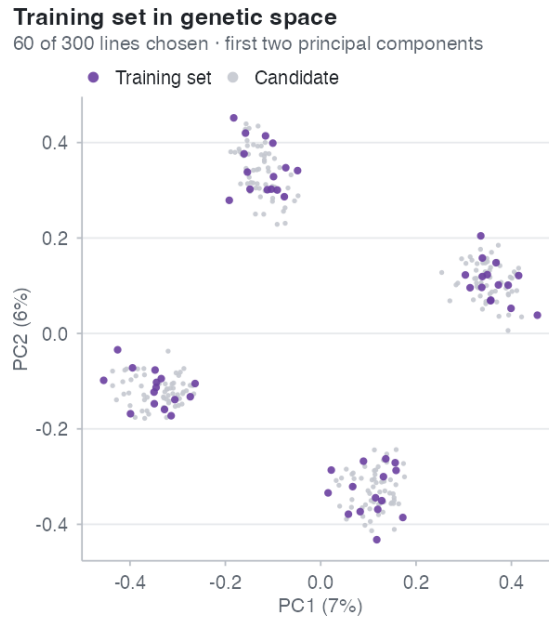
```
tso_plot(plan, "baseline")
```

Optimised vs random training sets

Better than 100% of 100 random sets · random mean 0.315



```
tso_plot(plan, "pca")
```



9.2 Targeted

```
plan_t <- tso_design(tp, n = n_train, target = targets, verbose = FALSE)
plan_t
#> TSOpt plan • inbred • 60 lines of 255 candidates • 45 named targets
#> Criterion CDmean, exact (fixed intercept) (averaged over h2 = 0.2, 0.5, 0.8) Search
  ↳ greedy (robust CDmean: average over h2) (0.0 s)
#> Expected CDmean 0.244 (at h2 = 0.50) (exact, intercept projected: 0.244) vs random
  ↳ 0.093 ± 0.022 (better than 100% of 100 random sets)
#> Verdict GREEN optimisation pays off here: expected accuracy ~63.3% above random
#> Structure discrete groups (4 groups, silhouette 0.65) -> stratified allocation
  ↳ 10/10/10/30
#> Coverage nearest training relative >= 0.20 for 100% of targets • 0 targets unreachable
  ↳ (< 0.05)
#> Divide 0 candidates effectively unselectable (reliability < 0.06) • see
  ↳ tso_plot(plan, "divide")
#> Alleles 100.0% of target alleles present in training (0 markers not covered) • 96.9%
  ↳ of target 5-marker haplotypes
#> Plot value dropping one plot costs 0.14% to 5.61% of the CDmean (median 0.53%) • see
  ↳ tso_plot(plan, "value")
#> Next tso_plot(plan) • tso_export(plan, "trial.csv") • tso_explain(plan)
chk("targets are never in the training set", !any(plan_t$selected$id %in% targets))
#> [PASS] targets are never in the training set
```

A fair comparison: the untargeted design above may phenotype target lines themselves, which flatters it on those targets. Restricted to the same candidates (targets excluded), the targeted design must predict the targets at least as well, at the heritability the score uses:

```
plan_u <- tso_design(tp, n = n_train, candidates = setdiff(ids, targets), h2 = params$h2,
  ↳ verbose = FALSE)
plan_t5 <- tso_design(tp, n = n_train, target = targets, h2 = params$h2, verbose = FALSE)
sc <- c(untargeted = tso_score(tp, train = plan_u$selected$id, target = targets, h2 =
  ↳ params$h2)[["cdmean"]],
  targeted = tso_score(tp, train = plan_t5$selected$id, target = targets, h2 =
  ↳ params$h2)[["cdmean"]])
round(sc, 4)
#> untargeted targeted
```

```
#>      0.1256      0.2443
chk("targeted design predicts the targets better than an untargeted one from the same
  ↪ candidates",
    sc[["targeted"]] >= sc[["untargeted"]] - 1e-6)
#> [PASS] targeted design predicts the targets better than an untargeted one from the same
  ↪ candidates
```

9.3 Heritability: known value or your own prior grid

```
p_h <- tso_design(tp, n = n_train, target = targets, h2 = params$h2, verbose = FALSE)
p_grid <- tso_design(tp, n = n_train, target = targets, h2 = c(0.1, 0.3, 0.5), verbose =
  ↪ FALSE)
round(p_grid$value$cd_by_h2, 4)
#> h2=0.1 h2=0.3 h2=0.5
#> 0.0632 0.1627 0.2442
chk("a single h2 is used as given", isTRUE(all.equal(p_h$spec$h2, params$h2)))
#> [PASS] a single h2 is used as given
```

9.4 A budget with unit costs

```
if (!is.null(costs)) {
  bud <- 2 * n_train * stats::median(tp$units$cost)
  p_bud <- tso_design(tp, budget = bud, target = targets, verbose = FALSE)
  c(budget = bud, spent = p_bud$spent, lines = nrow(p_bud$selected))
  chk("budget is respected", p_bud$spent <= bud + 1e-8)
} else cat("No cost file given: budget designs skipped.\n")
#> [PASS] budget is respected
```

9.5 Field reality: checks, lost seed and last season's data

```
chk_lines <- ids[1:3]; lost <- ids[4:10]; last_year <- ids[11:40]
p_real <- tso_design(tp, n = n_train, must_include = chk_lines, exclude = lost, history =
  ↪ last_year,
                      history_weight = 0.8, verbose = FALSE)
head(p_real$selected[, c("order", "id", "forced")], 5)
#>   order   id forced
#> 1     1 L001  TRUE
#> 2     2 L002  TRUE
#> 3     3 L003  TRUE
#> 4     4 L234 FALSE
#> 5     5 L185 FALSE
chk("checks are always included", all(chk_lines %in% p_real$selected$id))
#> [PASS] checks are always included
chk("excluded lines are never selected", !any(lost %in% p_real$selected$id))
#> [PASS] excluded lines are never selected
chk("history lines are not selected again", !any(last_year %in% p_real$selected$id))
#> [PASS] history lines are not selected again
chk("history lines are never evaluation targets", !any(p_real$index$history %in%
  ↪ p_real$index$eval))
#> [PASS] history lines are never evaluation targets
```

9.6 Quotas, caps and replicates

```
if (!is.null(groups)) {
  grp_lev <- names(table(tp$units$group))
  q <- setNames(rep(floor(n_train / length(grp_lev)), length(grp_lev)), grp_lev)
  p_q <- tso_design(tp, n = sum(q), group_quota = q, verbose = FALSE)
  print(table(p_q$selected$group))
  chk("group quotas are met exactly", all(table(p_q$selected$group)[grp_lev] == q))
}
#>
#> G1 G2 G3 G4
#> 15 15 15 15
#> [PASS] group quotas are met exactly
p_fam <- tso_design(tp_out, n = n_train, max_per_family = 3, verbose = FALSE)
chk("no family exceeds the cap of 3", max(table(p_fam$selected$family)) <= 3)
#> [PASS] no family exceeds the cap of 3
p_rep <- tso_design(tp, n = n_train, reps = 2, h2 = 0.2, stratify = "none", verbose = FALSE)
chk("replicated design fills all plots", nrow(p_rep$selected) == n_train)
#> [PASS] replicated design fills all plots
chk("no line gets more than 2 plots", max(table(p_rep$selected$id)) <= 2)
#> [PASS] no line gets more than 2 plots
```

9.7 Fixed effects: trials, blocks, covariates

Fixed effects are handled exactly. Every level of a factor fixed effect is opened with two units, because the first unit at a level only estimates that level's mean.

```
tp_tr <- tp; tp_tr$units$trial <- rep(c("A", "B", "C"), length.out = N)
p_fx <- tso_design(tp_tr, n = n_train, fixed = "trial", verbose = FALSE)
p_fx$spec$fixed
#> [1] "intercept + 2 fixed column(s) (exact)"
table(p_fx$selected$trial)
#>
#> A B C
#> 21 18 21
chk("every trial gets at least 2 lines", all(table(factor(p_fx$selected$trial, levels =
  ↪ c("A", "B", "C")) >= 2)))
#> [PASS] every trial gets at least 2 lines
cov <- matrix(rnorm(N), ncol = 1, dimnames = list(ids, "moisture"))
p_cov <- tso_design(tp, n = n_train, fixed = cov, verbose = FALSE)
chk("a covariate matrix is accepted as a fixed effect",
  ↪ identical(p_cov$spec$fixed_columns, "moisture"))
#> [PASS] a covariate matrix is accepted as a fixed effect
```

9.8 Genetic model: total or breeding value

With a non-additive model, a design can predict the total genetic value (the line itself) or the breeding value (the additive part, for choosing parents). Both are exact:

```
tp_aa <- tso_data(g, kernels = c("A", "AA"), qc = FALSE, verbose = FALSE)
p_tv <- tso_design(tp_aa, n = n_train, target = targets, verbose = FALSE)
p_bv <- tso_design(tp_aa, n = n_train, target = targets, value = "breeding", verbose =
  ↪ FALSE)
p_bv
#> TSOpt plan · inbred · 60 lines of 255 candidates · 45 named targets
#> Criterion CDmean, exact (fixed intercept) (averaged over h2 = 0.2, 0.5, 0.8) Search
  ↪ greedy (robust over 3 kernel-share scenarios x h2 grid) (0.1 s)
#> Expected CDmean 0.177 (at h2 = 0.50) (exact, intercept projected: 0.177) vs random
  ↪ 0.065 ± 0.015 (better than 100% of 100 random sets)
```

```

#> Model      A 70% + AA 30% (centre of the share scenarios) · predicts breeding values
↳ (the additive part)
#> Scenarios  CDmean non-additive 10% h2=0.2 0.105 · non-additive 10% h2=0.5 0.222 ·
↳ non-additive 10% h2=0.8 0.322 · non-additive 30% h2=0.2 0.083 · non-additive 30%
↳ h2=0.5 0.177 · non-additive 30% h2=0.8 0.256 · non-additive 50% h2=0.2 0.061 ·
↳ non-additive 50% h2=0.5 0.130 · non-additive 50% h2=0.8 0.188
#> Verdict    GREEN optimisation pays off here: expected accuracy ~66.0% above random
#> Structure  discrete groups (4 groups, silhouette 0.65) -> stratified allocation
↳ 10/10/30/10
#> Coverage   nearest training relative >= 0.20 for 100% of targets · 0 targets unreachable
↳ (< 0.05)
#> Divide     0 candidates effectively unselectable (reliability < 0.06) · see
↳ tso_plot(plan, "divide")
#> Alleles    100.0% of target alleles present in training (0 markers not covered) · 96.9%
↳ of target 5-marker haplotypes
#> Plot value  dropping one plot costs 0.16% to 5.60% of the CDmean (median 0.57%) · see
↳ tso_plot(plan, "value")
#> Next       tso_plot(plan) · tso_export(plan, "trial.csv") · tso_explain(plan)
chk("the robust design averages over 3 share x 3 heritability scenarios",
↳ length(p_bv$value$cd_by_scenario) == 9)
#> [PASS] the robust design averages over 3 share x 3 heritability scenarios
Sb <- match(p_bv$selected$id, tp_aa$units$id); Tb <- setdiff(match(targets,
↳ tp_aa$units$id), Sb)
Cb <- diag(length(Tb)) - 1 / length(Tb); Kb <- tp_aa$K; Ka <- tp_aa$K_add
Vb <- Kb[Sb, Sb] + diag(1, length(Sb))
cd_manual <- mean(diag(Cb %*% Ka[Tb, Sb] %*% solve(Vb, Ka[Sb, Tb]) %*% Cb) / diag(Cb %*%
↳ Ka[Tb, Tb] %*% Cb))
chk("breeding-value CDmean matches the direct formula",
↳ abs(tso_score(tp_aa, p_bv$selected$id, target = targets, value =
↳ "breeding")["cdmean"] - cd_manual) < 1e-8)
#> [PASS] breeding-value CDmean matches the direct formula
chk("the plan reports that it predicts breeding values", identical(p_bv$spec$value,
↳ "breeding"))
#> [PASS] the plan reports that it predicts breeding values
cat("lines in common between the total-value and breeding-value plans:",
↳ length(intersect(p_tv$selected$id, p_bv$selected$id)), "of", n_train, "\n")
#> lines in common between the total-value and breeding-value plans: 56 of 60

```

10 Training sets for every crop and breeding system

10.1 Wheat, rice, barley: inbred lines, several goals

```
goals <- c("accuracy", "gain", "information", "reference_panel", "correlation")
gres <- do.call(rbind, lapply(goals, function(gl) {
  p <- tso_design(tp, n = n_train, target = targets, goal = gl, iterations = 100, n_random
    ↪ = 20,
    verbose = FALSE)
  data.frame(goal = gl, criterion = p$spec$criterion, lines = nrow(p$selected),
    exact_CDmean = round(tso_score(tp, train = p$selected$id, target = targets,
    criteria = "cdmean_exact")["cdmean_exact"], 4))
}))
knitr::kable(gres, row.names = FALSE)
```

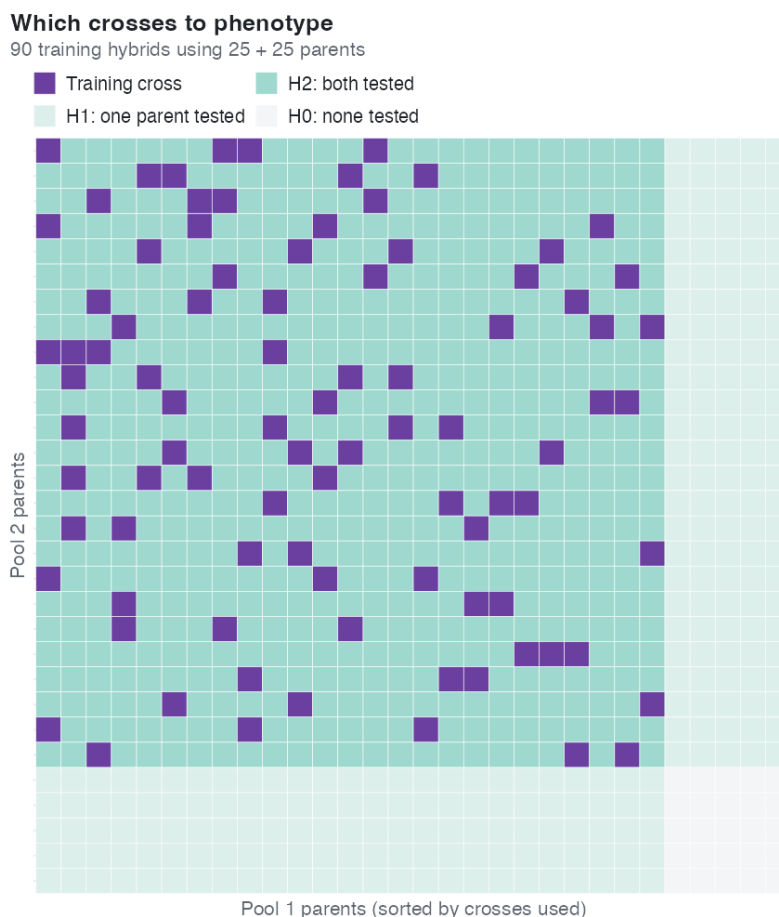
goal	criterion	lines	exact_CDmean
accuracy	cdmean	60	0.2437
gain	gain	60	0.2412
information	dopt	60	0.2404
reference_panel	cdmean	60	0.2437
correlation	rscore	60	0.2436

```
chk("every goal returns a full training set", all(gres$lines == n_train))
#> [PASS] every goal returns a full training set
```

10.2 Maize, sorghum: hybrids

```
p_hyb <- tso_design(tp_hyb, n = 3 * n_pool, max_per_parent = 6, verbose = FALSE)
p_hyb
#> TSOpt plan · hybrid · 90 hybrids of 625 candidates · untargeted (whole pool)
#> Criterion CDmean, exact (fixed intercept) (averaged over h2 = 0.2, 0.5, 0.8) Search
  ↪ greedy (robust CDmean: average over h2) (0.6 s)
#> Expected CDmean 0.449 (at h2 = 0.50) (exact, intercept projected: 0.449) vs random
  ↪ 0.419 ± 0.004 (better than 100% of 100 random sets)
#> Verdict AMBER modest gain (~3.5% accuracy over random); worth using, but do not
  ↪ expect much
#> Coverage nearest training relative >= 0.20 for 100% of targets · 0 targets unreachable
  ↪ (< 0.05)
#> Divide 0 candidates effectively unselectable (reliability < 0.06) · see
  ↪ tso_plot(plan, "divide")
#> Plot value dropping one plot costs 0.46% to 0.68% of the CDmean (median 0.55%) · see
  ↪ tso_plot(plan, "value")
#> Classes untested hybrids: H0 = 25, H1 = 250, H2 = 535
#> Next tso_plot(plan) · tso_export(plan, "trial.csv") · tso_explain(plan)
par_use <- table(c(p_hyb$selected$parent1, p_hyb$selected$parent2))
chk("no parent is used in more than 6 training hybrids", max(par_use) <= 6)
#> [PASS] no parent is used in more than 6 training hybrids
chk("only crossable parents appear in training",
  all(p_hyb$selected$parent1 %in% fem[1:(n_pool - 5)] & p_hyb$selected$parent2 %in%
    ↪ mal[1:(n_pool - 5)]))
#> [PASS] only crossable parents appear in training
p_h0 <- tso_design(tp_hyb, n = 3 * n_pool, predict = "H0", verbose = FALSE)
chk("predict = H0 targets hybrids of two untested parents", grepl("H0", p_h0$spec$target))
#> [PASS] predict = H0 targets hybrids of two untested parents
```

```
tso_plot(p_hyb, "hybrid_grid")
```



10.3 Potato (tetraploid) and cassava (diploid): clones

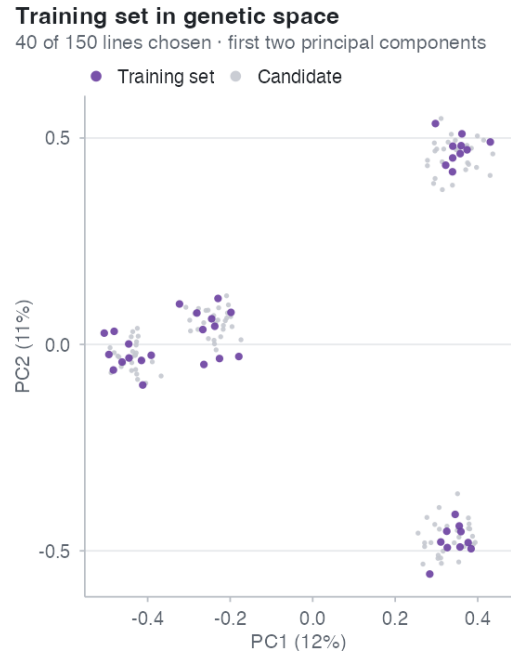
```
p_pot <- tso_design(tp_potato, n = 40, verbose = FALSE)
p_cas <- tso_design(tp_clone, n = n_train, verbose = FALSE)
c(potato = p_pot$value$cdmean, cassava = p_cas$value$cdmean)
#> potato cassava
#> 0.3787 0.3298
```

p_pot

```
#> TSOpt plan • clonal • 40 clones of 150 candidates • untargeted (whole pool)
#> Criterion CDmean, exact (fixed intercept) (averaged over h2 = 0.2, 0.5, 0.8) Search
  ↳ greedy (robust CDmean: average over h2) (0.0 s)
#> Expected CDmean 0.379 (at h2 = 0.50) (exact, intercept projected: 0.378) vs random
  ↳ 0.357 ± 0.003 (better than 100% of 100 random sets)
#> Verdict AMBER modest gain (~3.0% accuracy over random); worth using, but do not
  ↳ expect much
#> Structure discrete groups (4 groups, silhouette 0.74) -> stratified allocation
  ↳ 10/10/10/10
#> Coverage nearest training relative >= 0.20 for 100% of targets • 0 targets unreachable
  ↳ (< 0.05)
#> Divide 0 candidates effectively unselectable (reliability < 0.06) • see
  ↳ tso_plot(plan, "divide")
#> Alleles 100.0% of target alleles present in training (0 markers not covered) • 13.5%
  ↳ of target 5-marker haplotypes
#> Plot value dropping one plot costs 1.12% to 1.62% of the CDmean (median 1.30%) • see
  ↳ tso_plot(plan, "value")
```

```
#> Next      tso_plot(plan) · tso_export(plan, "trial.csv") · tso_explain(plan)
chk("tetraploid clonal design runs", nrow(p_pot$selected) == 40)
#> [PASS] tetraploid clonal design runs
```

```
tso_plot(p_pot, "pca")
```



```
chk("diploid clonal design runs", nrow(p_cas$selected) == n_train)
#> [PASS] diploid clonal design runs
```

10.4 Ryegrass, alfalfa: outcrossing families (CDpop)

```
p_pop <- tso_design(tp_out, n = n_train, goal = "families", verbose = FALSE)
chk("family-mean design (CDpop) runs", p_pop$spec$criterion == "cdpop")
#> [PASS] family-mean design (CDpop) runs
length(unique(p_pop$selected$family))
#> [1] 27
```

10.5 Genebank core collection

```
p_core <- tso_design(tp_bank, n = n_train, goal = "reference_panel", verbose = FALSE)
p_cov2 <- tso_design(tp_bank, n = n_train, criterion = "coverage", verbose = FALSE)
c(reference_panel = p_core$coverage$frac_020, coverage = p_cov2$coverage$frac_020)
#> reference_panel      coverage
#>           1           1
chk("coverage design reaches most lines with a relative >= 0.20", p_cov2$coverage$frac_020
  > 0.5)
#> [PASS] coverage design reaches most lines with a relative >= 0.20
```

10.6 Rice, multi-location: sparse multi-environment testing

Every line is tested in one site only (`max_per_line = 1`), the noisy Site2 gets fewer plots, and every site keeps at least 20.

```

p_met <- tso_design(tp_met, n = 3 * n_train, target = targets, max_per_line = 1,
  ↪ min_per_env = 20,
  ↪ verbose = FALSE)
p_met
#> TSOpt plan · inbred · 180 line x environment cells of 765 candidates · 45 target lines ·
  ↪ across-environment means
#> Criterion CDmean, exact (intercept + 2 fixed effects) (averaged over h2 = 0.2, 0.5,
  ↪ 0.8) Search greedy (robust CDmean: average over h2) (0.3 s)
#> Expected CDmean 0.196 (at h2 = 0.50) (exact, intercept projected: 0.195) vs random
  ↪ 0.142 ± 0.016 (better than 100% of 100 random sets)
#> Verdict GREEN optimisation pays off here: expected accuracy ~17.5% above random
#> Coverage nearest training relative >= 0.20 for 87% of targets · 0 targets unreachable
  ↪ (< 0.05)
#> Divide 0 candidates effectively unselectable (reliability < 0.06) · see
  ↪ tso_plot(plan, "divide")
#> Envs Site1: 102 · Site2: 23 · Site3: 55 · 180 lines with 1 env
#> Alleles 100.0% of target alleles present in training (0 markers not covered) · 97.8%
  ↪ of target 5-marker haplotypes
#> Plot value dropping one plot costs 0.07% to 5.05% of the CDmean (median 0.10%) · see
  ↪ tso_plot(plan, "value")
#> Next tso_plot(plan) · tso_export(plan, "trial.csv") · tso_explain(plan)
chk("every line is tested in one site only", all(table(p_met$selected$line_id) == 1))
#> [PASS] every line is tested in one site only
chk("every site gets at least 20 lines", all(p_met$envs$allocation >= 20))
#> [PASS] every site gets at least 20 lines
chk("target lines are never tested", !any(p_met$selected$line_id %in% targets))
#> [PASS] target lines are never tested
p_met_def <- tso_design(tp_met, n = 3 * n_train, target = targets, verbose = FALSE)
chk("without constraints every site is still opened", all(p_met_def$envs$allocation >= 2))
#> [PASS] without constraints every site is still opened

tso_plot(p_met, "met_allocation")

```

Sparse-testing allocation

180 plots in 3 environments · 180 of 300 lines tested

**10.7 Wheat quality: multi-trait index (which trait on which line)**

```
p_mt <- tso_design(tp_mt, budget = 40 * n_train, index = c(yield = 1, protein = 0.4, ndvi =
  ↪ 0),
                  verbose = FALSE)
p_mt$traits
#>   trait lines weight h2
#> 1  yield    61    1.0 0.3
#> 2 protein    62    0.4 0.5
#> 3  ndvi     62    0.0 0.6
chk("multi-trait budget is respected", p_mt$spent <= 40 * n_train + 1e-8)
#> [PASS] multi-trait budget is respected
```

10.8 Very large panels

```
big <- tso_simulate_panel(n = 3000, markers = 1500, seed = 5)
p_big <- timed(tso_design_large(big, n = 200, rank = 200, verbose = FALSE))
p_big
#> TSOpt plan (large panel) · 200 lines of 3,000 candidates · 3,000 lines x 1,435 markers ·
  ↪ untargeted (whole pool)
#> Features   randomised SVD (1 power iteration), rank 200, 78.5% of trace(K) captured (5
  ↪ MB, no N x N matrix)
#> Criterion  CDmean, exact (fixed intercept) (averaged over h2 = 0.2, 0.5, 0.8) on 3,000
  ↪ contrasts  Search greedy (robust CDmean: average over h2) (2.8 s)
#> Expected   CDmean 0.472 (at h2 = 0.50) (exact, intercept projected: 0.472) vs random
  ↪ 0.437 ± 0.001 (better than 100% of 20 random sets)
#> Verdict    AMBER modest gain (~3.9% accuracy over random); worth using, but do not
  ↪ expect much
```

```
#> Coverage nearest training relative >= 0.20 for 100% of targets · 0 targets unreachable  
↪ (< 0.05)  
#> Next plot(plan) · write.csv(plan$selected, "trial.csv")  
chk("large-panel engine returns 200 lines", nrow(p_big$selected) == 200)  
#> [PASS] large-panel engine returns 200 lines  
small <- tso_simulate_panel(n = 150, markers = 300, seed = 9)  
lr <- tso_design_large(small, n = 30, h2 = 0.5, rank = 300, verbose = FALSE)  
dn <- tso_design(small, n = 30, h2 = 0.5, search = "greedy", stratify = "none", verbose =  
↪ FALSE)  
chk("with exact features the large engine picks the same lines as the dense one",  
setequal(lr$selected$id, dn$selected$id))  
#> [PASS] with exact features the large engine picks the same lines as the dense one
```

11 Major genes and haplotypes

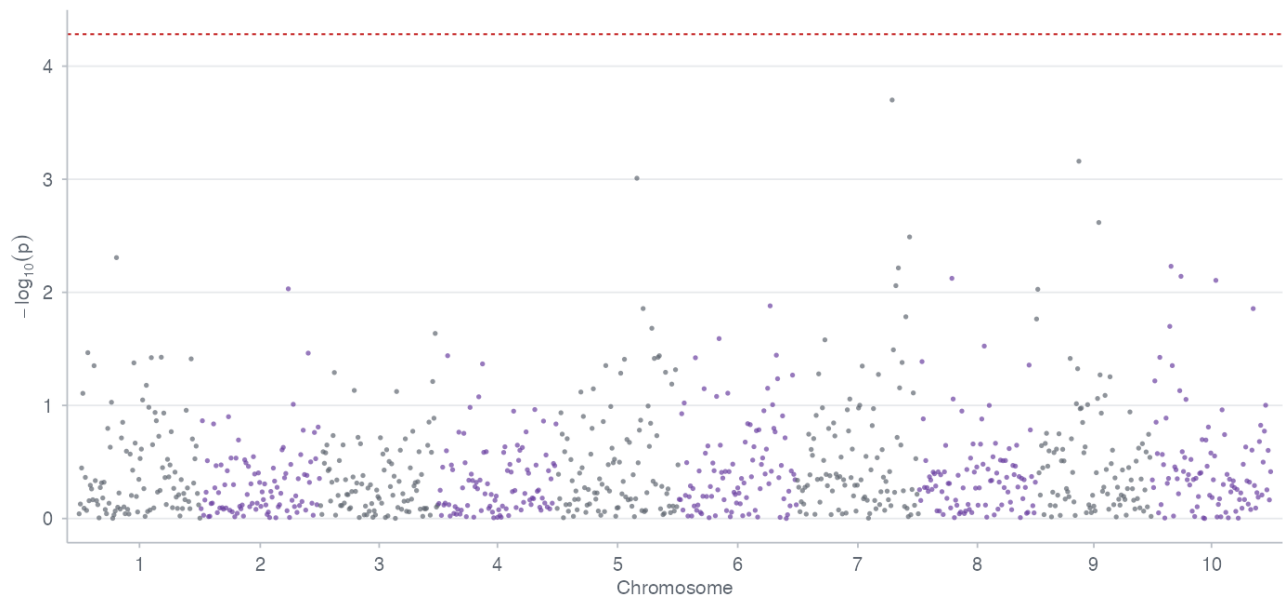
11.1 Scanning your phenotypes

```
ph <- utils::read.csv(pheno_file)
y <- stats::setNames(ph$y, as.character(ph$id))
chk("phenotype IDs match the genotypes", mean(names(y) %in% ids) > 0.9,
    paste0(round(100 * mean(names(y) %in% ids)), "% matched"))
#> [PASS] phenotype IDs match the genotypes (100% matched)
scan <- tso_major_scan(g, y)
#> TSOpt major-locus scan · 300 lines · 958 markers · mixed model (EMMAX) (h2 = 0.42)
#> Threshold -log10(p) >= 4.3
#> Major loci none: the trait looks polygenic, the genome-wide kernel is adequate
```

```
tso_plot(scan)
```

No major locus: a polygenic trait

mixed model (EMMAX) · 300 lines



11.2 A trait with a planted major gene

To check that the machinery works, a trait with one major QTL (40% of the genetic variance) is simulated on your genotypes:

```
tr_m <- tso_simulate_trait(g, h2 = 0.6, n_qtl = 80, major = 1, major_share = 0.4, seed = 11)
scan_m <- tso_major_scan(g, stats::setNames(tr_m$y, tr_m$id), verbose = FALSE)
scan_m
#> TSOpt major-locus scan · 300 lines · 958 markers · mixed model (EMMAX) (h2 = 0.52)
#> Threshold -log10(p) >= 4.3
#> Major loci 3: m855 (chr 9, -log10 p 15.1, 35% of genetic variance); m220 (chr 3, -log10
  ↪ p 5.7, 10% of genetic variance); m295 (chr 3, -log10 p 5.7, 9% of genetic variance)
#> Next tso_data(geno, major = scan) (uses a 71% variance share at these loci)
chk("the scan finds the planted major QTL", attr(tr_m, "major") %in% scan_m$leads$marker)
#> [PASS] the scan finds the planted major QTL
hp <- tso_haplotypes(g, around = attr(tr_m, "major"), window = 2)
hp
#> TSOpt haplotypes · 1 block(s), fixed 5-marker tiles · 300 lines · homozygous lines:
  ↪ strings are haplotypes
#> markers per block: median 5 (range 5-5) · haplotypes per block: median 19
```

```
#> hap_m855          5 markers · 19 haplotypes (17 with freq >= 2%) · top 14%
chk("haplotype block built around the major QTL", length(hp$blocks) == 1)
#> [PASS] haplotype block built around the major QTL
```

11.3 Marker effects you already have

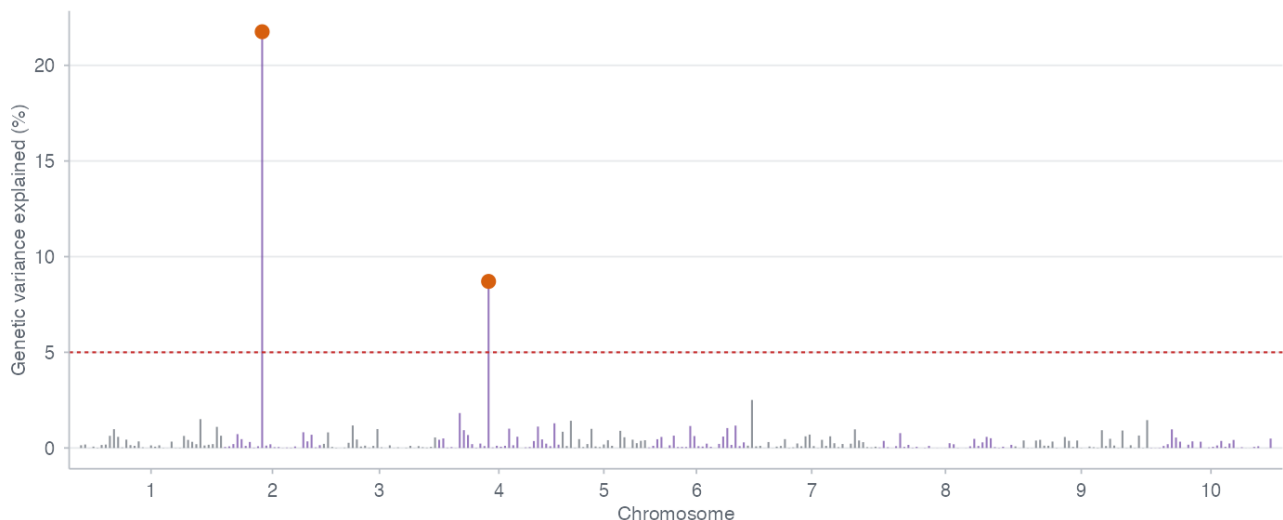
No phenotypes are needed when marker effects are known: from a GWAS, a QTL study, a BayesB model or cloned genes. `tso_major_effects()` matches them to your markers and alleles and computes each locus's share of genetic variance in your panel.

```
if (nzchar(eff_file)) {
  me <- tso_major_effects(g, eff_file)
  chk("marker effects matched to the genotypes", me$n_markers > 0,
      paste0(me$n_markers, " of ", me$n_given, " markers"))
  chk("each lead's share is a proportion", all(is.na(me$leads$share_region)) ||
      all(me$leads$share_region >= 0 & me$leads$share_region <= 1))
  if (nrow(me$leads)) {
    tp_me <- tso_data(g, major = me, qc = FALSE, verbose = FALSE)
    chk("tso_data() protects the loci from the effects", setequal(tp_me$major$loci,
        me$leads$marker))
  }
} else cat("No marker-effects file given (params$effects).\n")
#> TSOpt major loci from marker effects · 300 lines · 290 of 290 markers matched (1 effect
  ↳ alleles flipped)
#> Variance variance of the genomic values the effects predict here (0.120)
#> Major loci 2: m146 (chr 2, MAF 0.20, 22% of genetic variance); m348 (chr 4, MAF 0.23, 9%
  ↳ of genetic variance)
#> Next tso_data(geno, major = effects) (uses a 31% variance share at these loci)
#> [PASS] marker effects matched to the genotypes (290 of 290 markers)
#> [PASS] each lead's share is a proportion
#> [PASS] tso_data() protects the loci from the effects
```

`tso_plot(me)`

2 major locus/loci from the marker effects

290 marker effects · 300 lines



The same function reads GWAS summary statistics. The scan of the planted trait stands in for a published GWAS; the planted locus (40% of the genetic variance) must be found and recognised as major. Its share will be too high: a GWAS effect estimated on the same few hundred lines is inflated by the winner's curse (the Beavis effect), so the share from a small study is an upper bound.

Effects from a large published study, or the scan's joint estimate (share_joint), are better guides to the size.

```
gw <- data.frame(snp = scan_m$table$marker, beta = scan_m$table$effect, pval =
  ↪ 10^-scan_m$table$log10p)
me_gw <- tso_major_effects(g, gw, var_g = stats::var(tr_m$g))
#> TSOpt major loci from marker effects · 300 lines · 958 of 958 markers matched
#> Variance genetic variance given (1.000)
#> Major loci 3: m855 (chr 9, MAF 0.19, 65% of genetic variance); m220 (chr 3, MAF 0.24,
  ↪ 19% of genetic variance); m295 (chr 3, MAF 0.33, 19% of genetic variance)
#> Next tso_data(geno, major = effects) (uses a 80% variance share at these loci)
chk("GWAS effects find the planted major QTL", attr(tr_m, "major") %in% me_gw$leads$marker)
#> [PASS] GWAS effects find the planted major QTL
sh_gw <- me_gw$leads$share_genetic[me_gw$leads$marker == attr(tr_m, "major")][1]
chk("the planted QTL is recognised as a major locus (share above 20%)", isTRUE(sh_gw >
  ↪ 0.2),
  paste0(round(100 * sh_gw), "% from the GWAS effect; scan's joint estimate ",
    round(100 * scan_m$leads$share_joint[scan_m$leads$marker == attr(tr_m,
      ↪ "major")][1]), "%"))
#> [PASS] the planted QTL is recognised as a major locus (share above 20%) (65% from the
  ↪ GWAS effect; scan's joint estimate 35%)
```

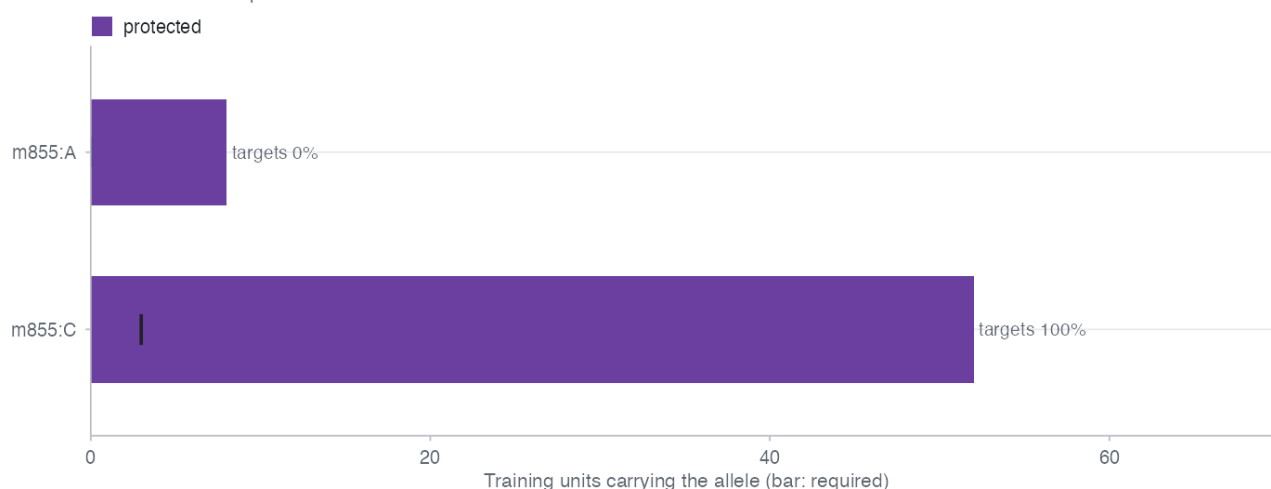
11.4 Protecting the alleles in the training set

```
tp_major <- tso_data(g, major = attr(tr_m, "major"), min_carriers = 3, qc = FALSE, verbose
  ↪ = FALSE)
p_major <- tso_design(tp_major, n = n_train, target = targets, verbose = FALSE)
p_major$major
#> class locus freq_candidates freq_targets carriers_training required
#> 1 m855:A m855 0.2235 0 8 0
#> 2 m855:C m855 0.7765 1 52 3
chk("every allele class carried by targets has the required carriers",
  all(p_major$major$carriers_training >= p_major$major$required))
#> [PASS] every allele class carried by targets has the required carriers
tp_fixed <- tso_data(g, major = list(markers = attr(tr_m, "major"), haplotypes = hp),
  major_model = "fixed", qc = FALSE, verbose = FALSE)
p_fixed <- tso_design(tp_fixed, n = n_train, target = targets, verbose = FALSE)
chk("the fixed model uses the major loci as fixed effects",
  ↪ !is.null(p_fixed$spec$fixed_columns))
#> [PASS] the fixed model uses the major loci as fixed effects

tso_plot(p_major, "major")
```

Major-locus alleles in the training set

1 loci • 2 of 2 allele classes protected

**11.5 Genome-wide allele and haplotype coverage**

```

unlist(plan_t$alleles[c("frac", "n_missing_markers")])
#>      frac n_missing_markers
#>      1      0
plan_t$alleles$haplotypes$frac
#> [1] 0.9693
chk("allele coverage is a proportion", plan_t$alleles$frac >= 0 && plan_t$alleles$frac <=
  1)
#> [PASS] allele coverage is a proportion

```

11.6 Haplotype blocks, the haplotype kernel and haplotype coverage

Blocks of adjacent markers in LD (or fixed tiles, or windows in kb) define haplotypes. Your material is homozygous, so the marker strings are haplotypes.

```

hp_ld <- tso_haplotypes(g, method = "ld")
hp_fx <- tso_haplotypes(g, method = "fixed", window = 2)
hp_ld
#> TSOpt haplotypes • 958 block(s), LD blocks (r2 >= 0.3, <= 20 markers) • 300 lines •
  homozygous lines: strings are haplotypes
#> markers per block: median 1 (range 1-1) • haplotypes per block: median 2
#> chr1_1      1 markers • 2 haplotypes (2 with freq >= 2%) • top 66%
#> chr1_2      1 markers • 2 haplotypes (2 with freq >= 2%) • top 64%
#> chr1_3      1 markers • 2 haplotypes (2 with freq >= 2%) • top 83%
#> chr1_4      1 markers • 2 haplotypes (2 with freq >= 2%) • top 77%
#> chr1_5      1 markers • 2 haplotypes (2 with freq >= 2%) • top 61%
#> chr1_6      1 markers • 2 haplotypes (2 with freq >= 2%) • top 93%
#> ... 952 more blocks
chk("LD blocks use every marker exactly once", setequal(unlist(hp_ld$blocks),
  colnames(g$dosage)) &&
  !anyDuplicated(unlist(hp_ld$blocks)))
#> [PASS] LD blocks use every marker exactly once
chk("haplotype frequencies sum to 1 in every block", all(abs(vapply(hp_fx$freq, sum, 1) -
  1) < 1e-10))
#> [PASS] haplotype frequencies sum to 1 in every block
hp1 <- tso_haplotypes(g, method = "fixed", window = 0, min_freq = 0)
H1 <- TSOpt:::.kernel_hap(hp1)
g1 <- g # haplotypes use rounded dosages
#> (homozygous lines: 0 or ploidy)

```

```

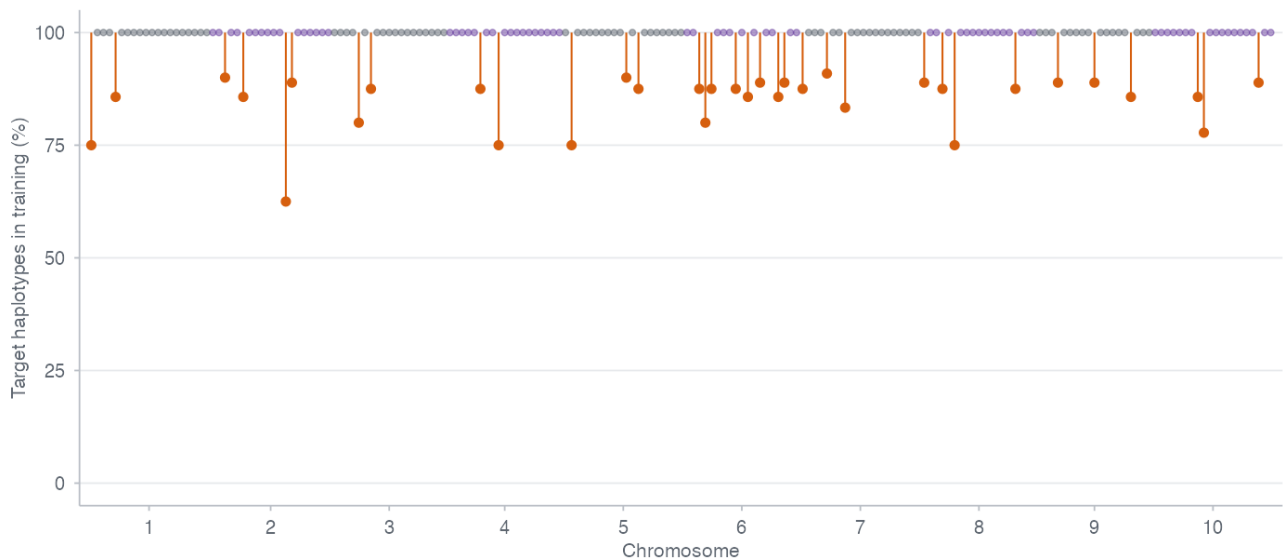
g1$dosage <- if (hp1$kind == "homozygous") g$ploidy * round(round(g$dosage) / g$ploidy)
  ↳ else round(g$dosage)
A1 <- tso_kinship(g1, base_freq = colMeans(g1$dosage) / g$ploidy, ridge = 0)
chk("single-marker blocks reproduce the VanRaden matrix exactly", max(abs(H1 - A1 /
  ↳ mean(diag(A1)))) < 1e-8)
#> [PASS] single-marker blocks reproduce the VanRaden matrix exactly
tp_h <- tso_data(g, haplotypes = hp_fx, qc = FALSE, verbose = FALSE)
p_def <- tso_design(tp_h, n = n_train, target = targets, verbose = FALSE)
p_hap <- tso_design(tp_h, n = n_train, target = targets, criterion = "haplotype", verbose =
  ↳ FALSE)
cov_tab <- rbind(default = unlist(p_def$alleles$haplotypes[c("frac", "frac_copies",
  ↳ "frac2")])),
  haplotype = unlist(p_hap$alleles$haplotypes[c("frac", "frac_copies",
    ↳ "frac2")])))
round(cov_tab, 3)
#>          frac frac_copies frac2
#> default   0.969         0.987 0.916
#> haplotype 0.975         0.992 0.930
chk("the haplotype criterion covers at least as many target haplotype copies",
  cov_tab["haplotype", "frac_copies"] >= cov_tab["default", "frac_copies"] - 1e-12)
#> [PASS] the haplotype criterion covers at least as many target haplotype copies
chk("the per-block report has one row per block", nrow(p_hap$alleles$haplotypes$per_block)
  ↳ == length(hp_fx$blocks))
#> [PASS] the per-block report has one row per block

```

```
tso_plot(p_def, "haplotypes")
```

97% of the targets' haplotypes are in the training set

195 blocks · 34 blocks miss a target haplotype (marked) · 99% of target haplotype copies covered · 92% with ≥ 2 carriers



12 Checking a plan

12.1 Why this set: tso_explain()

```
tso_explain(plan_t, steps = FALSE)
#> How this plan was made
#> System      inbred
#> Goal         accuracy -> CDmean: the mean coefficient of determination of target
#>   ↳ contrasts (each target minus the target mean),  $CD_i = 1 - c_i' A c_i / c_i' K c_i$ 
#> Heritability robust: the criterion is averaged over  $h_2 = 0.2, 0.5, 0.8$  (nobody knows)
#>   ↳  $h_2$  before the trial)
#> Targets      45 named targets (45 units)
#> Candidates    255 units
#> Size          n = 60
#> Constraints   group_quota = 10/10/10/30
#> Structure     discrete groups -> stratified
#> Search        greedy (robust CDmean: average over  $h_2$ )
#> Fixed effects intercept handled exactly (flat prior)
#> Reproduce     seed = 1, TSOpt 0.5.0
```

12.2 Any set, any criterion: tso_score()

```
set.seed(2); colleague <- sample(setdiff(ids, targets), n_train)
rbind(colleague = tso_score(tp, train = colleague, target = targets),
      optimised = tso_score(tp, train = plan_t$selected$id, target = targets))
#>          cdmean cdmean_exact pevmean dopt
#> colleague 0.08786      0.08673  0.7384 37.30
#> optimised 0.24426      0.24372  0.6112 36.81
chk("the optimised set scores higher than a random colleague's set",
    tso_score(tp, train = plan_t$selected$id, target = targets)[["cdmean"]] >
    tso_score(tp, train = colleague, target = targets)[["cdmean"]])
#> [PASS] the optimised set scores higher than a random colleague's set
```

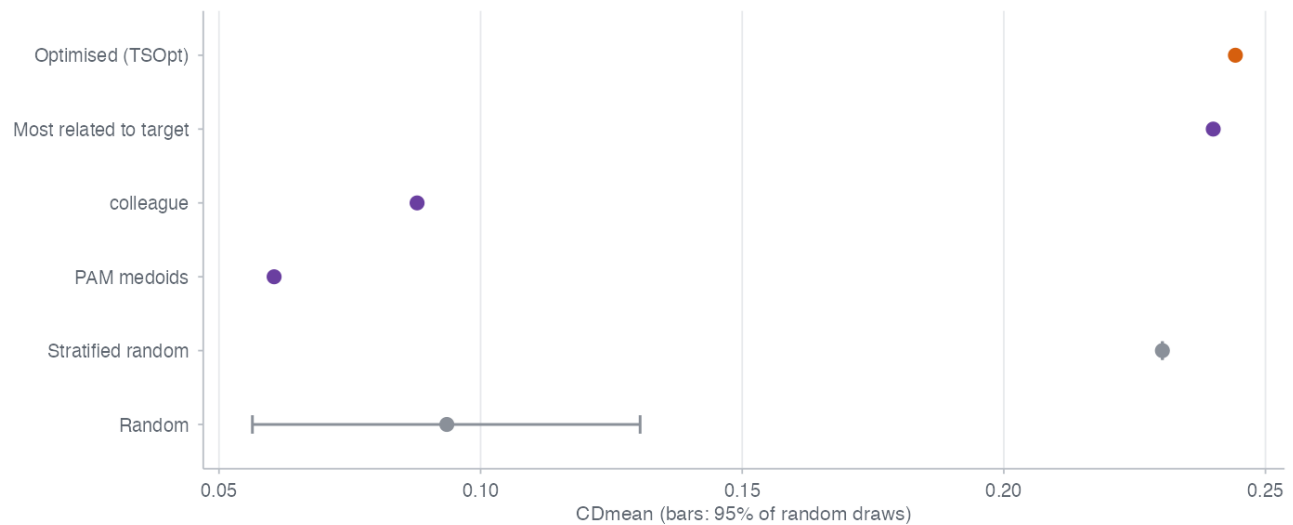
12.3 Against the alternatives: tso_compare()

```
cmp <- tso_compare(plan_t, others = list(colleague = colleague), n_random = 50)
cmp
#> TSOpt comparison · 60 units each · target: 45 named targets
#> Method          CDmean vs random Targets >= 0.20 Unreachable Unselectable
#> Optimised (TSOpt) 0.244 61.6%      100%          0.00          0
#> Random (mean)     0.094 0.0%       93%           0.08          0
#> Stratified random (mean) 0.230 56.9% 100%          0.00          0
#> Most related to target 0.240 60.2% 100%          0.00          0
#> PAM medoids       0.061 -19.6%    89%           0.00          0
#> colleague         0.088 -3.1%     100%          0.00          0
#> 'vs random' is on the accuracy scale (sqrt CDmean). Plot with tso_plot(comparison).
chk("the plan is at least as good as random sampling",
    cmp$table$cdmean[cmp$table$method == "optimised"] >= cmp$table$cdmean[cmp$table$method
  ↳ == "random"])
#> [PASS] the plan is at least as good as random sampling
```

```
tso_plot(cmp)
```

The plan against the alternatives

60 units per method; higher is better

**12.4 Top-k identification: tso_topk()**

```
tk <- tso_topk(plan_t, top = 0.1, n_sim = 300)
tk
#> TSOpt top-k · 60 training units → rank 45 units · select top 10% (k = 5) · h2 = 0.50
#>
#>      optimised [90% int.]      random [90% int.]
#> Top-k overlap      0.318 [0.000, 0.800]      0.227 [0.000, 0.600]
#> NDCG@5            0.760 [0.528, 0.933]      0.690 [0.452, 0.885]
#> Realised gain (SD) 0.759 [0.005, 1.435]      0.461 [-0.367, 1.239]
#> Correlation        0.453 [0.089, 0.722]      0.272 [-0.097, 0.608]
#> → Of the 5 lines you select, on average 1.6 are truly in the top 5 (random training
  ↳ sets: 1.1); chance alone gives 0.6.
#> Simulation          300 draws (Cholesky) · 20 random sets x 60 draws
chk("top-k analysis reports plan and random-set metrics", is.data.frame(tk$summary) &&
  ↳ nrow(tk$summary) > 0)
#> [PASS] top-k analysis reports plan and random-set metrics
```

```
tso_plot(tk)
```

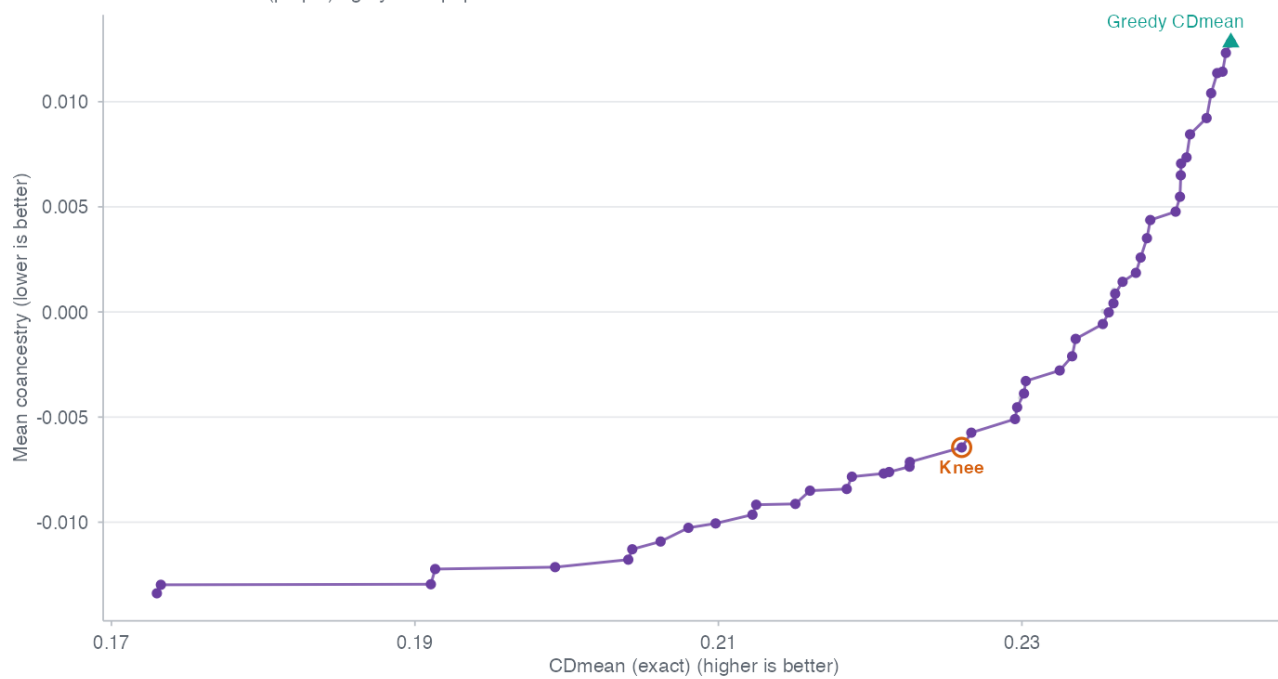
Finding the top 10%: 32% overlap vs 23% for random sets60 training units ranking 45 units · $h^2 = 0.50$ · 300 simulated traits**12.5 Accuracy against diversity: tso_pareto()**

```
fr <- tso_pareto(tp, n = n_train, objectives = c("cdmean", "coancestry"), target = targets,
  pop = 50, generations = 50, verbose = FALSE)
fr
#> TSOpt Pareto front · 50 non-dominated training sets of 60 from 255 candidates · 45 named
  ↳ targets
#> cdmean      [max]      0.173 – 0.244 (CDmean (exact))
#> coancestry  [min]     -0.0134 – 0.0128 (Mean coancestry)
#> Knee        set 30: cdmean 0.226 · coancestry -0.00644 (farthest from the line between
  ↳ the extremes)
#> Greedy seed cdmean 0.244 · coancestry 0.0128 (on the front)
#> Search      NSGA-II, pop 50, 50 generations, 2,557 sets evaluated (0.9 s) · hypervolume
  ↳ 0.927
#> Next        plot(front) · tso_pareto_pick(front) · tso_pareto_pick(front, "best_cdmean")
knee <- tso_pareto_pick(fr, "knee")
chk("the knee set has n_train lines", length(knee) == n_train)
#> [PASS] the knee set has n_train lines
chk("the front has more than one set", nrow(fr$front) > 1)
#> [PASS] the front has more than one set

tso_plot(fr)
```

Pareto front: CDmean (exact) vs Mean coancestry

50 non-dominated sets of 60 (purple) · grey: final population



13 After the season: evaluate the plan

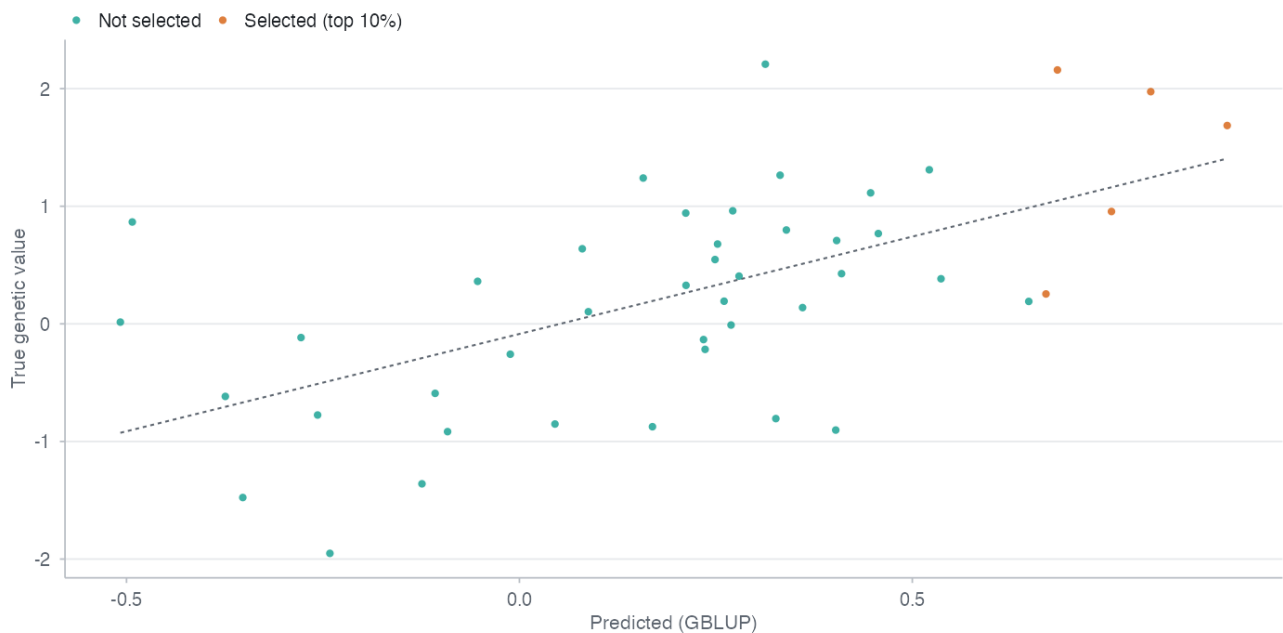
Once the training set is phenotyped, `tso_evaluate()` fits GBLUP on it and reports how well it predicted the targets: predictive ability, rank correlation, NDCG, realised gain from selecting the top 10% and the bias slope. With true genetic values (simulations) the metrics are computed against the truth.

```
tv <- if (nzchar(truth_file) && file.exists(truth_file)) {
  d <- utils::read.csv(truth_file); stats::setNames(d$g, as.character(d$id)) } else NULL
ev <- tso_evaluate(plan_t, y, truth = tv, h2 = params$h2)
ev
#> TSOpt evaluation · 60 training units -> 45 targets · against true genetic values
#> Predictive ability      0.593 (same season: optimistic; forward prediction is usually
  ↳ lower)
#> Rank correlation        0.588
#> NDCG@10%              0.883   top 10% overlap 60%
#> Realised gain          1.21 SD above the target mean (selecting the top 10%)
#> Bias slope             1.66 (1 = unbiased; > 1 = predictions shrunk)
#> Heritability used      0.50 (given)
chk("predictive ability is a correlation", abs(ev$predictive_ability) <= 1)
#> [PASS] predictive ability is a correlation
chk("evaluation never scores the training lines", !any(ev$predictions$id %in%
  ↳ plan_t$selected$id))
#> [PASS] evaluation never scores the training lines
set.seed(3)
rnd <- sample(setdiff(ids, targets), n_train)
ev_r <- tso_evaluate(tp, y, train = rnd, target = targets, truth = tv, h2 = params$h2)
c(optimised = ev$predictive_ability, random = ev_r$predictive_ability)
#> optimised      random
#>      0.5933      0.5245
```

`tso_plot(ev)`

Predictive ability 0.59 · realised gain 1.21 SD

60 training units predicting 45 targets



13.1 Real accuracy of every criterion

Section 6 compared the criteria on an expected quantity. Here each criterion's training set is phenotyped (the phenotypes are in y) and GBLUP predicts the targets. Against the true genetic values when they are known (simulated data), otherwise against the targets' own phenotypes. This is the measure that matters to a breeder.

```
real <- do.call(rbind, lapply(names(crit_sets), function(cr) {
  e <- tso_evaluate(tp, y, train = crit_sets[[cr]], target = targets, truth = tv, h2 =
    ↪ params$h2)
  data.frame(criterion = cr, predictive_ability = e$predictive_ability, rank_corr =
    ↪ e$spearman,
              ndcg = e$ndcg, realised_gain_sd = e$realised_gain)
}))
real <- real[order(-real$predictive_ability), ]
knitr::kable(real, digits = 3, row.names = FALSE)
```

criterion	predictive_ability	rank_corr	ndcg	realised_gain_sd
coverage	0.766	0.750	0.925	1.334
pevmax	0.637	0.595	0.927	1.428
pam	0.627	0.618	0.711	0.758
pevmean	0.596	0.599	0.871	1.205
cdmean	0.594	0.573	0.871	1.205
cdmin	0.584	0.582	0.848	1.027
haplotype	0.576	0.542	0.947	1.428
eopt	0.575	0.570	0.869	1.192
rscore	0.575	0.570	0.869	1.192
kinship	0.560	0.563	0.844	1.205
aopt	0.546	0.535	0.858	1.013
stratified	0.543	0.522	0.895	1.205
gain	0.540	0.533	0.854	1.013
dopt	0.535	0.478	0.921	1.428
cdpop	0.523	0.505	0.894	1.232
reliability	0.502	0.503	0.845	1.013
random	0.491	0.426	0.859	1.327
maximin	0.078	0.177	0.437	-0.348

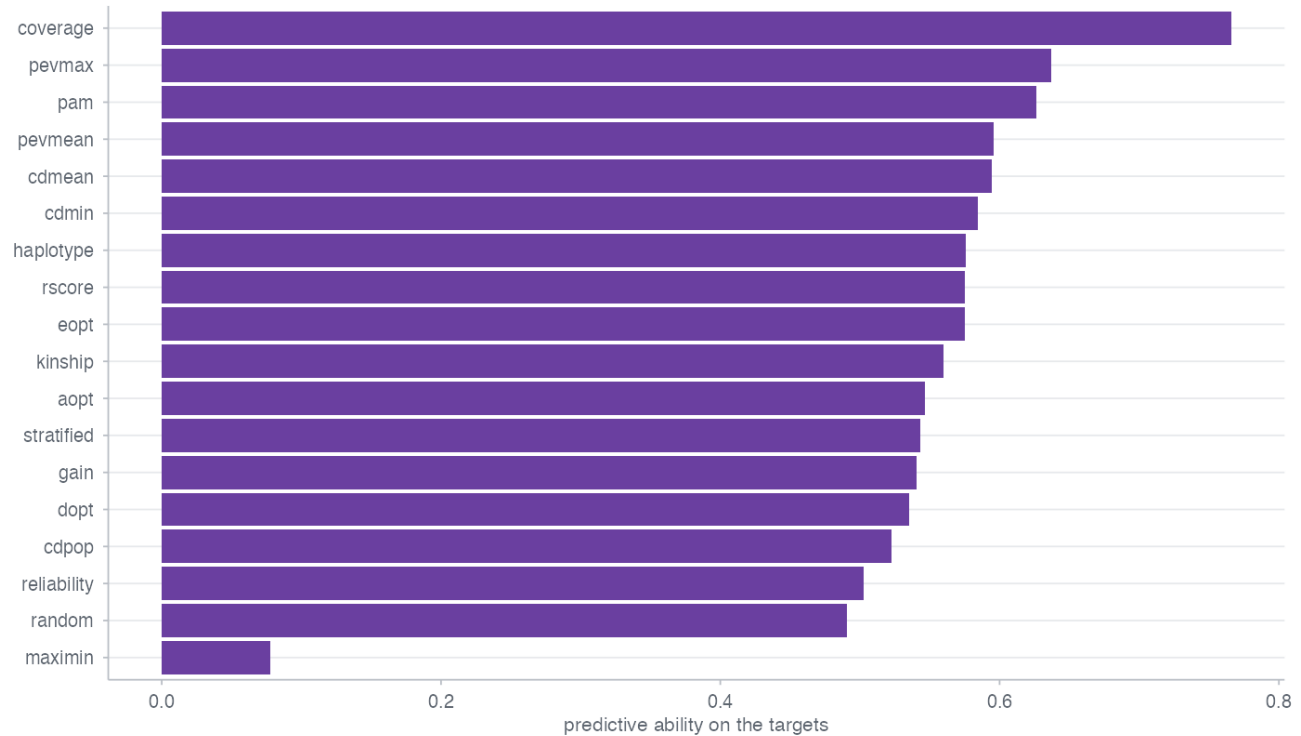
```
chk("every criterion's set can be evaluated", nrow(real) == length(crit_sets))
#> [PASS] every criterion's set can be evaluated
set.seed(5)
rand_pa <- mean(replicate(10, tso_evaluate(tp, y, train = sample(setdiff(ids, targets),
  ↪ n_train),
                                target = targets, truth = tv, h2 =
                                ↪ params$h2)$predictive_ability))
cat("mean predictive ability of 10 random training sets:", round(rand_pa, 3), "\n")
#> mean predictive ability of 10 random training sets: 0.339
chk("the default cdmean predicts at least as well as the average random set",
    real$predictive_ability[real$criterion == "cdmean"] >= rand_pa - 0.02)
#> [PASS] the default cdmean predicts at least as well as the average random set
```

What to look at: one panel and one trait give a noisy ranking (a single random set can be lucky), which is why the package default rests on a benchmark over six panels and 33 scenarios. On your own data, a criterion near the bottom of this table and near the top of the Section 6 table has been optimising the wrong quantity for this trait.

```
ggplot(real, aes(stats::reorder(criterion, predictive_ability), predictive_ability)) +
  geom_col(fill = tso_colors()[["training"]]) + coord_flip() + tso_theme() +
```

```
labs(x = NULL, y = "predictive ability on the targets", title = "Realised accuracy of
↳ every criterion")
```

Realised accuracy of every criterion



13.2 Unreliable heritability estimates

Without a heritability, REML estimates it from the training set. With few lines the estimate can hit its boundary, and TSOpt then warns that the bias slope and realised gain are unreliable:

```
ev_reml <- withCallingHandlers(tso_evaluate(plan_t, y, truth = tv), warning = function(w) {
  cat("TSOpt warns:", substr(conditionMessage(w), 1, 100), "...\\n");
  ↳ invokeRestart("muffleWarning") })
ev_reml$h2_used
#> [1] 0.1821
```

14 Across stages, seasons and generations

14.1 Second stage: tso_next_batch()

```

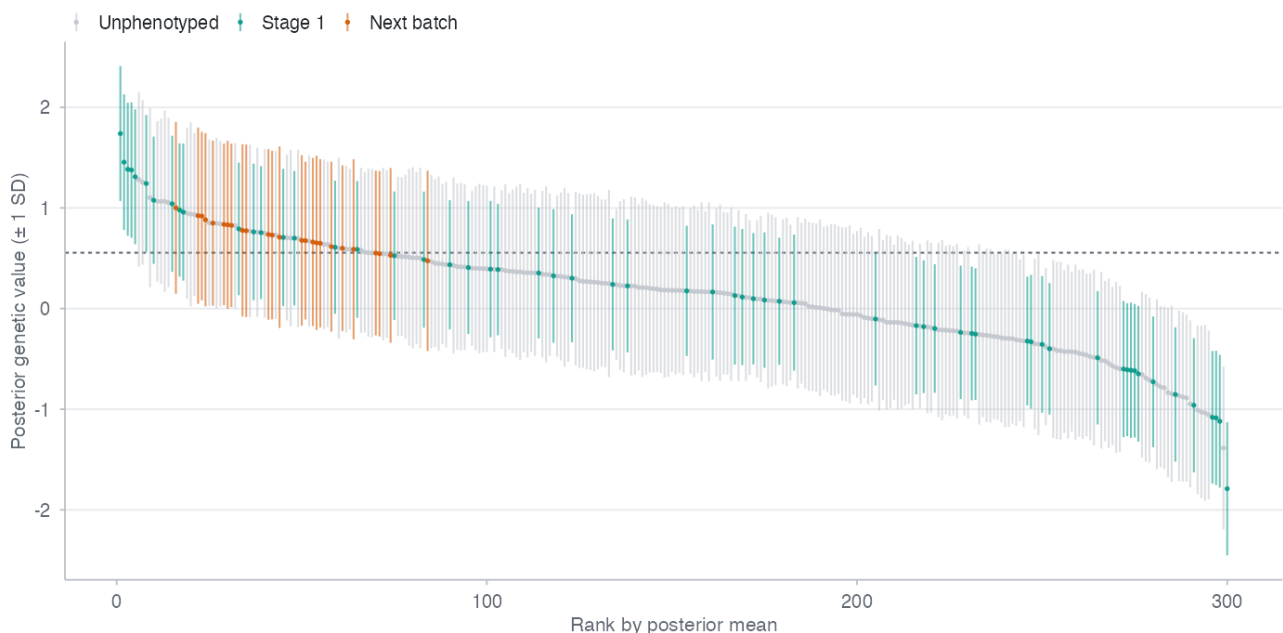
y1 <- y[intersect(plan$selected$id, names(y))]
nb <- tso_next_batch(plan, y1, n = 25, select_top = 0.1, h2 = params$h2)
nb
#> TSOpt next batch · 25 plots · selecting the top 10% (30 of 300) · 60 stage-1 observations
#> Posterior      h2 = 0.50 (given)
#> Batch          25 new units, 0 re-tests of promising phenotyped units
#> Boundary       112 units have a 10-90% chance of being selected: these decide where plots
  ↪ go
#> Next           tso_plot(batch) · batch$selected · batch$posterior
chk("next batch has 25 plots", nrow(nb$selected) == 25)
#> [PASS] next batch has 25 plots
chk("posterior probabilities of selection lie in [0, 1]",
    all(nb$posterior$p_top >= 0 & nb$posterior$p_top <= 1, na.rm = TRUE))
#> [PASS] posterior probabilities of selection lie in [0, 1]

```

tso_plot(nb)

Where the next plots go

Dashed: selection threshold (top 10%) · plots go where the ranking is uncertain



14.2 Next season: tso_update()

```

p_next <- tso_update(plan, data = tp, n = 40, history_weight = 0.8, verbose = FALSE)
p_next$spec$history
#> [1] 60
chk("last season's lines become history, not new plots", !any(plan$selected$id %in%
  ↪ p_next$selected$id))
#> [PASS] last season's lines become history, not new plots

```

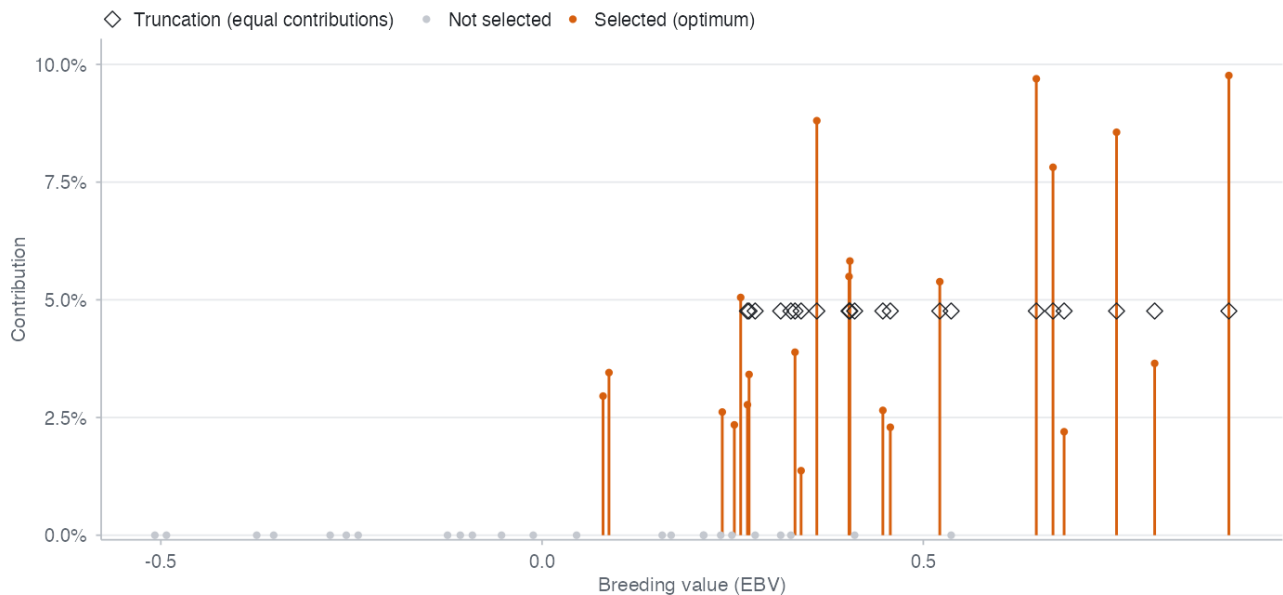
14.3 Choosing parents: tso_contribution()

```
ebv <- stats::setNames(ev$predictions$predicted, ev$predictions$id)
ocs <- tso_contribution(ebv, tp$K_lines[names(ebv), names(ebv)], delta_F = 0.01)
ocs
#> TSOpt optimum contributions · 45 candidates ·  $\Delta F \leq 0.01 \rightarrow$  group coancestry  $\leq 0.1082$ 
#> Optimum          21 parents · mean EBV of offspring 0.505 · group coancestry 0.1082 ·
  ↪ expected  $\Delta F$  0.0100
#> Truncation       top 21 by EBV, equal contributions · mean EBV 0.482 · group coancestry
  ↪ 0.1177 ·  $\Delta F$  0.0205
#> Status           coancestry limit binding ·  $\lambda = 7.906$ 
#> Parents          L266 9.8%, L297 9.7%, L275 8.8%, L295 8.6%, L281 7.8%, L279 5.8%, L293
  ↪ 5.5%, L287 5.4%, L290 5.1%, L277 3.9%, ... (11 more)
chk("contributions sum to 1", abs(sum(ocs$contributions$contribution) - 1) < 1e-8)
#> [PASS] contributions sum to 1
chk("contributions are non-negative", all(ocs$contributions$contribution >= -1e-12))
#> [PASS] contributions are non-negative
chk("the coancestry limit is respected", ocs$coancestry <= ocs$limit + 1e-8)
#> [PASS] the coancestry limit is respected
```

tso_plot(ocs)

Optimum contributions: 21 parents, mean EBV 0.50 vs 0.48 by truncation

Group coancestry 0.1082 (limit 0.1082) vs 0.1177 for the top 21 with equal contributions



14.4 The long term: tso_simulate()

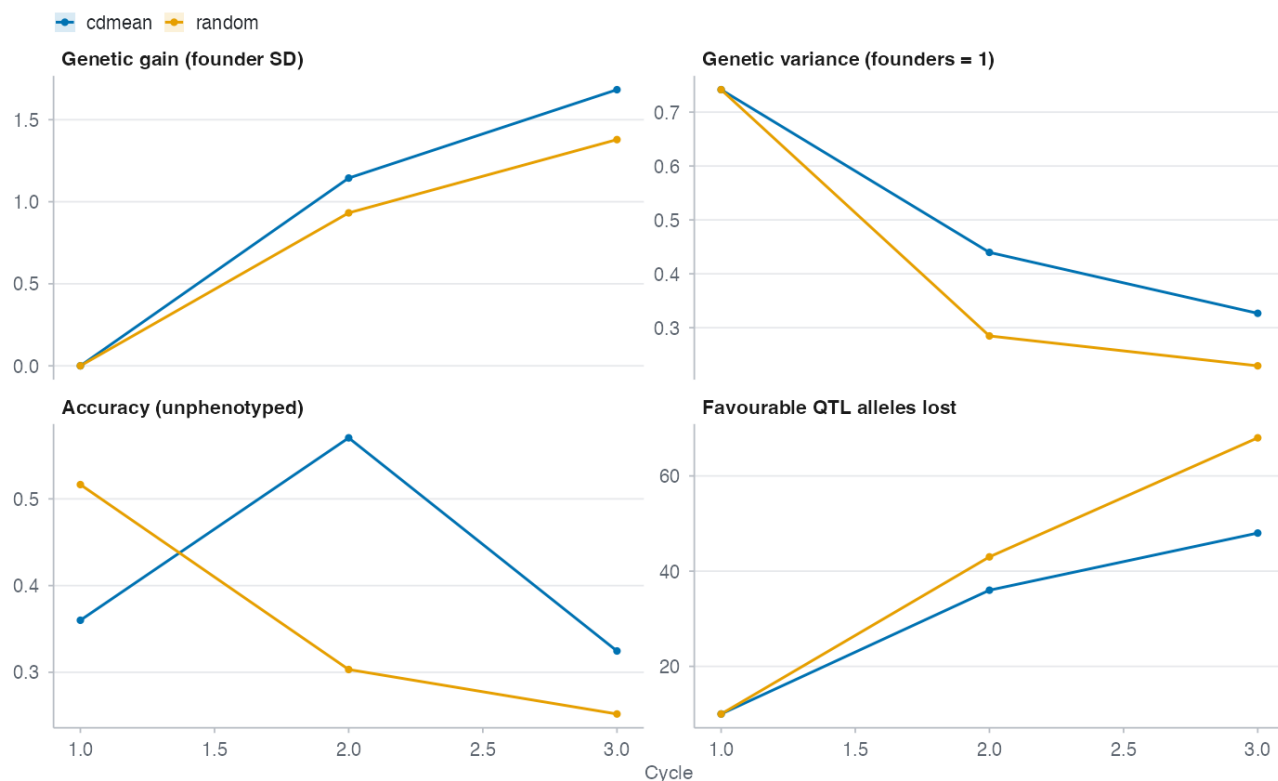
```
sim <- tso_simulate(strategies = list(cdmean = "cdmean", random = "random"), cycles = 3,
  ↪ reps = 1,
  n_candidates = 120, n_train = 40, n_parents = 12, markers = 500, verbose
  ↪ = FALSE)
sim
#> TSOpt long-term simulation · 3 cycles · 1 replicates · 40 of 120 candidates phenotyped,
  ↪ 12 parents
#> Strategy Gain (founder SD) Genetic variance (vs founders) Mean accuracy
#> cdmean    1.68              0.33              0.42
#> random    1.38              0.23              0.36
#> Favourable QTL lost Coancestry of parents
```

```
#> 48.0          0.52
#> 68.0          0.69
#> Values at the last cycle, averaged over replicates (accuracy averaged over cycles).
chk("the simulation reports every cycle and strategy", nrow(sim$results) == 3 * 2)
#> [PASS] the simulation reports every cycle and strategy
```

```
tso_plot(sim)
```

Long-term consequences of the training-set strategy

1 replicates · mean ± 1 SE · 40 of 120 phenotyped per cycle



15 Decisions around the design

15.1 Traits and environments together, hedged over uncertain correlations

A unit is now one trait on one line in one environment. The design hedges the index over uncertain heritabilities and genetic correlations, and prints the CDmean under every scenario.

```
g150 <- g; g150$dosage <- g$dosage[seq_len(min(150, N)), ]
tp_mte <- tso_data(g150, traits = c("yield", "protein", "ndvi"), trait_h2 = c(0.3, 0.5,
  ↪ 0.6),
  gcor = matrix(c(1, .5, .6, .5, 1, .3, .6, .3, 1), 3),
  trait_cost = c(yield = 20, protein = 35, ndvi = 0.5), plot_cost = 10,
  envs = c("Site1", "Site2"), env_cor = matrix(c(1, .6, .6, 1), 2),
  env_noise = c(Site1 = 1, Site2 = 2), qc = FALSE, verbose = FALSE)
p_mte <- tso_design(tp_mte, budget = 2500, index = c(yield = 1, protein = 0.4), verbose =
  ↪ FALSE)
p_mte
#> TSOpt plan · inbred · 166 line x trait cells of 900 candidates · untargeted (all lines),
  ↪ index yield=1 + protein=0.4 + ndvi=0 · averaged over 2 environments
#> Criterion index CDmean, exact (intercept + 5 fixed effects) (trait heritabilities from
  ↪ the data) Search greedy (robust over 5 trait heritability / correlation scenarios)
  ↪ (1.0 s)
#> Expected CDmean 0.326 (exact, intercept projected: 0.322) vs random 0.251 ± 0.008
  ↪ (better than 100% of 100 random sets)
#> Scenarios CDmean given 0.326 · h2_lower 0.239 · h2_higher 0.400 · gcor_weaker 0.265 ·
  ↪ gcor_stronger 0.411
#> Verdict GREEN optimisation pays off here: expected accuracy ~13.9% above random
#> Coverage nearest training relative >= 0.20 for 100% of targets · 0 targets unreachable
  ↪ (< 0.05)
#> Divide 0 candidates effectively unselectable (reliability < 0.06) · see
  ↪ tso_plot(plan, "divide")
#> Traits yield: 64 lines · protein: 6 lines · ndvi: 96 lines
#> Envs Site1: 124 · Site2: 42 · 96 lines with 1 env
#> Plot value dropping one plot costs 0.09% to 1.07% of the CDmean (median 0.21%) · see
  ↪ tso_plot(plan, "value")
#> Budget spent 2498.0 of 2500.0
#> Next tso_plot(plan) · tso_export(plan, "trial.csv") · tso_explain(plan)
table(p_mte$selected$trait, p_mte$selected$env)
#>
#>      Site1 Site2
#> ndvi      62   34
#> protein    4    2
#> yield     58    6
chk("line x trait x environment units", nrow(tp_mte$units) == nrow(g150$dosage) * 6)
#> [PASS] line x trait x environment units
chk("the multi-trait budget across sites is respected", p_mte$spent <= 2500 + 1e-8)
#> [PASS] the multi-trait budget across sites is respected
chk("the robust multi-trait design reports every scenario",
  ↪ length(p_mte$value$cd_by_scenario) >= 4)
#> [PASS] the robust multi-trait design reports every scenario
chk("every trait is measured at least twice (trait means are estimable)",
  all(table(factor(p_mte$selected$trait, levels = c("yield", "protein", "ndvi")) >= 2))
#> [PASS] every trait is measured at least twice (trait means are estimable)
```

15.2 What is each trait worth?

```
tp_tv <- tso_data(g, traits = c("yield", "protein", "ndvi"), trait_h2 = c(0.3, 0.5, 0.6),
  gcor = matrix(c(1, .5, .6, .5, 1, .3, .6, .3, 1), 3),
```

```

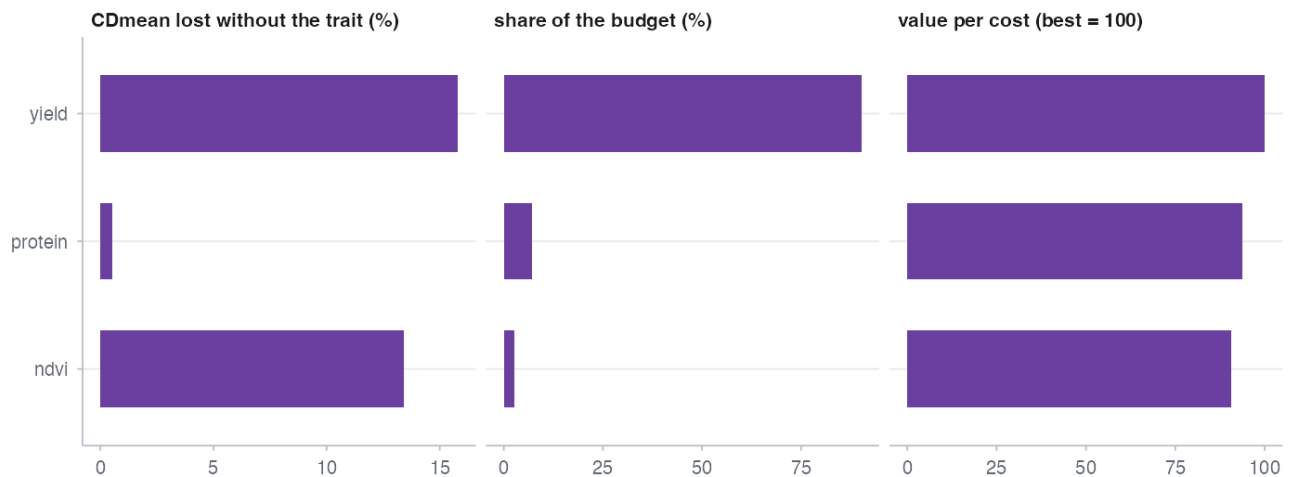
      trait_cost = c(yield = 20, protein = 35, ndvi = 0.5), plot_cost = 10, qc =
        FALSE,
        verbose = FALSE)
p_tv <- tso_design(tp_tv, budget = 3000, index = c(yield = 1, protein = 0.4), verbose =
  FALSE)
tv <- tso_trait_value(p_tv)
tv
#> TSOpt trait value · 3 traits · index CDmean of the plan 0.345 · exchange rates in yield
  measurements
#>   trait weight cost lines budget value per cost worth 1 ref CDmean lost without it
#>   yield    1.0 20.0   88   90%          1.00          1.0          15.8%
#>   protein   0.4 35.0    4    7%          0.94          0.7          0.5%
#>   ndvi     0.0 0.5  104    3%          0.91          2.6          13.4%
#> -> yield gives the most information per unit of cost at the margin; ndvi (index weight
  0) is measured as a proxy.
chk("budget shares sum to 1", abs(sum(tv$table$share_of_budget) - 1) < 1e-8)
#> [PASS] budget shares sum to 1
chk("the reference trait has exchange rate 1", abs(tv$table$exchange_rate[tv$table$trait
  == "yield"] - 1) < 1e-8)
#> [PASS] the reference trait has exchange rate 1

```

tso_plot(tv)

What each trait is worth in the design

index CDmean 0.345 · exchange rates in yield measurements



15.3 The value of each plot

```

knitr::kable(utils::head(plan_t$selected[order(-plan_t$selected$value_pct), c("id",
  "value_pct")], 5),
  digits = 2, row.names = FALSE)

```

id	value_pct
L234	5.61
L244	4.25
L238	4.04
L226	3.91
L227	3.20

```

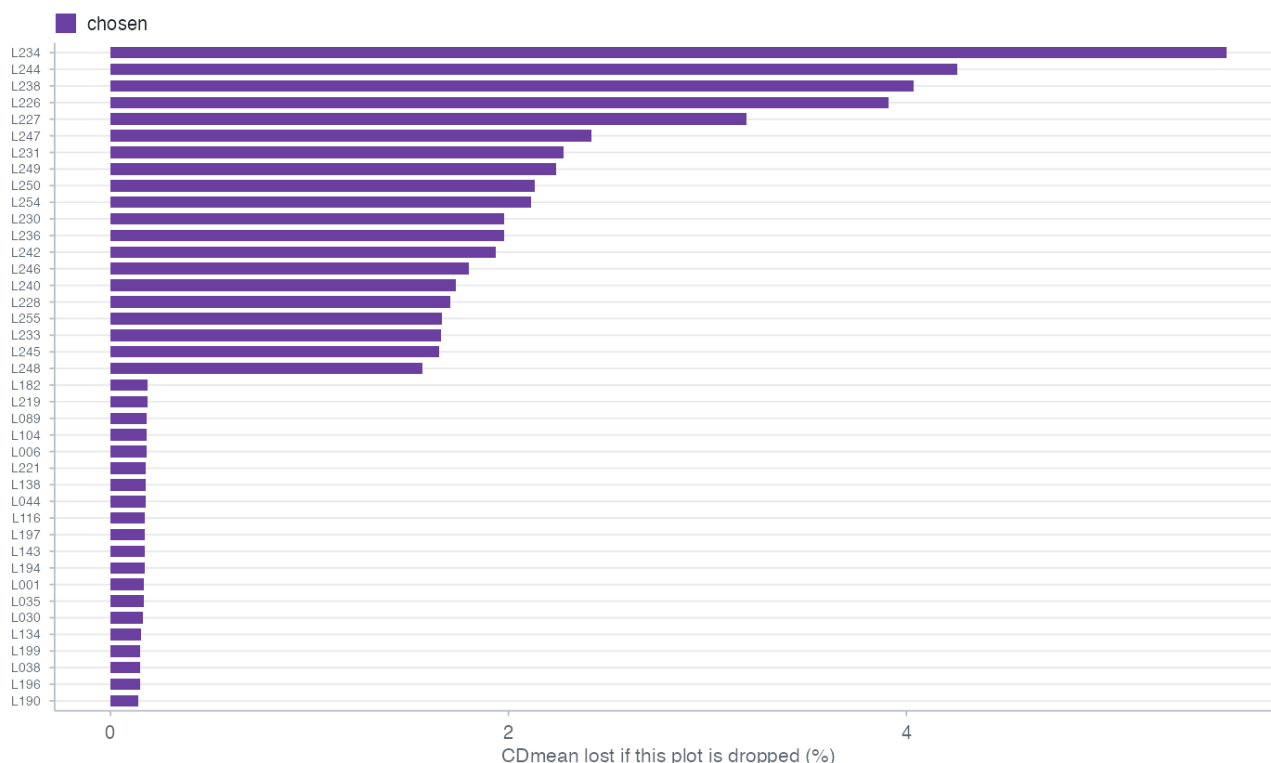
K_t <- tp$K; Ke_t <- K_t + 1e3 * mean(diag(K_t)); tix <- match(targets, ids)
ctr_t <- TSOpt:::.contrasts(K_t, tix, "centered")
s2_t <- rep(max((1 - p_h$spec$h2) / p_h$spec$h2, 1e-6), N)
cdv <- function(S) { st <- TSOpt:::.engine_init(Ke_t, ctr_t); for (j in S)
  ↪ TSOpt:::.engine_add(st, j, s2_t[j])
  TSOpt:::.crit_value("cdmean", st) }
S_h <- p_h$index$selected
chk("plot value equals the brute-force loss of dropping the plot (first 5 plots)",
    max(abs(sapply(1:5, function(i) cdv(S_h) - cdv(S_h[-i])) - p_h$selected$value[1:5])) <
    ↪ 1e-8)
#> [PASS] plot value equals the brute-force loss of dropping the plot (first 5 plots)

```

```
tso_plot(plan_t, "value")
```

The value of each plot

the 20 most and 20 least valuable of 60 plots · drop from the bottom when seed or land runs short



15.4 How sure is the advantage? tso_uncertainty()

```

unc <- tso_uncertainty(plan_t, B = 30)
unc
#> TSOpt uncertainty · marker bootstrap, 30 replicates of 958 markers · 20 random sets each
  ↪ (0.6 s)
#>          estimate [lower, upper] (90% intervals; estimate on all markers)
#> Plan CDmean    0.244 [0.239, 0.250]
#> Random mean    0.084 [0.077, 0.091]
#> Gain (CD)      189.8% [175.4%, 203.4%]
#> Gain (acc.)    71.3% [66.4%, 75.9%] (plan reported 63.3%)
#> Beats random   100% of replicates
#> Verdict        ROBUST: the 90% interval of the accuracy gain lies above 2%: the
  ↪ advantage holds whichever markers are used
chk("the uncertainty interval contains the estimate",

```

```
all(unc$intervals$lower <= unc$intervals$estimate & unc$intervals$estimate <=
  ↪ unc$intervals$upper))
#> [PASS] the uncertainty interval contains the estimate
```

```
tso_plot(unc)
```

Advantage over random: robust

90% interval 66.4% to 75.9% · plan beats random in 100% of 30 marker bootstraps



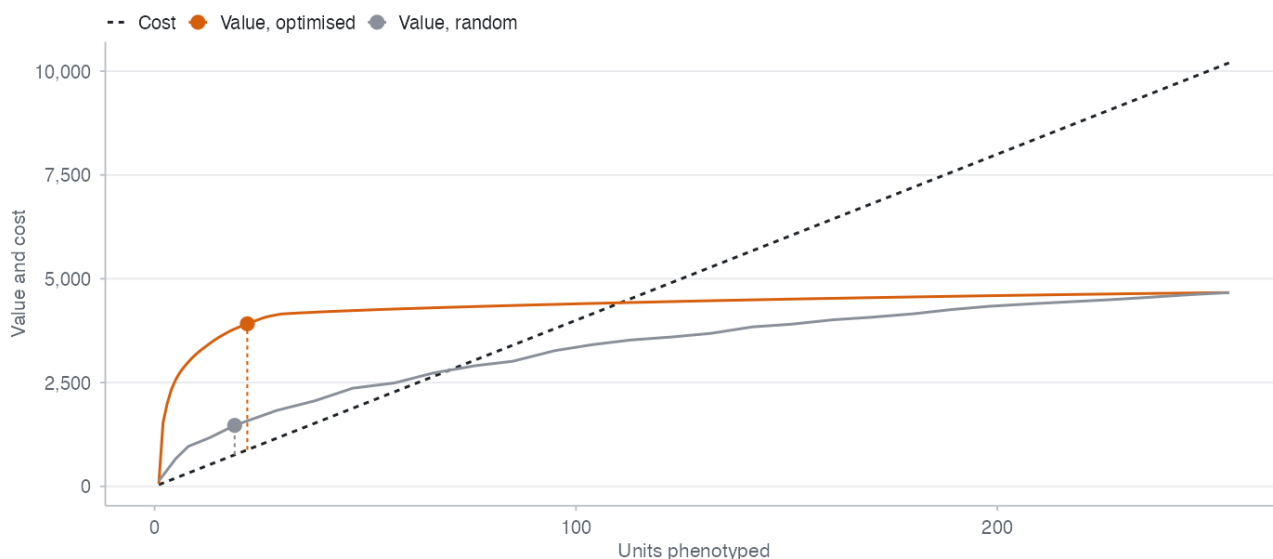
15.5 How many plots pay off? tso_economics()

```
eco <- tso_economics(tp, cost_per_plot = 40, value_per_sd = 5000, select_top = 0.1, target
  ↪ = targets)
eco
#> TSOpt economics · 255 candidates · select top 10% (i = 1.75) · averaged over h2 = 0.2,
  ↪ 0.5, 0.8
#> Prices      40 per plot · fixed 0 · 5,000 per genetic SD · 1 cycle(s)
#> → Phenotype 22 lines: net value 3,036; each extra plot now returns 38.7 against a cost
  ↪ of 40 (the last one returned 40.1)
#>
#>          n      accuracy  value      net      break-even
#>    optimised  22      0.446   3,916   3,036           1
#>    random    19      0.167   1,469    709           1
#> Advantage optimised design is worth 2,327 more at its optimum (it pays to phenotype 3
  ↪ more plots)
#> Method      exact greedy accuracy curve (optimised) · mean of 20 random sets at 30 sizes,
  ↪ interpolated (random)
chk("the economic optimum maximises net value on the curve",
  abs(eco$optimum$opt$net - max(eco$curve$net_opt, na.rm = TRUE)) < 1e-6)
#> [PASS] the economic optimum maximises net value on the curve
```

```
tso_plot(eco)
```

Phenotype 22 lines: net value 3,036

Dashed segments: net value at each design's optimum (optimised 22, random 19) · advantage 2,327

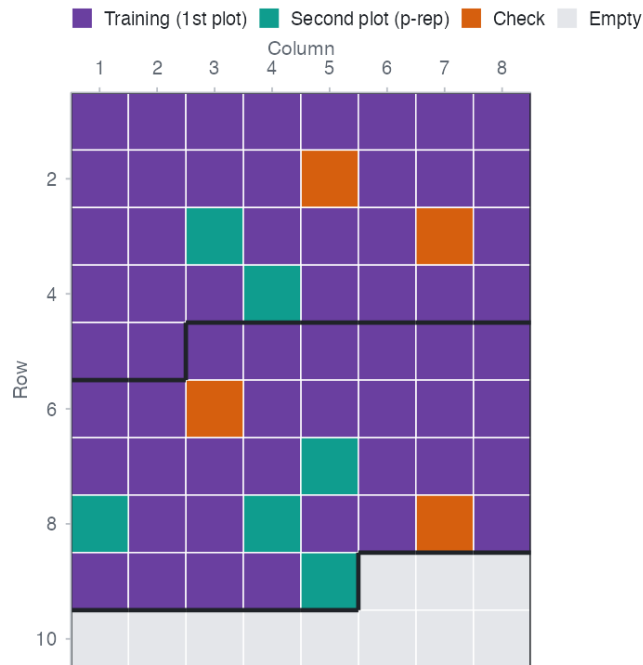
**15.6 Laying out the field: tso_layout()**

```
lay <- tso_layout(plan_t, rows = 10, cols = ceiling((n_train + 6) / 10) + 1, blocks = 2,
  checks = ids[1:2], check_reps = 1, prep = 0.1)
lay
#> TSOpt field layout · 10 x 8 plots · 2 blocks (serpentine runs (row bands))
#> Plots      59 lines + 6 second plots (p-rep) + 4 check plots (2 checks) · 69 of 80 plots
  ↳ used, 11 empty
#> Blocks      34/35 entries per block · balanced over 4 strata (plan strata)
#> Neighbours mean kinship of neighbouring plots 0.025 -> -0.089 after 20,000 annealing
  ↳ steps; close relatives (>= 0.5) adjacent: 1 -> 0
#> Expected CDmean 0.266 with block effects fixed · 0.267 ignoring blocks (cost of
  ↳ blocking 0.0010)
#> p-rep      second plots chosen by plan$selected$value
#> Next       plot(layout) · as.data.frame(layout) for the field book
fld <- as.data.frame(lay)
chk("every training line is planted", all(plan_t$selected$id %in% fld$id))
#> [PASS] every training line is planted
chk("blocking never raises the CDmean", lay$metrics$cd_blocks <= lay$metrics$cd_no_blocks
  ↳ + 1e-8)
#> [PASS] blocking never raises the CDmean
```

```
tso_plot(lay)
```

Field layout

2 blocks · neighbour kinship 0.025 → -0.089 · CDmean 0.266 (blocks fixed)

**15.7 Whom to genotype, and hedging over the model of gene action**

```

gs <- tso_genotype_set(tp, n = 20)
gs
#> TSOpt genotyping set · 20 of 300 candidates · all 300 individuals · relationships from
  ↳ the genotypes
#> Method      coverage (facility location)    (0.0 s)
#> → Genotype these 20: 95% of the others then have a genotyped relative at ≥ 0.25
  ↳ (grandparent, half sib or closer) and 25% one at ≥ 0.5 (parent, full sib)
#> Coverage    mean best relationship 0.424 · median 0.415 · 10% of targets below 0.277
#> Compared with coverage mean best ≥ 0.25 ≥ 0.5
#> Optimised    0.462    0.424    95%    25%
#> Most connected 0.366    0.321    71%    13%
#> Random (mean) 0.386    0.342    78%    11%
#> Random      the set beats 100% of 50 random sets of the same size
#> First       L127, L185, L023, L234, L193, L128, L296, L030, ...
chk("the genotyped set beats random sets on coverage",
    gs$comparison$mean_best[1] >= gs$comparison$mean_best[nrow(gs$comparison)] - 1e-8 ||
    gs$percentile >= 0.5)
#> [PASS] the genotyped set beats random sets on coverage
p_kern <- tso_design(tp, n = n_train, target = targets, kernels = c("additive",
  ↳ "gaussian"), h2 = params$h2,
    verbose = FALSE)
round(p_kern$value$cd_by_scenario, 4)
#> additive gaussian
#> 0.2442 0.1091
chk("the kernel-robust design reports every kernel", length(p_kern$value$cd_by_scenario)
  ↳ == 2)
#> [PASS] the kernel-robust design reports every kernel

```

15.8 Training in one programme, predicting another: tso_transfer()

```
if (!is.null(groups) && length(unique(tp$units$group)) >= 2) {
  grp_names <- sort(unique(tp$units$group))
  tr_out <- tso_transfer(tp, from = grp_names[-length(grp_names)], to =
    ↪ grp_names[length(grp_names)],
    n = n_train, n_local = 10)
  tr_out
  chk("transferability is a ratio between 0 and 1", tr_out$transferability > 0 &&
    ↪ tr_out$transferability <= 1.05)
} else cat("No groups: transfer skipped.\n")
#> [PASS] transferability is a ratio between 0 and 1
```

15.9 Straight from a breeding database (BrAPI)

With a real server you connect and read genotypes and phenotypes directly:

```
con <- tso_brapi_connect("https://my-breedbase.org", token = Sys.getenv("BRAPI_TOKEN"))
tso_brapi_studies(con)
g_db <- tso_brapi_genotype(con, variant_set = "vs1")
y_db <- tso_brapi_trait(con, study = "trial2025", trait = "grain yield")
tp_db <- tso_data(g_db)
```

Here the connector is checked offline against the recorded BrAPI responses that ship with the package's tests, when the repository is available:

```
fx <- normalizePath(file.path("../..", "tests", "testthat", "fixtures", "brapi"),
  ↪ mustWork = FALSE)
if (dir.exists(fx)) {
  con <- tso_brapi_connect(paste0("file://", fx), verbose = FALSE)
  g_db <- tso_brapi_genotype(con, verbose = FALSE)
  g_db
  chk("BrAPI genotypes are read into a tso_genotype", inherits(g_db, "tso_genotype") &&
    ↪ nrow(g_db$dosage) > 0)
} else cat("BrAPI fixtures not found (run from the repository): offline check skipped.\n")
#> BrAPI fixtures not found (run from the repository): offline check skipped.
```

16 Programme planning and hand-over

16.1 Variance components from your phenotypes: `tso_variance()`

```
v_y <- tso_variance(tp, data.frame(id = names(y), y = y))
v_y
#> TSOpt variance components (AI-REML) · single · 300 records on 300 lines · logL -259.84 ·
  ↳ converged in 4 iterations
#> heritability: y 0.42 (SE 0.10)
#> 5 scenarios drawn from the sampling distribution. Next: tso_data(geno, variance = v)
chk("REML converged", isTRUE(v_y$converged))
#> [PASS] REML converged
chk("the heritability is a proportion", all(v_y$h2 > 0 & v_y$h2 < 1))
#> [PASS] the heritability is a proportion
chk("the same estimate as the GBLUP fit", abs(v_y$h2 - TSOpt:::gblup(tp$K_lines,
  ↳ match(names(y)[names(y) %in% rownames(tp$K_lines)],
    rownames(tp$K_lines)), y[names(y) %in% rownames(tp$K_lines)])$h2) < 0.01)
#> [PASS] the same estimate as the GBLUP fit
tp_v <- tso_data(g, variance = v_y, qc = FALSE, verbose = FALSE)
chk("robust designs use the estimated heritability grid", length(tp_v$h2_grid) >= 1 &&
  all(tp_v$h2_grid > 0 & tp_v$h2_grid < 1))
#> [PASS] robust designs use the estimated heritability grid
```

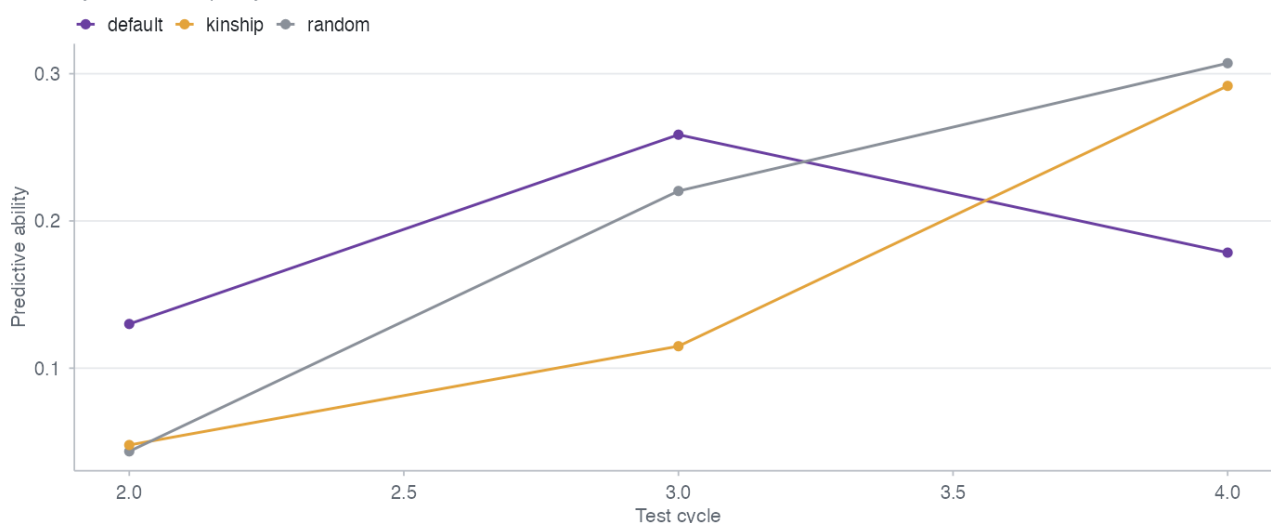
16.2 Forward validation: `tso_backtest()`

```
bt_hist <- if (nzchar(params$history)) utils::read.csv(params$history, stringsAsFactors =
  ↳ FALSE) else NULL
if (!is.null(bt_hist)) {
  bt <- tso_backtest(tp, bt_hist, n = 0.2, traits = params$history_trait, methods =
  ↳ c("random", "default", "kinship"),
    n_random = 3, verbose = FALSE)
} else {
  hs <- tso_simulate_cycles(cycles = 4, n_candidates = 100, n_crosses = 10, n_parents = 10,
  ↳ markers = 500, seed = 4)
  bt <- tso_backtest(hs$geno, hs$pheno, n = 0.25, methods = c("random", "default",
  ↳ "kinship"), n_random = 3,
    verbose = FALSE)
}
bt
#> TSOpt forward validation · within_cycle · 3 test cycles x 1 trait(s) · n = 0.25 per
  ↳ cycle (1.8 s)
#> method PA rank top10 PA vs random (95% CI) · wins
#> random 0.190 0.196 0.194 reference
#> default 0.189 0.201 0.208 -0.001 (-0.282 to +0.279) · 67% of 3
#> kinship 0.151 0.169 0.208 -0.039 (-0.184 to +0.106) · 33% of 3
#> Predictive ability within each test cycle, from data available at the time. plot(bt)
  ↳ shows every cycle.
chk("every test cycle and method is scored", nrow(bt$results) ==
  ↳ length(unique(bt$results$cycle)) * 3)
#> [PASS] every test cycle and method is scored
chk("predictive abilities are correlations", all(abs(bt$results$pa) <= 1))
#> [PASS] predictive abilities are correlations
```

```
tso_plot(bt)
```

Forward validation: accuracy in each cycle, from the data available at the time

within_cycle · n = 0.25 per cycle



16.3 The season across stages: tso_season()

```
sea <- tso_season(tp, budget = round(0.6 * N), n1 = unique(round(N * c(0.1, 0.2, 0.3))),
                 k1 = unique(pmax(5, round(N * c(0.03, 0.06)))), sites2 = 1:3, reps = 10,
                 verbose = FALSE)

sea
#> TSOpt season plan · 300 candidates · budget 180 plots · 18 allocations x 10 simulated
#>   seasons (0.4 s)
#>   one plot: h2 0.30, GxE 0.50 x genetic variance · stage 1 at 1 site(s) x 1 rep(s),
#>   TSOpt training set
#>   Best: stage 1 = 90 lines phenotyped (90 plots) · advance 18 to 3 site(s) (54 plots) ·
#>   select 5
#>   expected gain 1.74 genetic SD (SE 0.20) · 14% of the true top 5 selected ·
#>   stage-1 accuracy 0.46
#>   8 other allocation(s) not clearly worse (paired difference within 2 SE; see $table)
chk("every allocation fits the budget", all(sea$table$cost <= round(0.6 * N)))
#> [PASS] every allocation fits the budget
chk("the best allocation has the highest expected gain", sea$best$gain ==
    max(sea$table$gain))
#> [PASS] the best allocation has the highest expected gain
```

16.4 Which crosses to make: tso_crosses()

```
has_map <- !is.null(g$map$cm) || params$cm_per_mb > 0
if (has_map) {
  parents <- names(sort(y[names(y) %in% ids], decreasing = TRUE))[1:10]
  cr <- tso_crosses(g, parents = parents, pheno = y, cm_per_Mb = if (params$cm_per_mb > 0)
    params$cm_per_mb else NULL)
  cr
  chk("45 crosses among 10 parents", nrow(cr$table) == 45)
  i10 <- stats::dnorm(stats::qnorm(0.9)) / 0.1
  chk("usefulness = mean + i x SD", max(abs(cr$table$usefulness - cr$table$mean - i10 *
    cr$table$sd)) < 1e-10)
} else cat("No genetic map: give params$cm_per_mb (map density of your crop) to predict
  crosses.\n")
#> No genetic map: give params$cm_per_mb (map density of your crop) to predict crosses.
```

16.5 Genotyping, imputation and plots: tso_imputation_plan()

```
M <- ncol(g$dosage)
ipl <- tso_imputation_plan(tp, budget = 40 * N, cost_hd = 40, cost_ld = 12, plot_cost = 25,
  n_ref = unique(round(N * c(0.1, 0.25))), ld_markers =
    ↪ unique(round(M * c(0.05, 0.2))))

ipl
#> TSOpt genotyping, imputation and phenotyping plan · 300 lines x 958 HD markers · budget
  ↪ 12,000 (HD 40, LD 12, plot 25) · 4 plans (2.3 s)
#> plan
#> 75 HD + 225 LD (192 markers)      5,700      252      0.763      0.939      0.560      0.432
#> 30 HD + 270 LD (192 markers)      4,440      270      0.646      0.897      0.584      0.407
#> 30 HD + 270 LD (48 markers)       4,440      270      0.514      0.702      0.682      0.295
#> 75 HD + 225 LD (48 markers)       5,700      252      0.557      0.724      0.566      0.260
#> Best: 75 HD + 225 LD (192 markers), 252 plots
#> realised = reliability of GBLUP with the imputed relationships, on the true ones;
  ↪ expected = what the design believes.
chk("every plan fits the budget", all(ipl$stable$genotyping + 25 * ipl$stable$plots <= 40 *
  ↪ N))
#> [PASS] every plan fits the budget
chk("realised reliability equals the expected one when nothing is imputed",
  with(ipl$stable[ipl$stable$plan == "all high density", , drop = FALSE],
    !length(reliability) || abs(reliability - cdmean_expected) < 1e-10))
#> [PASS] realised reliability equals the expected one when nothing is imputed
```

16.6 Testcrosses and the factorised form on your panel

```
tcd <- tso_data(g, system = "testcross", testers = ids[1:2], qc = FALSE, verbose = FALSE)
chk("testcross units are lines x testers", nrow(tcd$units) == (N - 2) * 2)
#> [PASS] testcross units are lines x testers
ptc <- tso_design(tcd, n = min(40, N - 2), max_per_line = 1, verbose = FALSE)
chk("the testcross plan predicts GCA", identical(ptc$spec$value, "gca"))
#> [PASS] the testcross plan predicts GCA
sub <- ids[seq_len(min(80, N))]; gs <- g; gs$dosage <- g$dosage[sub, ]
e0 <- tso_data(gs, envs = c("E1", "E2", "E3"), qc = FALSE, verbose = FALSE)
e1 <- tso_data(gs, envs = c("E1", "E2", "E3"), qc = FALSE, implicit = TRUE, verbose = FALSE)
pe0 <- tso_design(e0, n = 30, h2 = 0.5, verbose = FALSE); pe1 <- tso_design(e1, n = 30, h2
  ↪ = 0.5, verbose = FALSE)
chk("the factorised form gives the same design as the dense one",
  identical(pe0$selected$id, pe1$selected$id) && abs(pe0$value$cdmean -
    ↪ pe1$value$cdmean) < 1e-10)
#> [PASS] the factorised form gives the same design as the dense one
```

16.7 Omics profiles: tso_omics_kernel() and tso_omics_plan()

```
if (nzchar(params$omics)) {
  ok_all <- tso_omics_kernel(params$omics, type = params$omics_type, geno = tp)
} else {
  om_sim <- tso_simulate_omics(g, features = 100, seed = 3)      # simulated profiles of
  ↪ your lines
  ok_all <- tso_omics_kernel(om_sim$profiles, geno = tp)
}
ok_all
#> TSOpt omics kernel · OMICS (other, linear) · 300 lines x 100 features
#> Type other (detected)
#> Kernel linear on 100 standardised features · 300 lines
#> Genetics on 300 genotyped lines: median feature heritability 0.38 · agreement of
  ↪ omics and genomic relationships r = 0.43
```

```

#> Feature heritability: quartiles 0.26 / 0.38 / 0.54
#> The profiles are largely genetic: genomics can stand in for lines without a profile.
#> Next: tso_data(geno, kernels = list("A", OMICS = om)) (lines without a profile are
  ↪ filled from genomics)
chk("the omics kernel has mean diagonal 1", abs(mean(diag(ok_all$K)) - 1) < 1e-10)
#> [PASS] the omics kernel has mean diagonal 1
chk("the omics kernel is positive semi-definite", min(eigen(ok_all$K, TRUE, TRUE)$values)
  ↪ > -1e-8)
#> [PASS] the omics kernel is positive semi-definite
prof <- intersect(ok_all$lines, ids)
half <- prof[seq_len(max(3, floor(length(prof) / 2)))]
ok_half <- ok_all; ok_half$K <- ok_all$K[half, half]; ok_half$lines <- half
ok_half$blend <- 0 # no genomic blend (default 5%), so the profiled block is the
  ↪ measured one up to scale
tp_om <- tso_data(g, kernels = ok_half, qc = FALSE, verbose = FALSE)
chk("lines without a profile are kept (predicted from genomics)", nrow(tp_om$K) == N)
#> [PASS] lines without a profile are kept (predicted from genomics)
off <- upper.tri(ok_half$K)
chk("the profiled block keeps the measured relationships",
  stats::cor(tp_om$kernels$K[[ok_half$name]][half, half][off], ok_half$K[off]) > 0.9999)
#> [PASS] the profiled block keeps the measured relationships
pl_om <- tso_design(tp_om, n = min(params$n_train, N - 10), verbose = FALSE)
chk("the omics-aware design reports genetic + omics values",
  any(grepl("genetic \\+ omics", utils::capture.output(print(pl_om)))))
#> [PASS] the omics-aware design reports genetic + omics values
if (length(prof) >= 40) {
  opl <- tso_omics_plan(tp, ok_all, budget = 30 * length(prof), profile_cost = 2, plot_cost
    ↪ = 40,
    n_profile = unique(round(length(prof) * c(0.25, 0.5))), strategy =
      ↪ c("coverage", "targets"))
  opl
  chk("profiling everyone: the realised reliability is the expected one",
    with(opl$table[opl$table$strategy == "all", , drop = FALSE], !length(realised) ||
      ↪ abs(realised - expected) < 1e-10))
} else cat("Fewer than 40 profiled lines: no profiling plan.\n")
#> [PASS] profiling everyone: the realised reliability is the expected one

```

16.8 To the field and the database

```

fb <- tso_export_field(plan_t, format = "fieldbook", trial = "WALK")
chk("the Field Book file has one row per plot", nrow(fb) == nrow(plan_t$selected))
#> [PASS] the Field Book file has one row per plot
chk("FieldHub entries are unique", !anyDuplicated(tso_export_field(plan_t, format =
  ↪ "fieldhub")$NAME))
#> [PASS] FieldHub entries are unique
chk("the plan's manifest matches its data", all(tso_verify(plan_t, tp)$match))
#> [PASS] the plan's manifest matches its data
js <- tempfile(fileext = ".json"); tso_plan_json(plan_t, js)
chk("the plan file reads back", identical(jsonlite::fromJSON(js)$selected$id,
  ↪ plan_t$selected$id))
#> [PASS] the plan file reads back
code <- tso_analysis_script(plan_t, engine = "sommer")
chk("the analysis script parses", !inherits(try(parse(text = code), silent = TRUE),
  ↪ "try-error"))
#> [PASS] the analysis script parses

```

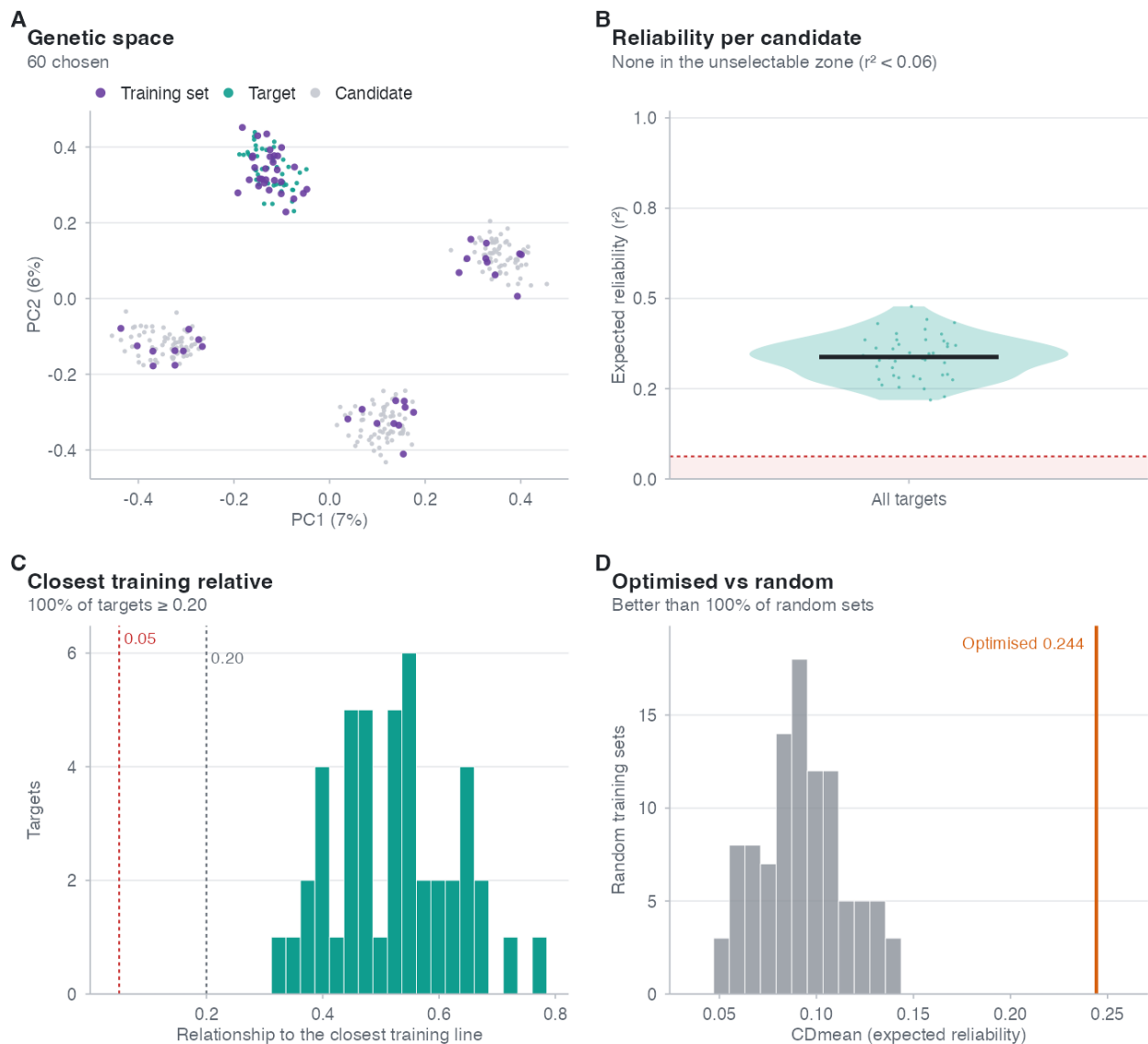
17 Figures, export and reports

17.1 Every plan figure

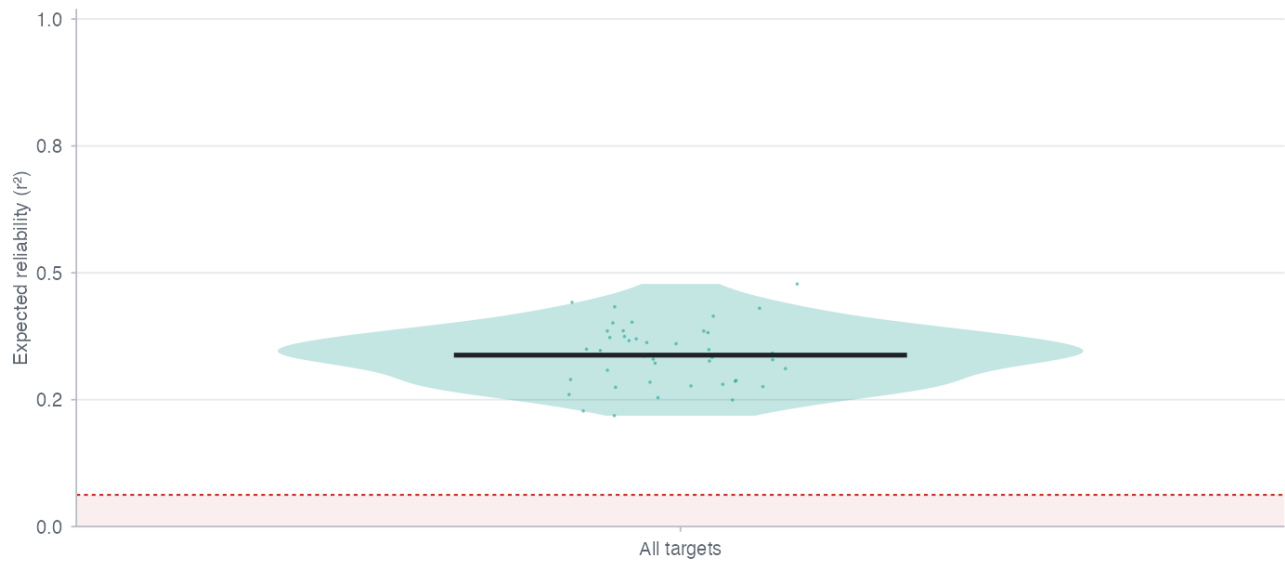
```
types <- c("summary", "pca", "divide", "coverage", "baseline", "search", "kinship")
made <- sapply(types, function(ty) inherits(tso_plot(plan_t, ty), c("ggplot",
  ↪ "patchwork")))
made
#> summary      pca      divide coverage baseline      search      kinship
#>      TRUE      TRUE      TRUE      TRUE      TRUE      TRUE      TRUE
chk("every plan figure type is drawn", all(made))
#> [PASS] every plan figure type is drawn
```

`tso_plot(plan_t)`

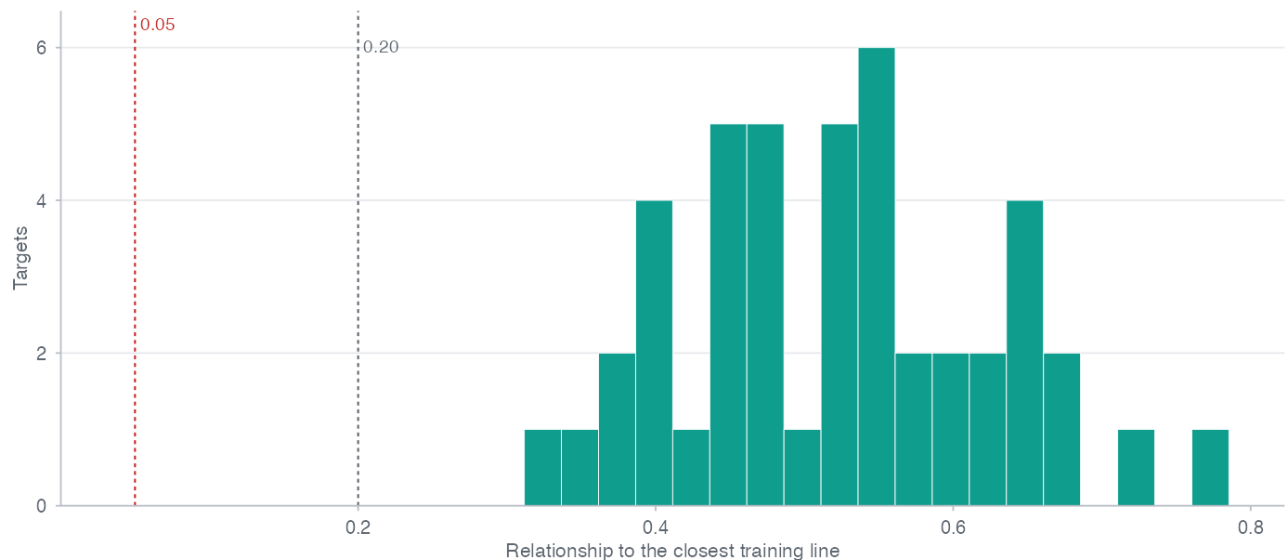
Training-set plan: 60 units · GREEN



`tso_plot(plan_t, "divide")`

Expected reliability per candidateNone in the unselectable zone ($r^2 < 0.06$)

```
tso_plot(plan_t, "coverage")
```

Closest training relative100% of targets ≥ 0.20 · 0 below 0.05

17.2 Journal-ready files, the field list and the report

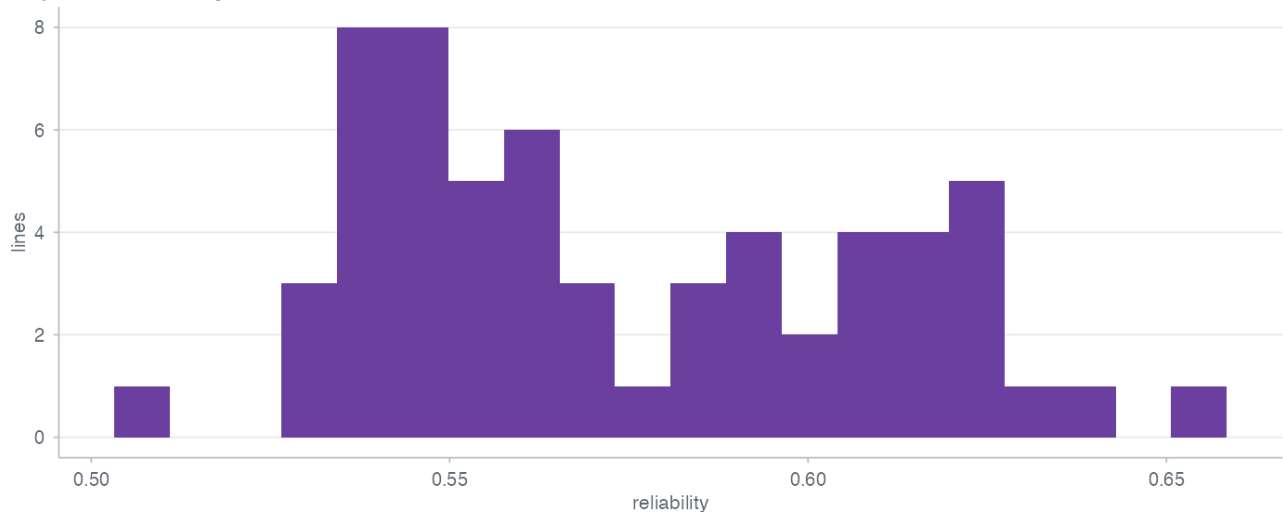
```
f <- tso_save_figure(tso_plot(plan_t, "pca"), file.path(out, "fig_pca"), journal = "g3",
                    width = "single", format = c("pdf", "png", "tiff", "svg"), dpi = 300)
basename(f)
#> [1] "fig_pca.pdf" "fig_pca.png" "fig_pca.tiff" "fig_pca.svg"
chk("all four figure formats were written", all(file.exists(f)))
#> [PASS] all four figure formats were written
chk("unknown journal presets are refused",
    inherits(try(tso_save_figure(tso_plot(plan_t, "pca"), file.path(out, "x"), journal =
        ↪ "nope"),
        silent = TRUE), "try-error"))
```

```
#> [PASS] unknown journal presets are refused
csv <- tso_export(plan_t, file.path(out, "plan.csv"))
chk("the field list has one row per training line", nrow(utils::read.csv(file.path(out,
  ↪ "plan.csv"))) == n_train)
#> [PASS] the field list has one row per training line
rep_file <- tso_report(plan_t, file.path(out, "report.html"), comparison = cmp)
chk("the HTML report is written", file.exists(rep_file) && file.size(rep_file) > 10000)
#> [PASS] the HTML report is written
```

The theme and colours in your own figures:

```
ggplot(plan_t$selected, aes(reliability)) +
  geom_histogram(bins = 20, fill = tso_colors()[["training"]]) + tso_theme() +
  labs(title = "Expected reliability of the selected lines", x = "reliability", y = "lines")
```

Expected reliability of the selected lines



18 Robustness: inputs that should fail clearly

A good tool fails loudly and clearly on impossible requests instead of returning something misleading. Each line below must produce an error or a warning.

```
err <- function(expr) inherits(try(expr, silent = TRUE), "try-error")
wrn <- function(expr) { w <- FALSE; withCallingHandlers(try(expr, silent = TRUE),
  warning = function(x) { w <-< TRUE; invokeRestart("muffleWarning") }); w }
chk("n larger than the pool is refused", err(tso_design(tp, n = N + 1, verbose = FALSE)))
#> [PASS] n larger than the pool is refused
chk("neither n nor budget is refused", err(tso_design(tp, verbose = FALSE)))
#> [PASS] neither n nor budget is refused
chk("unknown target IDs are refused", err(tso_design(tp, n = 10, target = "no_such_line",
  ↪ verbose = FALSE)))
#> [PASS] unknown target IDs are refused
chk("must_include larger than n is refused", err(tso_design(tp, n = 3, must_include =
  ↪ ids[1:6], verbose = FALSE)))
#> [PASS] must_include larger than n is refused
chk("an unknown criterion is refused", err(tso_design(tp, n = 10, criterion = "magic",
  ↪ verbose = FALSE)))
#> [PASS] an unknown criterion is refused
chk("h2 outside (0, 1] is refused", err(tso_design(tp, n = 10, h2 = 1.5, verbose = FALSE)))
#> [PASS] h2 outside (0, 1] is refused
chk("a constant custom criterion is refused",
  err(tso_design(tp, n = 10, criterion = function(i, d) 1, iterations = 10, verbose =
    ↪ FALSE)))
#> [PASS] a constant custom criterion is refused
one_group <- if (!is.null(groups)) ids[tp$units$group == tp$units$group[1]] else NULL
if (length(one_group) > 5)
  chk("CDpop with targets from one group gives a clear error",
    err(tso_design(tp, n = 10, target = one_group[1:5], goal = "families", verbose =
      ↪ FALSE)))
#> [PASS] CDbop with targets from one group gives a clear error
chk("a single target works (no crash)",
  !err(tso_design(tp, n = 10, target = ids[1], verbose = FALSE)))
#> [PASS] a single target works (no crash)
if (!is.null(groups)) {
  lev <- names(table(tp$units$group))
  chk("an unmeetable quota warns instead of silently shrinking",
    wrn(tso_design(tp, n = 3 * length(lev) + 10, group_quota = setNames(rep(3,
      ↪ length(lev)), lev),
      verbose = FALSE)))
}
#> [PASS] an unmeetable quota warns instead of silently shrinking
chk("history and must_include overlap is resolved (no duplicates)",
  !anyDuplicated(tso_design(tp, n = 10, history = ids[1:5], must_include = ids[3:7],
    verbose = FALSE)$index$selected))
#> [PASS] history and must_include overlap is resolved (no duplicates)
```

19 Scorecard

```
summary_tab <- aggregate(result ~ section, checks, function(r) sprintf("%d / %d", sum(r ==
  ↪ "PASS"), length(r)))
names(summary_tab)[2] <- "passed"
knitr::kable(summary_tab, row.names = FALSE)
```

section	passed
1 Upload	13 / 13
10 Major genes	16 / 16
11 Check	5 / 5
12 Evaluate	4 / 4
13 Seasons	7 / 7
13b Decisions around the design	14 / 14
14 Outputs	5 / 5
15 Robustness	11 / 11
2 Quality control	8 / 8
3 Relationship matrices	18 / 18
4 Data object	12 / 12
5 Diagnose and size	4 / 4
6 Criteria	5 / 5
7 Searches	3 / 3
8 Core designs	20 / 20
9 Breeding systems	15 / 15
Programme	24 / 24

```
cat("\nTotal:", sum(checks$result == "PASS"), "of", nrow(checks), "checks passed.\n")
#>
#> Total: 184 of 184 checks passed.
fails <- checks[checks$result == "FAIL", ]
if (nrow(fails)) knitr::kable(fails, row.names = FALSE) else cat("No check failed: nothing
  ↪ looks odd.\n")
#> No check failed: nothing looks odd.
```

Every check, in order:

```
knitr::kable(checks, row.names = FALSE)
```

section	check	result
1 Upload	dosage matrix has lines in rows and markers in columns	PASS
1 Upload	line IDs are unique	PASS
1 Upload	marker IDs are unique	PASS
1 Upload	dosages lie between 0 and the ploidy	PASS
1 Upload	missing rate below 50%	PASS
1 Upload	VCF, HapMap, CSV and PLINK give the same relationship matrix	PASS
1 Upload	DARtseq one-row file is read	PASS
1 Upload	every writable format round-trips to the same kinship	PASS
1 Upload	an Excel sheet is read	PASS
1 Upload	allele pairs, slashes, IUPAC and AB calls give the same dosages	PASS
1 Upload	markers in rows are transposed automatically	PASS
1 Upload	-1/0/1 coding is recognised	PASS
1 Upload	0-4 dosages are read as an autotetraploid	PASS
2 Quality control	QC keeps at least half of the markers	PASS

section	check	result
2 Quality control	no missing values left after imputation	PASS
2 Quality control	every kept marker has MAF ≥ 0.05	PASS
2 Quality control	base allele frequencies are stored, one per marker	PASS
2 Quality control	counted alleles are stored with the base frequencies	PASS
2 Quality control	stricter thresholds keep fewer (or equal) markers	PASS
2 Quality control	re-oriented HapMap dosages equal the VCF dosages	PASS
2 Quality control	and give the same (unscaled) relationship matrix	PASS
3 Relationship matrices	K is symmetric	PASS
3 Relationship matrices	K is positive semi-definite	PASS
3 Relationship matrices	mean diagonal of K is 1 (entry-mean scale)	PASS
3 Relationship matrices	row and column names are the line IDs	PASS
3 Relationship matrices	dominance kernel is symmetric and PSD	PASS
3 Relationship matrices	weighted kinship differs from the plain one	PASS
3 Relationship matrices	weights all equal to 1 reproduce the plain kinship	PASS
3 Relationship matrices	pedigree: full sibs S1, S2 are related 0.5	PASS
3 Relationship matrices	pedigree: half sibs S1, S3 are related 0.25	PASS
3 Relationship matrices	pedigree: G1 (S1 x S3) is inbred, $A = 1.125$	PASS
3 Relationship matrices	every kernel has mean diagonal 1	PASS
3 Relationship matrices	every kernel is positive semi-definite	PASS
3 Relationship matrices	AA is the element-wise square of A	PASS
3 Relationship matrices	epistatic relationships fall off faster than additive ones	PASS
3 Relationship matrices	pedigree dominance: full sibs share 0.25, half sibs 0	PASS
3 Relationship matrices	H covers the ungenotyped crosses	PASS
3 Relationship matrices	a new cross is closer to its parent than to other lines	PASS
3 Relationship matrices	tso_data combines the kernels at the centre shares	PASS
4 Data object	one unit per line	PASS
4 Data object	the kernel covers every unit	PASS
4 Data object	costs attached to every unit	PASS
4 Data object	clonal kernel differs from the additive one	PASS
4 Data object	tetraploid ploidy detected	PASS
4 Data object	families attached	PASS
4 Data object	every female x male cross is a unit	PASS
4 Data object	hybrids of non-crossable parents are not candidates	PASS
4 Data object	genebank object built	PASS
4 Data object	a non-PSD kinship is detected and bent	PASS
4 Data object	units are line x environment cells	PASS

section	check	result
4 Data object	units are line x trait cells	PASS
5 Diagnose and size	diagnosis gives a traffic-light verdict	PASS
5 Diagnose and size	optimised accuracy $\geq$ random at every size	PASS
5 Diagnose and size	n for 95% $\leq$ n for 99%	PASS
5 Diagnose and size	the optimised curve never needs more plots than random	PASS
6 Criteria	every criterion returns n_train lines	PASS
6 Criteria	no criterion puts a target in the training set	PASS
6 Criteria	cdmean (the default) beats random sampling	PASS
6 Criteria	maximising a similarity triggers the wrong-direction warning	PASS
6 Criteria	minimised similarity is below that of random sets	PASS
7 Searches	exchange is at least as good as greedy	PASS
7 Searches	tabu is at least as good as exchange	PASS
7 Searches	restarts never make tabu worse	PASS
8 Core designs	default returns n_train lines	PASS
8 Core designs	default beats every random set	PASS
8 Core designs	robust design reports CD at each heritability	PASS
8 Core designs	targets are never in the training set	PASS
8 Core designs	targeted design predicts the targets better than an untargeted one from the same candidates	PASS
8 Core designs	a single h2 is used as given	PASS
8 Core designs	budget is respected	PASS
8 Core designs	checks are always included	PASS
8 Core designs	excluded lines are never selected	PASS
8 Core designs	history lines are not selected again	PASS
8 Core designs	history lines are never evaluation targets	PASS
8 Core designs	group quotas are met exactly	PASS
8 Core designs	no family exceeds the cap of 3	PASS
8 Core designs	replicated design fills all plots	PASS
8 Core designs	no line gets more than 2 plots	PASS
8 Core designs	every trial gets at least 2 lines	PASS
8 Core designs	a covariate matrix is accepted as a fixed effect	PASS
8 Core designs	the robust design averages over 3 share x 3 heritability scenarios	PASS
8 Core designs	breeding-value CDmean matches the direct formula	PASS
8 Core designs	the plan reports that it predicts breeding values	PASS
9 Breeding systems	every goal returns a full training set	PASS
9 Breeding systems	no parent is used in more than 6 training hybrids	PASS
9 Breeding systems	only crossable parents appear in training	PASS
9 Breeding systems	predict = H0 targets hybrids of two untested parents	PASS
9 Breeding systems	tetraploid clonal design runs	PASS
9 Breeding systems	diploid clonal design runs	PASS
9 Breeding systems	family-mean design (CDpop) runs	PASS
9 Breeding systems	coverage design reaches most lines with a relative ≥ 0.20	PASS
9 Breeding systems	every line is tested in one site only	PASS
9 Breeding systems	every site gets at least 20 lines	PASS
9 Breeding systems	target lines are never tested	PASS
9 Breeding systems	without constraints every site is still opened	PASS
9 Breeding systems	multi-trait budget is respected	PASS
9 Breeding systems	large-panel engine returns 200 lines	PASS
9 Breeding systems	with exact features the large engine picks the same lines as the dense one	PASS
10 Major genes	phenotype IDs match the genotypes	PASS
10 Major genes	the scan finds the planted major QTL	PASS
10 Major genes	haplotype block built around the major QTL	PASS
10 Major genes	marker effects matched to the genotypes	PASS

section	check	result
10 Major genes	each lead's share is a proportion	PASS
10 Major genes	tso_data() protects the loci from the effects	PASS
10 Major genes	GWAS effects find the planted major QTL	PASS
10 Major genes	the planted QTL is recognised as a major locus (share above 20%)	PASS
10 Major genes	every allele class carried by targets has the required carriers	PASS
10 Major genes	the fixed model uses the major loci as fixed effects	PASS
10 Major genes	allele coverage is a proportion	PASS
10 Major genes	LD blocks use every marker exactly once	PASS
10 Major genes	haplotype frequencies sum to 1 in every block	PASS
10 Major genes	single-marker blocks reproduce the VanRaden matrix exactly	PASS
10 Major genes	the haplotype criterion covers at least as many target haplotype copies	PASS
10 Major genes	the per-block report has one row per block	PASS
11 Check	the optimised set scores higher than a random colleague's set	PASS
11 Check	the plan is at least as good as random sampling	PASS
11 Check	top-k analysis reports plan and random-set metrics	PASS
11 Check	the knee set has n_train lines	PASS
11 Check	the front has more than one set	PASS
12 Evaluate	predictive ability is a correlation	PASS
12 Evaluate	evaluation never scores the training lines	PASS
12 Evaluate	every criterion's set can be evaluated	PASS
12 Evaluate	the default cdmean predicts at least as well as the average random set	PASS
13 Seasons	next batch has 25 plots	PASS
13 Seasons	posterior probabilities of selection lie in [0, 1]	PASS
13 Seasons	last season's lines become history, not new plots	PASS
13 Seasons	contributions sum to 1	PASS
13 Seasons	contributions are non-negative	PASS
13 Seasons	the coancestry limit is respected	PASS
13 Seasons	the simulation reports every cycle and strategy	PASS
13b Decisions around the design	line x trait x environment units	PASS
13b Decisions around the design	the multi-trait budget across sites is respected	PASS
13b Decisions around the design	the robust multi-trait design reports every scenario	PASS
13b Decisions around the design	every trait is measured at least twice (trait means are estimable)	PASS
13b Decisions around the design	budget shares sum to 1	PASS
13b Decisions around the design	the reference trait has exchange rate 1	PASS
13b Decisions around the design	plot value equals the brute-force loss of dropping the plot (first 5 plots)	PASS
13b Decisions around the design	the uncertainty interval contains the estimate	PASS
13b Decisions around the design	the economic optimum maximises net value on the curve	PASS
13b Decisions around the design	every training line is planted	PASS
13b Decisions around the design	blocking never raises the CDmean	PASS

section	check	result
13b Decisions around the design	the genotyped set beats random sets on coverage	PASS
13b Decisions around the design	the kernel-robust design reports every kernel	PASS
13b Decisions around the design	transferability is a ratio between 0 and 1	PASS
Programme	REML converged	PASS
Programme	the heritability is a proportion	PASS
Programme	the same estimate as the GBLUP fit	PASS
Programme	robust designs use the estimated heritability grid	PASS
Programme	every test cycle and method is scored	PASS
Programme	predictive abilities are correlations	PASS
Programme	every allocation fits the budget	PASS
Programme	the best allocation has the highest expected gain	PASS
Programme	every plan fits the budget	PASS
Programme	realised reliability equals the expected one when nothing is imputed	PASS
Programme	testcross units are lines x testers	PASS
Programme	the testcross plan predicts GCA	PASS
Programme	the factorised form gives the same design as the dense one	PASS
Programme	the omics kernel has mean diagonal 1	PASS
Programme	the omics kernel is positive semi-definite	PASS
Programme	lines without a profile are kept (predicted from genomics)	PASS
Programme	the profiled block keeps the measured relationships	PASS
Programme	the omics-aware design reports genetic + omics values	PASS
Programme	profiling everyone: the realised reliability is the expected one	PASS
Programme	the Field Book file has one row per plot	PASS
Programme	FieldHub entries are unique	PASS
Programme	the plan's manifest matches its data	PASS
Programme	the plan file reads back	PASS
Programme	the analysis script parses	PASS
14 Outputs	every plan figure type is drawn	PASS
14 Outputs	all four figure formats were written	PASS
14 Outputs	unknown journal presets are refused	PASS
14 Outputs	the field list has one row per training line	PASS
14 Outputs	the HTML report is written	PASS
15 Robustness	n larger than the pool is refused	PASS
15 Robustness	neither n nor budget is refused	PASS
15 Robustness	unknown target IDs are refused	PASS
15 Robustness	must_include larger than n is refused	PASS
15 Robustness	an unknown criterion is refused	PASS
15 Robustness	h2 outside (0, 1] is refused	PASS
15 Robustness	a constant custom criterion is refused	PASS
15 Robustness	CDpop with targets from one group gives a clear error	PASS
15 Robustness	a single target works (no crash)	PASS
15 Robustness	an unmeetable quota warns instead of silently shrinking	PASS
15 Robustness	history and must_include overlap is resolved (no duplicates)	PASS

```

sessionInfo()
#> R version 4.5.3 (2026-03-11)
#> Platform: aarch64-apple-darwin20
#> Running under: macOS Tahoe 26.4.1
#>

```

```

#> Matrix products: default
#> BLAS:
  ↳ /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRblas.0.dylib
#> LAPACK:
  ↳ /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib;
  ↳ LAPACK version 3.12.1
#>
#> locale:
#> [1] C.UTF-8/C.UTF-8/C.UTF-8/C/C.UTF-8/C.UTF-8
#>
#> time zone: Asia/Kolkata
#> tzcode source: internal
#>
#> attached base packages:
#> [1] stats      graphics  grDevices  utils      datasets  methods   base
#>
#> other attached packages:
#> [1] ggplot2_4.0.3 TSOpt_0.5.0
#>
#> loaded via a namespace (and not attached):
#> [1] Matrix_1.7-6      jsonlite_2.0.0      gtable_0.3.6        dplyr_1.2.1
#> [5] compiler_4.5.3    tidyselect_1.2.1    cluster_2.1.8.3     splines_4.5.3
#> [9] systemfonts_1.3.2 scales_1.4.0         textshaping_1.0.5   yaml_2.3.12
#> [13] fastmap_1.2.0     readxl_1.5.0        lattice_0.22-9      R6_2.6.1
#> [17] labeling_0.4.3    generics_0.1.4      patchwork_1.3.2     knitr_1.51
#> [21] tibble_3.3.1      svglite_2.2.2       pillar_1.11.1       RColorBrewer_1.1-3
#> [25] rlang_1.3.0       xfun_0.60           S7_0.2.2            otel_0.2.0
#> [29] viridisLite_0.4.3 cli_3.6.6           mgcv_1.9-4          withr_3.0.3
#> [33] magrittr_2.0.5    digest_0.6.39       grid_4.5.3          nlme_3.1-170
#> [37] lifecycle_1.0.5   vctrs_0.7.3         writexl_2.0.0       evaluate_1.0.5
#> [41] glue_1.8.1        data.table_1.18.6.1 farver_2.1.2        cellranger_1.1.0
#> [45] ragg_1.5.2        rmarkdown_2.31      tools_4.5.3         pkgconfig_2.0.3
#> [49] htmltools_0.5.9

```